

Università degli Studi di Milano-Bicocca
Department of Informatics, Systems and Communication
Master's Degree in Computer Science



Università della Svizzera italiana
Faculty of Informatics
Master of Science in Informatics

Double Degree Program

On Using Wave Function Collapse for Depicting Software Systems

USI Advisor: Prof. Dr. Michele Lanza

USI Co-advisor: Dr. Marco Raglianti

UniMiB Co-advisor: Prof. Dr. Claudia Raibulet

Thesis by:
Simone Lesinigo
ID: 899540

Academic Year 2025–2026

Abstract

Grasping the overall architecture of large software systems is a challenging task. A plethora of software visualization approaches has been proposed to ease the comprehension of relationships among components and classes in such systems. However, the effectiveness of these visualizations depends on how elements are placed and connected. Poor layouts can lead to excessive overlaps and edge crossings, making the resulting visualizations cluttered and difficult to understand.

This thesis presents a novel approach based on Wave Function Collapse (WFC), a procedural generation algorithm inspired by quantum mechanics. WFC progressively assigns tiles to cells in a grid according to local compatibility constraints, to connect (software) entities while discouraging overlaps and preserving the connectivity imposed by global constraints. We implement the approach in Wassis, a tool developed in Pharo, and investigate how design choices affect both the generation process and the resulting visualization.

We show how WFC can be tuned to reduce the number of path crossings and explore the challenges of scaling it to represent real-world software systems. Through single- and multi-relationship examples, we evaluate the characteristics of Wassis that produce structured, "organic" visualizations, discussing their potential and current limitations.

Acknowledgments

First of all, I would like to thank Prof. Dr. Michele Lanza, head of the REVEAL group, and Dr. Marco Raglianti for giving me the opportunity to work on this thesis and for their support and guidance throughout this journey. I am also grateful to Prof. Dr. Claudia Raibulet for her role as my co-advisor at the Università degli Studi di Milano-Bicocca.

I would also like to thank the other members of the REVEAL group, Carmen, Mattia, and Phillip, for their ideas, discussions, and help with my work. Special thanks go to Luca and Francesco, fellow Master's students in the REVEAL group, for sharing this experience with me.

I am particularly grateful to Prof. Dr. Alberto Dennunzio, who encouraged me to pursue a Master's degree and continue my studies after completing my Bachelor's degree.

My thanks go to my parents, Mario and Maria, for always supporting me throughout my studies and making it possible for me to take part in the Double Degree program.

I would like to express my gratitude to my girlfriend, Angela, for being by my side throughout this entire journey. Her constant support, patience, and encouragement have meant a great deal to me, especially during the most challenging moments of my studies. I am grateful to have shared this experience with her and to have always been able to count on her.

I am also grateful to my closest friends, Federico, Vito, Mattia, Manuel, and Alessandro. We have been friends since high school, and I am glad we have stayed close and continued to share our lives as we grew up.

I would also like to thank Luca and Andrea, my former Bachelor's classmates, with whom I have stayed in touch over the years. Special thanks go to Andrea for all the projects we worked on together during the first year of my Master's degree.

Finally, I would like to thank my fellow Double Degree students, Luca, Manuel, Alessandro, Michele, Amedeo, and Marco, for sharing this adventure with me. Working on projects and spending many memorable evenings together made this year at USI truly special.

Contents

1 Introduction **1**
- 1.1 Context and Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Proposed Approach . . . 2
- 1.4 Contributions . . . 3
- 1.5 Document Structure . . . 4

2 Software Visualization **5**
- 2.1 Graph-Based Visualizations . . . 5
  - 2.1.1 Force-Directed Layout . . . 5
  - 2.1.2 Orthogonal Layouts . . . 5
  - 2.1.3 Edge Bundling . . . 5
- 2.2 Polymetric Views . . . 6
- 2.3 3D and VR Visualizations . . . 7
- 2.4 Cartographic Visualization . . . 9
- 2.5 Summary . . . 10

3 Wave Function Collapse **11**
- 3.1 Wave Function Collapse Algorithm . . . 11
- 3.2 WFC Extensions and Applications . . . 14
- 3.3 Summary . . . 16

4 From Code to Visualization **17**
- 4.1 Initialization . . . 17
- 4.2 Layout via MDS . . . 19
  - 4.2.1 Multi-cell Classes . . . 19
  - 4.2.2 Distance Matrix . . . 19
  - 4.2.3 Target Position on the Grid . . . 19
  - 4.2.4 Class Placement Strategy . . . 20
- 4.3 Relationship Representation . . . 22
- 4.4 Path Generation via WFC . . . 23
  - 4.4.1 Observation Phase . . . 23
  - 4.4.2 Propagation Phase . . . 26
- 4.5 Computed Metrics . . . 27
- 4.6 Challenges and Enhancements . . . 28
  - 4.6.1 Managing the Exploding Number of Tiles . . . 28
  - 4.6.2 Guiding Paths Towards Their Targets . . . 29
  - 4.6.3 Reducing the Number of Crossings . . . 30
  - 4.6.4 Backtracking Strategies . . . 30
  - 4.6.5 Path Optimization . . . 31
- 4.7 Summary . . . 32

5 Implementation **33**
5.1 Development Environment . . . 33
5.2 Software Architecture . . . 33
5.3 Code Parsing . . . 34
5.4 Class Layout and Placement . . . 34
5.4.1 Distance Matrix and MDS . . . 35
5.4.2 Mapping Positions to the Grid . . . 35
5.4.3 Class Placement . . . 35
5.4.4 Path Exit Assignment . . . 37
5.5 Wave Function Collapse . . . 40
5.5.1 Heap Management . . . 40
5.5.2 Dynamic Placeholder Resolution . . . 43
5.5.3 Dynamic Tile Weights . . . 43
5.5.4 Constraint Propagation . . . 44
5.5.5 Backtracking . . . 44
5.6 Path Optimization . . . 45
5.7 Memory Management . . . 45
5.8 Parallel Execution . . . 45
5.9 Tool . . . 46
5.9.1 Class Selection and Configuration Setup . . . 46
5.9.2 Visualization and Additional Views . . . 47
5.9.3 Step-by-Step Animation . . . 48
5.9.4 Metrics Dashboard . . . 48
5.10 Summary . . . 49

6 Experiments and Results **51**
6.1 Class Hierarchy: Wassis vs. PlantUML . . . 51
6.1.1 Layout . . . 51
6.1.2 Path Representation . . . 53
6.1.3 Class Identification . . . 53
6.1.4 Execution Time . . . 53
6.2 Multi-Relationship: Wassis vs. PlantUML . . . 54
6.2.1 Path Crossings and Visual Clutter . . . 54
6.2.2 Execution Statistics . . . 55
6.2.3 Summary . . . 55
6.3 Minimum Distance Between Classes Comparison . . . 55
6.4 Impact of Path Boost . . . 57
6.5 Min-Heap Behavior . . . 58
6.5.1 How Min-Heap Can Guide Paths . . . 58
6.5.2 Example: Slot Hierarchy . . . 59
6.6 Evaluation of Backtracking Strategies . . . 59
6.6.1 Experimental Results and Analysis . . . 60
6.7 WFC Generation Analysis . . . 61
6.7.1 PCP versus Execution Time . . . 63
6.8 Summary . . . 64

7 Conclusion **65**
7.1 Limitations . . . 66
7.2 Future Work . . . 66
7.2.1 Performance and Scalability . . . 66
7.2.2 WFC and Layout Improvements . . . 67
7.2.3 Evaluation . . . 67
7.3 Closing Words . . . 67

Bibliography **70**

List of Figures

1.1 WFC-pure, 7 classes, 3 relationship types, no path optimization. 3

2.1 Example of Edge Bundling. 6
2.2 Example of a Polymetric View. 6
2.3 Example of a Class Blueprint V2. 7
2.4 The city of Azureus and its package topology. 8
2.5 Relationships in a street-layout city. 8
2.6 ISA's VR application. 9
2.7 Example of a Software Map. 10

3.1 Example of a tile-based model. 12
3.2 Examples of the overlapping model. 12
3.3 Example of an iteration step of the main WFC loop. 13
3.4 Examples of WFC applications in game development. 14
3.5 Graph-based WFC applications. 15

4.1 General flow of the approach. 17
4.2 Initial possible tiles. 18
4.3 Example of compatible tiles for each direction. 18
4.4 Example of target positions calculated by MDS (normalized and scaled to the grid). 20
4.5 Candidate search example (minimum distance: 2). 21
4.6 Conceptual representation of path merging. 22
4.7 Comparison of min-heap and max-heap collapse order. 23
4.8 Example of unnecessary paths generated by WFC. 24
4.9 Initial class placement on the grid, with fixed path entry and exit points. 25
4.10 Multiple paths remaining open during the WFC generation process. 25
4.11 Animation of the execution of WFC. 26
4.12 Visualization after WFC execution. 27
4.13 Resolution of path ID placeholders during generation. 29
4.14 Backtracking workflow. 31

5.1 Core Architecture of Wassis. 34
5.2 Distance matrix before and after Floyd-Warshall. 36
5.3 Example of preferred direction of a path ID calculation. 38
5.4 Workflow of the border cell assignment process. 39
5.5 Example of the border cell assignment process. 40
5.6 Next cell selection workflow. 41
5.7 Lazy heap entry example. 42
5.8 Example of tile weight boost. 44
5.9 Wassis visualization configuration. 46
5.10 Wassis configuration settings window. 47
5.11 Main grid visualization. 48
5.12 WFC step-by-step animation window. 49
5.13 Overview of the tool metrics panels. 50

6.1 Comparison of the `Magnitude` class hierarchy in PlantUML (a) and Wassis (b). 52
6.2 Comparison of Wassis and PlantUML for multi-relationship visualization. . . . 54
6.3 Subset of 9 classes from the `org.jetuml.diagram` package. 55
6.4 Execution times for different minimum class distances. 56
6.5 Execution times grouped by minimum class distance (with path boost). 57
6.6 `Slot` hierarchy generated with path boost 50 and minimum distance 2 (37.43s). . . 58
6.7 Min-heap generation process. 59
6.8 `HashedCollection` full hierarchy. 60
6.9 Results for the five backtracking strategies. 61
6.10 PCP distribution for 193 instances. 62
6.11 Average node counts and path space occupancy for different PCP values. 62
6.12 Examples of solutions with low and high PCP values. 63
6.13 PCP over execution time for 193 WFC instances. 64

List of Tables

6.1 Comparison between Wassis and PlantUML for the `Magnitude` hierarchy. 53
6.2 Wassis execution metrics. 55
6.3 Comparison of Wassis and PlantUML. 55
6.4 Results obtained with and without path boost for each minimum class distance. . . 57

Chapter 1

Introduction

As Thomas Ball stated:

> "Software is intangible, having no physical shape or size." [1]

Unlike concrete objects, software has no shape that we can directly perceive. This raises a fundamental question: *how can we represent something that has no physical form*? Software visualization addresses this question by mapping abstract software entities, such as classes, and their relationships onto visual representations that developers can explore. This thesis investigates how to place software entities and how to connect them, using Wave Function Collapse (WFC) [34], a procedural content generation algorithm.

1.1 Context and Motivation

Program comprehension is older than software engineering itself: the term *software engineering* was coined at the 1968 NATO conference in Garmisch [2], but programmers had been reading and trying to understand code long before.

As software systems evolve and grow in complexity, developers spend a significant portion of their time reading, navigating, and mentally reconstructing code structures rather than writing new features. Understanding a codebase requires developers to build and maintain a mental model of the software, combining their existing programming and domain knowledge with information obtained from source code and related documentation [3, 4]. This process is not necessarily linear: developers may start from high-level goals and previous knowledge and move towards specific implementation details, or instead build their understanding bottom-up by gradually grouping low-level code structures into higher-level concepts. During this process, they continuously formulate hypotheses about the purpose and behavior of program components, check them against the available information, and update their mental model as their understanding improves.

Despite decades of improvements in integrated development environments (IDEs), new programming paradigms, and new programming languages, the amount of time developers spend trying to understand source code has remained surprisingly stable for over thirty years [5]. In her review of program comprehension research, Siegmund discusses the models developed to explain how developers understand source code and highlights the renewed interest in human factors in software engineering, as well as the need for stronger empirical evidence on how developers perform their everyday tasks [5]. Studies using techniques such as eye-tracking, electroencephalography (EEG), and functional magnetic resonance imaging (fMRI) further investigate the cognitive processes involved in code comprehension, showing that reading source code involves both language-processing and higher-level cognitive areas of the brain.

Empirical studies also highlight the practical consequences of this effort. LaToza *et al.* [6] found that developers invest substantial effort in creating and maintaining mental models of software systems, while much of the rationale behind existing code remains undocumented. When source code and documentation do not provide enough information, developers often turn to their colleagues to understand past design decisions, historical changes, or temporary workarounds. These interactions can interrupt both their own work and that of their coworkers, reducing overall productivity.

Similarly, Minelli *et al.* [3] found that program comprehension accounts for roughly 70% of developers' time, highlighting how much of software development involves understanding existing code rather than writing new code.

These challenges motivate the use of visual representations to support program comprehension. Visual modeling languages (*e.g.,* UML [7]) and software visualizations [8, 9, 10] represent software structures and relationships in ways that developers can explore without relying exclusively on source code. By making these structures more explicit, visualizations can help developers understand how software entities are organized and connected. However, producing useful visualizations of large software systems is itself challenging: entities must be positioned meaningfully, and their relationships must be represented without introducing excessive visual clutter.

1.2 Problem Statement

Applying software visualization to real-world, large-scale systems introduces two fundamental and interconnected challenges:

- **The Layout Problem:** Given a set of software entities, one must decide how to position them meaningfully in the visual representation. Different layouts can highlight different structural or semantic aspects of the software, such as inheritance relationships or package structure. A poor layout can hide important patterns and make the overall structure of the system harder to understand.
- **The Connectivity Problem:** While displaying relationships (*e.g.,* inheritance, method calls, and dependencies) enriches the information about the system, large software graphs often become excessively dense. As the number of entities increases, connections scale rapidly, leading to visual clutter, edge crossings, and unintelligible diagrams.

1.3 Proposed Approach

In this thesis, we explore a novel approach designed to tackle both problems. The approach is implemented in Wassis, a tool developed in Pharo that combines Multidimensional Scaling (MDS) for class placement with Wave Function Collapse for path generation. This combination aims to produce layouts that are both structurally informative and easy to interpret.

First, we use Multidimensional Scaling [11, 12], a dimensionality reduction technique, to establish the placement of software entities on the canvas according to their structural relationships, aiming to position related entities closer to each other.

Second, to address the connectivity problem, we leverage WFC [34], a procedural generation algorithm inspired by quantum mechanics. WFC generates coherent tile-based structures by propagating *local constraints* across a *grid* of *cells*. In our approach, the tiles represent small segments of the paths connecting software entities. WFC generates these paths by selecting a tile for each cell and propagating the constraints imposed by neighboring tiles. By combining a sequence of compatible tiles, complete paths can be generated while respecting the required connections between software entities. Since constraints are defined at the *tile* level (*i.e.,* specifying which elements can be placed in adjacent cells), WFC provides fine-grained control over the generated connections. This allows us to reduce visual clutter, such as overlapping lines, by restricting the set of possible tiles and adjusting their probabilities.

Furthermore, WFC is intrinsically non-deterministic. For a given software system, it can generate multiple different yet valid layouts, offering developers distinct perspectives of the same system. However, dependency relationships are inherently global constraints, while WFC operates locally. We devised our approach to generate valid and consistent paths (also good but not necessarily optimal without a cleanup step), as exemplified in Figure 1.1, while ensuring that the generated connections reflect the actual relationships in the system.

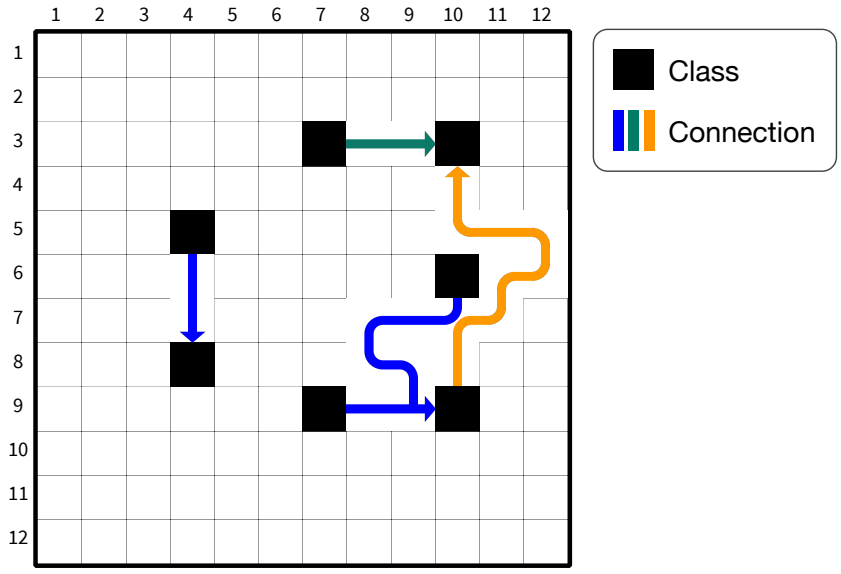


Figure 1.1: WFC-pure, 7 classes, 3 relationship types, no path optimization.

1.4 Contributions

The primary goal of this thesis is to investigate how Wave Function Collapse can be adapted to generate software visualizations. The main contributions of this work are:

- **Combining MDS with WFC:** We combine Multidimensional Scaling to position software classes according to their structural relationships with Wave Function Collapse to generate paths connecting the intended classes. This separates class placement from path generation while integrating both into a single visualization pipeline.
- **A method for imposing global constraints using WFC's local constraints:** We show how global connectivity constraints can be imposed using only the local constraints of WFC. This allows the algorithm to generate paths that connect only the intended classes, preventing connections between unrelated classes and preserving the intended dependency relationships throughout the visualization.
- **Adapting WFC to software visualization:** We present several extensions to adapt WFC to software visualization. These include multi-cell classes, a maximum-entropy heuristic, directional path exits, dynamic tile weights, crossing avoidance, and backtracking strategies. Together, these changes make WFC suitable for the layout and the connectivity problems.
- **Tool implementation:** We implement the proposed approach in Wassis, a tool developed in Pharo. Wassis provides the complete pipeline from source code analysis to class placement, WFC path generation, optional path optimization, and visualization. It also includes mechanisms for backtracking, memory management, and parallel execution to support and optimize the generation process.

Part of the research presented in this thesis has been peer-reviewed and accepted for publication.

Title	Toward Using Wave Function Collapse to Depict Software Systems
Authors	Simone Lesinigo, Marco Raglianti, Michele Lanza
Conference	14th Working Conference on Software Visualization (VISSOFT) - NIER Track
Publisher	IEEE
Year	2026
Pages	111–115

1.5 Document Structure

The thesis is organized as follows:

- **Chapter 2 - Software Visualization**. This chapter reviews software visualization techniques, including graph-based layouts, polymetric views, 3D and VR visualizations, and cartographic approaches. It discusses their main characteristics and limitations.
- **Chapter 3 - Wave Function Collapse**. This chapter explains the WFC algorithm, its main variants, and its applications. It also reviews extensions relevant to this thesis.
- **Chapter 4 - From Code to Visualization**. This chapter presents the proposed approach, from the spatial layout of software systems using Multidimensional Scaling to relationship merging and path generation using WFC. It also describes the main improvements introduced to handle larger systems.
- **Chapter 5 - Implementation**. This chapter describes the implementation of Wassis in Pharo. It covers code parsing, class placement, path exit assignment, max-heap optimization, dynamic tile weights, backtracking, path optimization and parallel execution.
- **Chapter 6 - Experiments and Results**. This chapter evaluates Wassis on selected software systems. It compares the generated visualizations and investigates how different configurations affect the ability to find valid solutions and the time required for generation.
- **Chapter 7 - Conclusion**. This chapter summarizes the main findings and contributions of this work. It also discusses the current limitations and possible future work.

Appendix A provides the mathematical derivation of the unnormalized Shannon entropy used during the WFC observation phase.

Chapter 2

Software Visualization

Software visualization helps developers understand software systems by representing their entities and relationships. Different techniques have been proposed to organize entities and display their connections. In this chapter, we review the main approaches and their limitations, identifying the challenges addressed by this thesis.

2.1 Graph-Based Visualizations

Software systems can naturally be represented as graphs [13], where software entities (such as classes or packages) are represented as nodes, and their relationships (such as dependencies or method calls) are represented as edges. To enhance graph readability, various layout strategies have been proposed.

2.1.1 Force-Directed Layout

Fruchterman and Reingold [14] introduced a widely used layout algorithm based on a physical analogy. The algorithm is designed for undirected graphs and aims to distribute nodes evenly on the frame while keeping connected nodes close and reducing edge crossings. Nodes act as particles that repel each other, while edges act as springs that attract connected nodes. Starting from an initial node placement (that can be random), these forces move the nodes until the system reaches a stable state. The resulting layout should follow two main principles: connected nodes should be placed near each other, but nodes should not be placed too close to each other.

2.1.2 Orthogonal Layouts

Tamassia [15] introduced an approach for orthogonal graph drawing in which edges follow horizontal and vertical lines on a rectilinear grid. The method aims to minimize the total number of bends while preserving the original planar structure of the graph. To achieve this, the problem is transformed into a minimum-cost problem, where the cost of the flow corresponds to the number of bends. The algorithm can then compute an orthogonal representation with the minimum number of bends in $O(n^2 \log n)$ time. This approach is useful for automatic layout of graph-based diagrams, where horizontal and vertical segments can provide a clear and structured representation.

2.1.3 Edge Bundling

Holten [16] introduced hierarchical edge bundling to reduce visual clutter in compound graphs. Instead of drawing relationships as straight lines, edges are curved along paths between nodes. This technique groups edges into bundles, clarifying connections between groups. Bundling strength is adjustable: lower values ($\beta = 0$) make individual connections easier to follow, while higher values ($\beta = 1$) highlight relationships between groups of nodes. An example is shown in Figure 2.1.

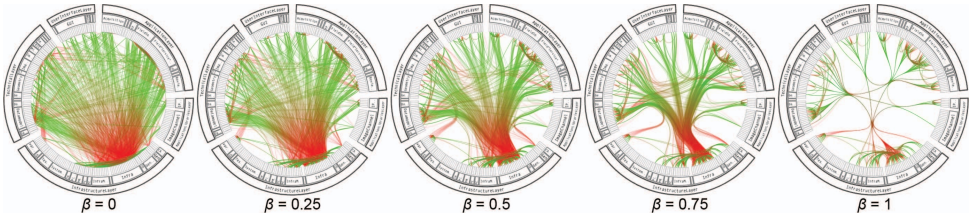


Figure 2.1: Example of Edge Bundling [16].

2.2 Polymetric Views

In 2003, Lanza and Ducasse introduced *polymetric views*, a lightweight software visualization technique for reverse engineering and program comprehension [17]. It is based on two-dimensional graphs, where nodes represent software entities, such as classes or methods, and edges represent their relationships. A polymetric view adds information to these graphs by mapping up to five software metrics to visual properties of each node: width, height, position (X, Y), and color (using a grayscale from white to black). These metrics are directly derived from the source code rather than being computed by combining other metrics. For example, they can describe different aspects of a class, method, or attribute. Class metrics include the number of methods (NOM), the number of attributes (NOA), and the number of immediate subclasses (NOC). Method metrics include lines of code (LOC), the number of parameters (NOP), and the number of statements (NOS). Attribute metrics include the number of direct accesses from outside the class (NGA) and from within the class (NLA). The authors defined several predefined views at different levels of detail, while also allowing developers to create custom views. This approach helps developers assess the overall structure and quality of a system, locate important components, and identify unusual entities that may need refactoring or further analysis. An example of a polymetric view is shown in Figure 2.2.

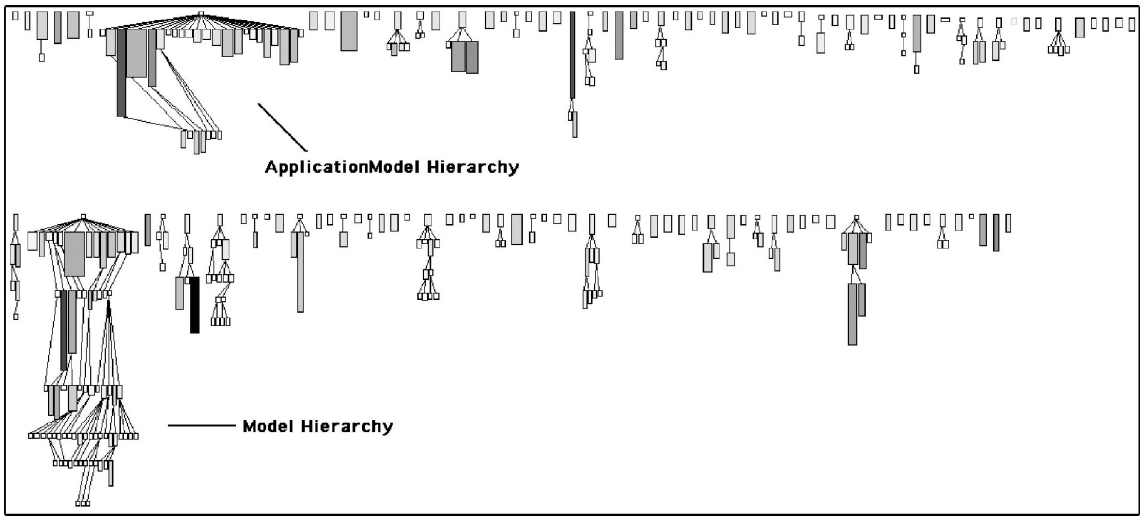


Figure 2.2: Example of a Polymetric View [17].

In 2022, Agouf *et al.* [8] introduced *Class Blueprint V2*, an evolution of the original Class Blueprint polymetric view [17]. The original Class Blueprint was designed to help developers understand the structure of a class, but it had some limitations. It did not show whether a method was covered by tests, did not identify dead code, did not clearly distinguish between class-side and instance-side methods, and did not show cyclomatic complexity. Class Blueprint V2 addresses these limitations by enhancing the original layout. Horizontally, the visualization is divided into three layers: the class side at the top, the instance side in the middle, and a bottom layer, called the "cemetery", for dead methods and unused attributes. Vertically, instance methods are organized into different groups. These include methods responsible for object creation, methods that compose the external interface of the class, and methods used internally by the class itself.

Getter and setter methods are also placed close to the attributes they access to reduce the space usage. The visualization also uses colors and visual annotations to provide additional information. For example, tested methods have their upper third colored in green, attributes accessed by at least one subclass are shown in dark blue, while other attributes are shown in dark green. Different colors are also used for methods, such as brown for overriding methods and pale orange for overridden methods. Methods with high cyclomatic complexity are highlighted with a red border. These visual elements allow developers to quickly identify potentially problematic methods and attributes. In particular, the detection of dead code was the most appreciated feature, as it can reveal code that developers forgot to remove. An example of Class Blueprint V2 is shown in Figure 2.3.

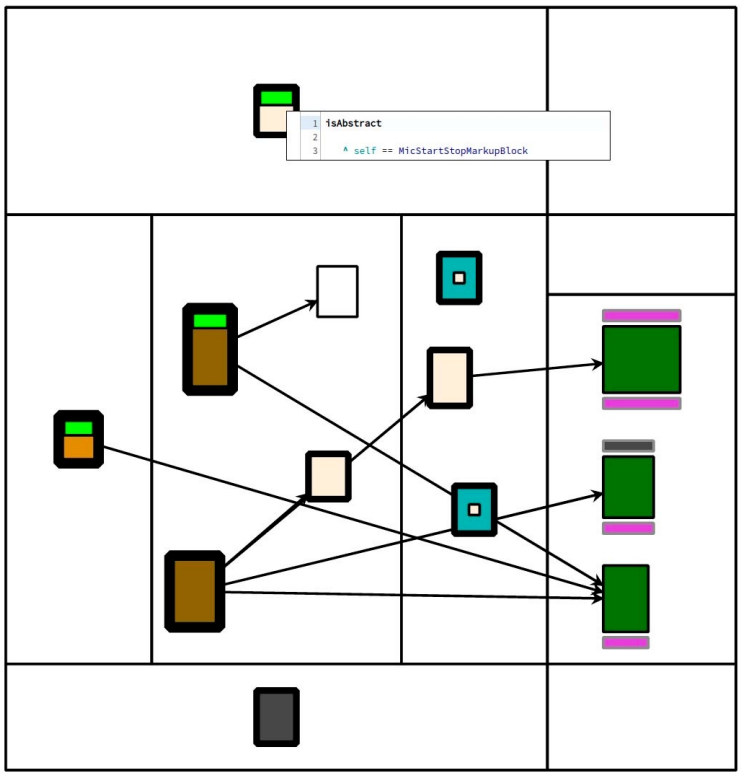


Figure 2.3: Example of a Class Blueprint V2 [8].

2.3 3D and VR Visualizations

Three-dimensional and immersive visualizations provide alternative ways to represent software structure and explore relationships between software entities. Existing approaches differ in how they organize entities in space and depict their relationships, using techniques such as hierarchical layouts, edge bundling, and semantic clustering. These approaches illustrate different ways to organize software entities and represent their relationships in three-dimensional environments.

In 2007, Wettel and Lanza [9] introduced CodeCity, a 3D software visualization tool. Three-dimensional software visualizations can make navigation difficult, as moving freely through a 3D space can lead to disorientation. CodeCity addresses this problem by representing a software system as a city, providing a more familiar context. Classes are represented as buildings, whose visual properties reflect software metrics. In particular, building height represents the Number of Methods (NOM), while the base size represents the Number of Attributes (NOA). Classes are grouped into districts representing packages, which can be nested to reflect the package structure. Nested packages are represented as platforms at different heights, making the package topology visible. The buildings are arranged using a modified treemap algorithm based on rectangle packing. An example is shown in Figure 2.4.

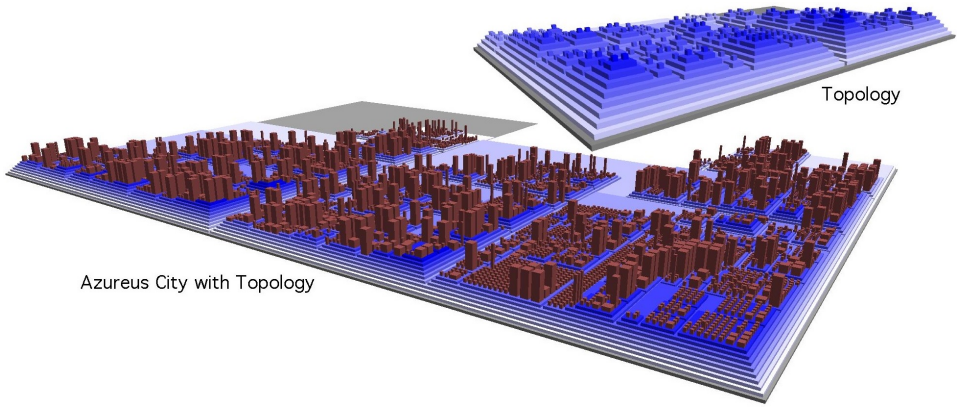


Figure 2.4: The city of Azureus and its package topology [9].

Caserta *et al.* [18] extended the city metaphor to show relationships between software classes. They used the Hierarchical Edge Bundling technique proposed by Holten [16] (already presented in Section 2.1) and adapted it to 3D, creating *3D-HEB* (*3D Hierarchical Edge Bundles*). The technique bundles the edges between buildings in a software city, reducing the visual clutter that would result from drawing direct edges between individual buildings. As in the original technique, the parameter β controls the strength of the bundling. The authors also use color to encode the direction and magnitude of the relationships. To evaluate the technique in different settings, they applied it to two types of software city layouts: nested and street layouts. This shows that 3D edge bundling can be used with different spatial organizations of the software city. An example is provided in Figure 2.5.

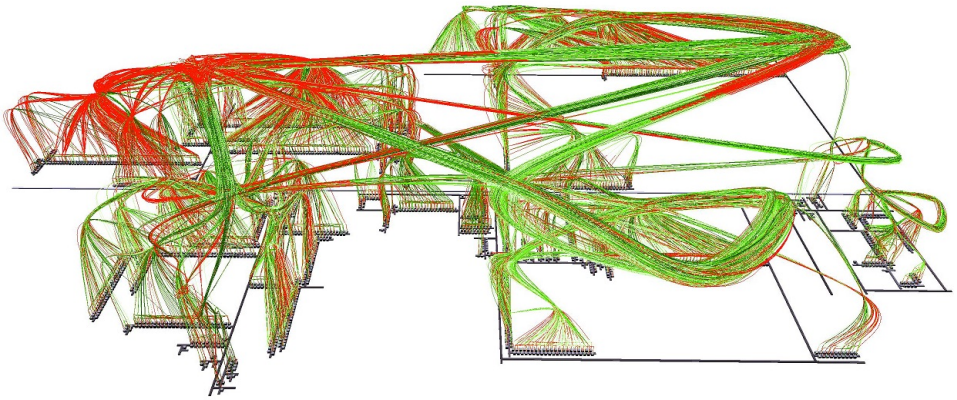


Figure 2.5: Relationships in a street-layout city [18].

More recently, Moreno-Lumbreras *et al.* [19] developed a web version of CodeCity, called BabiaXR-CodeCity, which supports both traditional screens and Virtual Reality (VR). Unlike the original CodeCity, the new implementation uses a spiral layout instead of a treemap. This choice makes it easier to represent software evolution, since new elements can be added without moving the existing ones. The authors compared the two visualization modes through two experiments, involving 24 and 26 participants from academia and industry. The participants were asked to perform some tasks using the same visualization either on-screen or in VR. The results showed that VR participants completed the tasks significantly faster, sometimes in about half the time required by on-screen participants. At the same time, their correctness was comparable to that of on-screen participants, and in the second experiment it was even higher for several tasks. The authors suggest that the faster results in VR may be related to its more natural spatial interaction, such as moving the head or body to inspect the visualization, compared with navigating a 3D scene using a keyboard and mouse.

Unlike CodeCity and BabiaXR-CodeCity, which mainly use physical structures such as packages to organize software entities, Hoff *et al.* [20] proposed ISA (*Immersive Software Archaeology*), a VR tool that organizes software according to its logical architecture. ISA first uses automated clustering to identify groups of classes that form cohesive architectural components. These components are represented at different levels of abstraction, which users can navigate from the architectural level down to the design level and back using semantic zoom. At the highest level, architectural components are represented as planets in a solar system. As users zoom in, these components are divided into continents and cities, with buildings representing classes. At the class level, the floors of each building encode method metrics, such as cognitive complexity. An example of the ISA interface is shown in Figure 2.6. ISA also provides interactive 3D relationship graphs that allow users to explore dependencies between software entities. A controlled experiment with 54 participants showed that grouping classes according to logical architectural components, rather than only their physical structure, improved information retrieval and helped participants build better mental models of unfamiliar software systems.

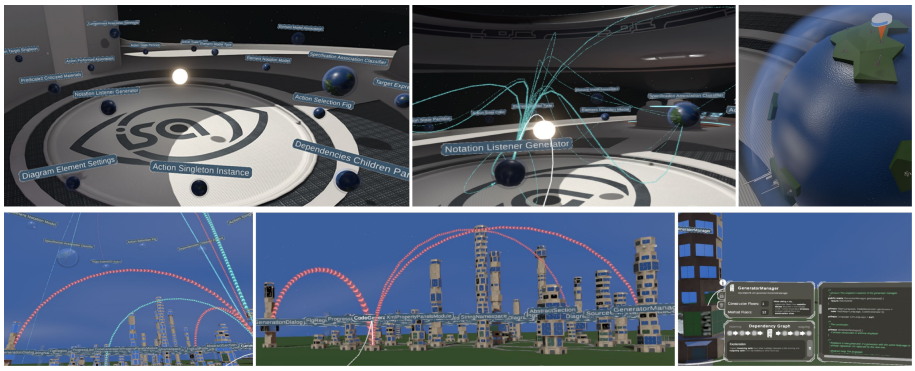


Figure 2.6: ISA's VR application [20].

While VR can make software visualization easier to explore, configuring the visualization itself can still be difficult. Traditional tools often use external 2D interfaces or Domain-Specific Languages (DSLs), forcing users to leave the VR environment or interact with 2D menus inside the virtual space. To address this problem, Giannaccari *et al.* [21] introduced IVAR-NI, a tool that allows users to customize visualizations directly in VR. Instead of using traditional menus, IVAR-NI represents meta-entities in the system as 3D shapes. Users can interact with these objects using VR controllers to configure properties such as layout, size, and color, to define the desired visualization. The tool also provides an interactive preview of the visualization, allowing users to immediately see the effect of their changes.

2.4 Cartographic Visualization

Kuhn *et al.* [10] introduced *Software Cartography*, an approach that represents software systems as maps. Its main goal is to place software entities according to their semantic similarity. Entities with similar vocabulary are placed close to each other, while less similar entities are placed farther apart. This also provides a consistent layout that remains stable as the software evolves, unlike layouts based on physical structures such as package hierarchies.

To create the map, Software Cartography uses source code vocabulary to measure similarity. Source files are represented as term-frequency histograms and processed with Latent Semantic Indexing (LSI). MDS then maps the high-dimensional representation to a two-dimensional space. As a result, semantically similar entities are placed close to each other.

The map can then be enriched with cartographic elements to represent additional software properties. For example, the landscape's height is represented by the number of thousands of lines of code (KLOC), and hill shading and contour lines are then added to make the landscape more visually appealing. A software map generated via this approach appears in Figure 2.7.



Figure 2.7: Example of a Software Map [10].

2.5 Summary

In this chapter, we reviewed different techniques for software visualization and discussed how they represent software entities and their relationships.

State of the Art. Existing techniques offer different ways to organize and visualize software systems. Graph-based layouts focus on positioning entities and drawing connections [13, 14, 15, 16], while polymetric views use visual properties to represent software metrics [8, 17]. 3D and VR visualizations use spatial representations to support software exploration [9, 18, 19, 20, 21], while cartographic approaches place entities according to their similarities [10]. These techniques provide different ways to understand software structure, but representing relationships clearly remains an important aspect of software visualization.

Research Gap. Despite the variety of existing techniques, drawing software relationships while preserving connectivity and reducing overlaps remains challenging. Entity placement and relationship drawing must be coordinated to produce clear visualizations. This motivates exploring approaches that draw connections clearly while ensuring all software relationships are represented correctly. In the next chapter, we introduce Wave Function Collapse, an algorithm using local compatibility rules to assign tiles to a grid, and discuss its potential for addressing this problem.

Chapter 3

Wave Function Collapse

Wave Function Collapse is a procedural generation algorithm that assigns tiles to cells in a grid while satisfying local compatibility rules. Since its introduction, several extensions have been proposed to address its limitations and support different applications. In this chapter, we explain how the standard WFC algorithm works and review its main variants and applications, focusing on the techniques relevant to this thesis.

3.1 Wave Function Collapse Algorithm

Introduced by Maxim Gumin in 2016 [34], WFC progressively assigns tiles to cells in a grid, restricting the possible tiles for neighboring cells according to local compatibility rules [22].

Borrowing concepts and terminology from quantum mechanics, Wave Function Collapse starts with a grid of cells, where each cell contains a set of possible tiles. This set represents the states that the cell can have before a final configuration is selected by the algorithm. A cell with multiple possible tiles is therefore said to be in a "superposition" state. The choice is followed by constraint propagation to update the possible tiles in neighboring cells. The goal of WFC is to reduce these possibilities until every cell has been assigned to exactly one tile, while ensuring that all local constraints are satisfied. This process gradually narrows the available choices until a valid configuration is reached across the entire grid.

WFC is based on two main components: a *tileset* and a set of *adjacency rules*. The tileset defines the elements to which a cell can collapse, while the adjacency rules specify which tiles are compatible with each other along each direction. These components are defined differently depending on the model, leading to two variants:

- **Tile-based model**: The tiles and their adjacency rules are manually defined by the user. Each tile has a set of compatible neighbors for each direction. This model provides direct control over both the elements that can be generated and their adjacency constraints, which can be specified independently for each direction. An example of the tile-based model from Gumin's GitHub repository [34] is shown in Figure 3.1.
- **Overlapping model**: The tiles and their adjacency rules are extracted from an input example. This allows WFC to reproduce local patterns from the input while generating a new configuration. Some examples of the overlapping model from Gumin's GitHub repository [34] are shown in Figure 3.2.

The choice of model depends on the type of output and the information available. The tile-based model is useful when the desired elements and their compatibility rules can be defined directly. In contrast, the overlapping model is suitable when the goal is to learn local patterns from an existing example. In both cases, the quality of the result depends on the tiles and constraints provided to the algorithm.

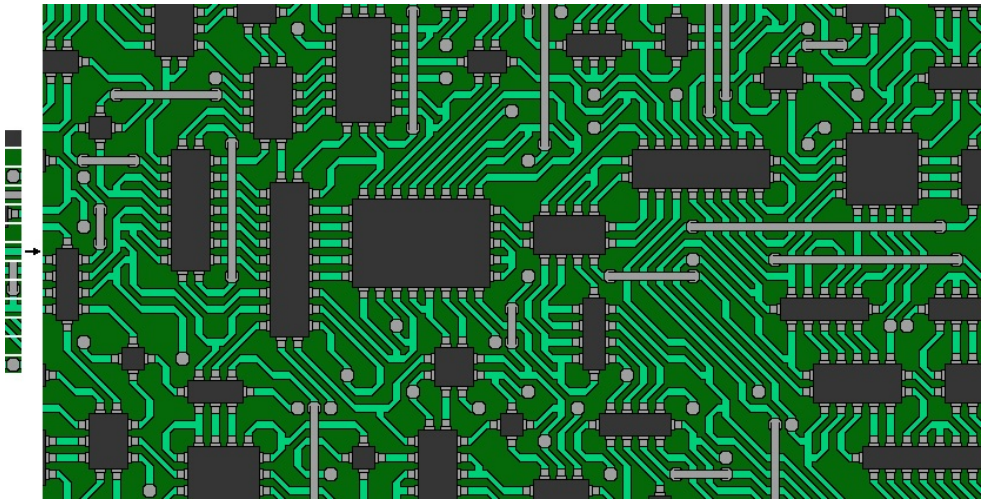


Figure 3.1: Example of a tile-based model.

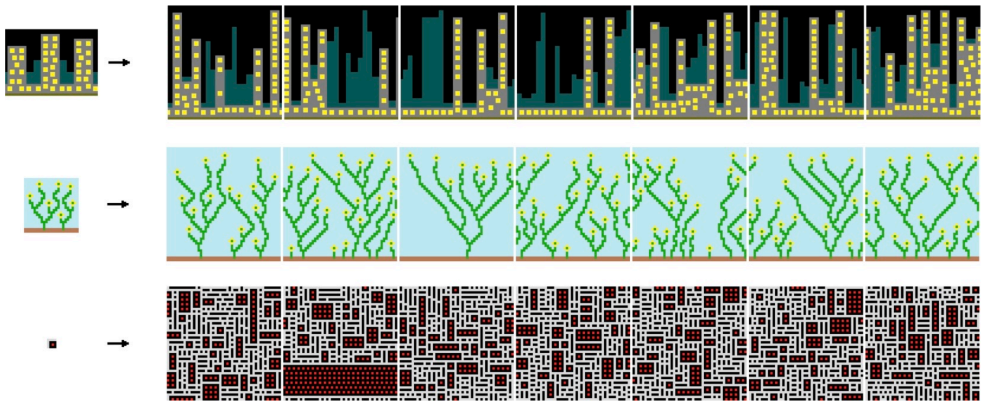


Figure 3.2: Examples of the overlapping model.

Regardless of the model, the generation process consists of two main phases: *observation* and *propagation*. The overall process is summarized in Algorithm 1.

Algorithm 1 Wave Function Collapse

1: Build possible tiles and constraint tables
2: Initialize grid: cells in superposition of all possible tiles
3: **repeat**
4: Choose next cell to collapse
5: Collapse it to a tile
6: Propagate the constraints
7: **until** all cells are collapsed **or** a cell has no possible tiles remaining

Initialization (Lines 1 and 2). The algorithm builds the tileset and the corresponding constraint tables. The grid is then initialized so that each cell contains the tiles allowed at that position. Thus, no cell has been assigned a final tile, and every cell is in a superposition state.

Observation Phase (Lines 4 and 5). During the observation phase, the algorithm selects an uncollapsed cell and collapses its state from a superposition of potential tiles down to a single, definite tile. The choice of cell can be guided by various heuristics [22]. A common strategy is to select the cell with the lowest Shannon entropy, corresponding to the cell with the least uncertainty and the most constrained neighborhood. Other strategies include random selection, maximum entropy, and predefined deterministic orders such as left-to-right, spiral, or Hilbert curves.

Entropy can be defined simply as the number of remaining possible tiles. However, when a weight is assigned to each tile, Shannon entropy can be used. Given a set of possible tiles with weights $w_1, \ldots, w_n$, we first define the probability p_i of picking tile i by normalizing its weight over the sum of all remaining weights:

$$p_i = \frac{w_i}{\sum_{j=1}^{n} w_j} \tag{3.1}$$

The standard Shannon entropy H of the cell is then expressed as:

$$H = -\sum_{i=1}^{n} p_i \ln p_i \tag{3.2}$$

After selecting a cell, WFC chooses one of its possible tiles. This choice can also be guided by different strategies. A weighted strategy assigns different probabilities to the possible tiles, while other strategies can select random, rarest, or most common tile [22]. As multiple tiles may be valid choices for a cell, different observations can lead to different configurations. WFC is inherently non-deterministic: running the same instance multiple times can produce different outputs.

Propagation Phase (Line 6). Once a cell is collapsed, the algorithm propagates the information to its uncollapsed neighbors verifying the constraints and reducing their remaining tiles accordingly. For each neighbor, any tile that is no longer compatible with the tile of the collapsed cell is removed from its tileset. If this removal reduces the candidates of a neighbor, the information is recursively propagated. This continues until no more candidates are eliminated from any cell.

If a cell reaches an empty set of candidate tiles, a contradiction has occurred, the current attempt is invalid, and the algorithm stops. In this case, the standard implementation by Gumin performs no backtracking. Instead, the current generation attempt is discarded, and the algorithm starts again. If repeated attempts fail to produce a valid configuration, the generation is considered unsuccessful. Other WFC variants [22] introduce backtracking to recover from contradictions without restarting the entire generation. The algorithm can restore a previously saved state and make different choices, relying on the fact that a different sequence of collapses may lead to a valid solution. In Figure 3.3, we show a step of the main loop in the WFC algorithm.

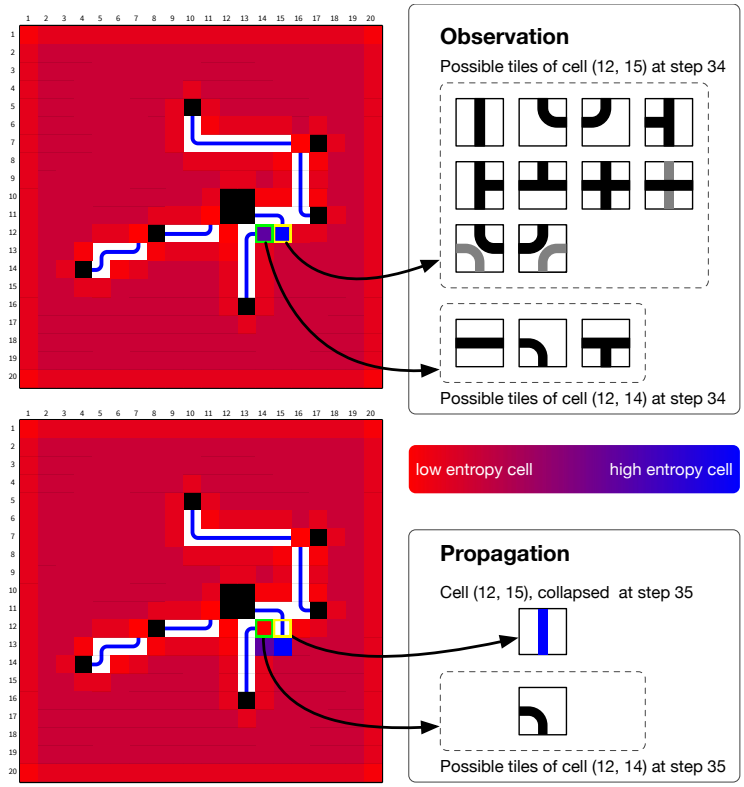


Figure 3.3: Example of an iteration step of the main WFC loop.

3.2 WFC Extensions and Applications

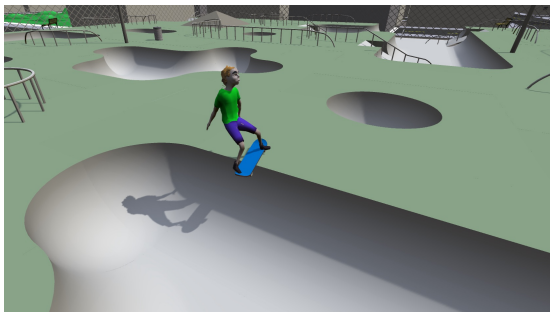
Since its introduction by Gumin [34], Wave Function Collapse has been applied to many different domains and extended to overcome some of the limitations of the original algorithm.

Formalization and Different Strategies. Karth and Smith [22] provided a formal description of the WFC algorithm. They studied different strategies for cell and tile selection, as well as the use of backtracking and global constraints. Their results showed that these choices can significantly affect both the generation process and the final output. In particular, they showed that the restart from scratch strategy used in Gumin's original implementation works well with simple local rules, but becomes less effective when global constraints are introduced. In such cases, the number of contradictions increases significantly, making backtracking essential.

Global Constraints. Standard Wave Function Collapse relies on local neighbor rules, making it hard to control patterns across multiple cells or the entire map. Sandhu *et al.* [23] expanded WFC by adding non-local and design-level constraints, such as minimum and maximum item limits, tile dependencies, and changing weights dynamically during generation. These improvements allow the algorithm to follow global design requirements.

Designer Control. Langendam and Bidarra [24] focused instead on giving designers more direct control over the generation process. Their *miWFC* introduces interactive tools for directly editing the output. A pencil tool allows users to select a cell and manually collapse it to a tile, while a brush tool allows them to select areas to reset or preserve. Users can also define templates from an area of the output and place them multiple times in different locations or generations.

Game Development. WFC has been widely used in game development for environment and level generation. Parker and Jones used it in *Proc Skater 2016* [35] to generate unique skateboard parks. Stålberg used WFC in *Bad North* [36] to generate 3D islands and in *Townscaper* [37] to turn player clicks into coherent buildings. Kleineberg [38] created *Infinite City*, a game that generates a city as the player explores it. Examples are shown in Figure 3.4.



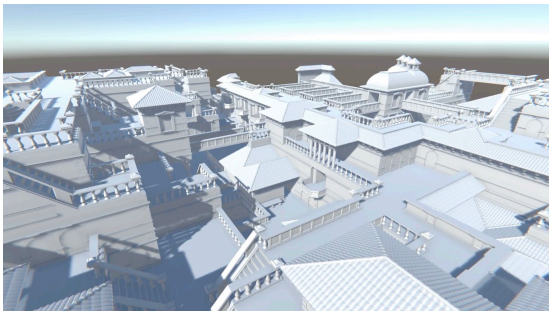
(a) *Proc Skater 2016* [35].



(b) *Bad North* [36].



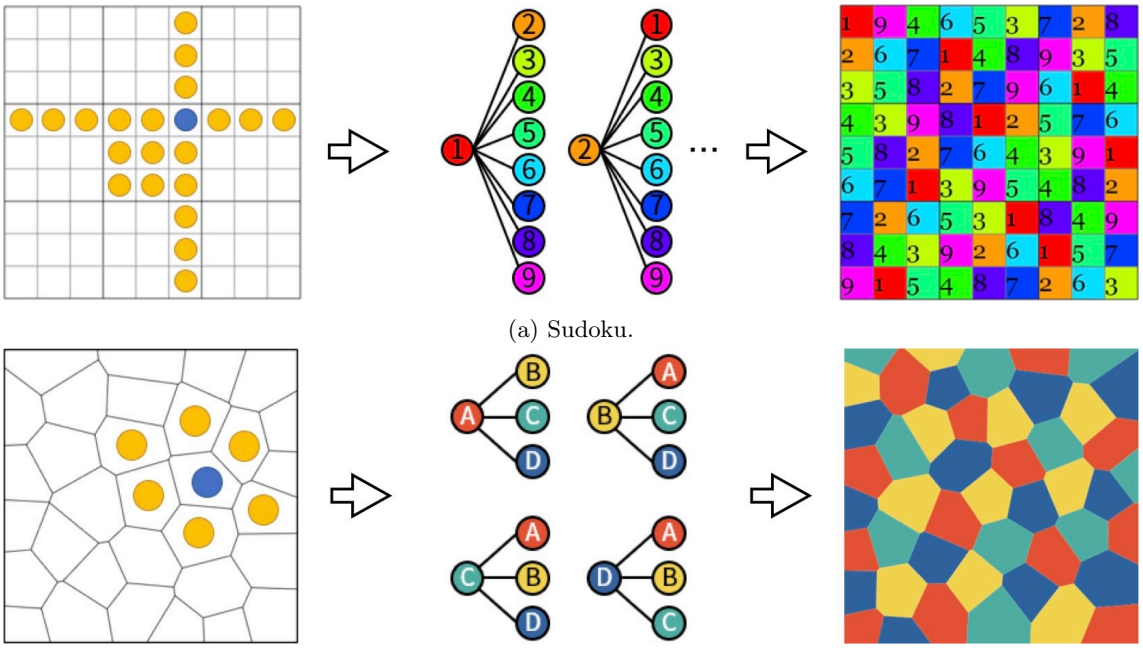
(c) *Townscaper* [37].



(d) *Infinite City* [38].

Figure 3.4: Examples of WFC applications in game development.

Graph-based WFC. Original WFC operates on a square grid. Kim *et al.* [25] extended WFC to graphs, allowing it to work with structures where nodes can have a different number of neighbors. They applied their graph-based WFC to problems such as Sudoku and the Voronoi graph 4-coloring problem, as shown in Figure 3.5.



(a) Sudoku.

(b) Voronoi graph 4-coloring.

Figure 3.5: Examples of graph-based WFC applied to Sudoku and Voronoi graph 4-coloring [25].

Kim *et al.* [26] later adapted this approach for procedural level generation in logic puzzle games. They applied WFC to *Strimko* and *Flow* puzzles, showing that it can generate levels that satisfy the constraints of these games faster than depth-first search and genetic algorithms.

Scalability. Another limitation of standard WFC is the cost of generating large outputs. To fix this issue, Nie *et al.* [27] introduced Nested Wave Function Collapse (*N-WFC*), which splits a large map into a grid of smaller subgrids. Instead of solving the whole map at once, smaller WFC instances generate these subgrids step by step while matching edges with neighboring blocks. This decomposition allows each subgrid to be processed independently, reducing the size of the individual problems. This reduces computation time from exponential to linear or polynomial, avoiding heavy backtracking and making it easy to create very large or infinite game worlds.

Non-Uniform Tiles. Standard WFC assumes that all tiles have the same size and shape. Piepenbrink and Bidarra [28] addressed this limitation with Non-Uniform Tile Wave Function Collapse (*nutWFC*). Their approach allows tiles with different shapes and sizes to occupy multiple cells. To preserve the structure of these multi-cell tiles, they split each tile into single-cell units called *atoms*, each identified by a unique ID. They then defined strict adjacency constraints between these atoms. These constraints ensure that all atoms of the same tile are collapsed together, preserving their original shape and size during generation.

Poetry. WFC has also been applied to less conventional domains. For example, O'Leary [39] used WFC to generate poetry by treating syllables as tiles and defining constraints based on rhymes.

3.3 Summary

In this chapter, we reviewed the Wave Function Collapse algorithm, its main extensions, and its applications in different domains.

WFC Extensions. WFC assigns tiles to grid cells while enforcing local compatibility rules [34]. Its behavior depends on the strategies used to select cells and tiles, as well as on how contradictions are handled [22]. Several extensions introduce global constraints and dynamic tile weights [23], interactive control [24], graph-based structures [25, 26], nested grids for scalability [27], and tiles of different sizes [28]. WFC has also been applied to procedural content generation, particularly to game-environment and level generation [35, 36, 37, 38] and poetry generation [39].

Influence on this Thesis. These extensions provide several ideas relevant to our approach. We adopt dynamic tile weight adjustment [23], backtracking strategies [22], and non-uniform tiles [28]. However, applying WFC to software visualization requires additional mechanisms because local compatibility alone does not ensure that software relationships connect the correct entities. We therefore define tiles and compatibility rules based on software relationships, combine WFC with Multidimensional Scaling for entity placement, and introduce mechanisms to guide paths towards their destinations and handle contradictions. These adaptations let us explore WFC for drawing software relationships while preserving connectivity and reducing path crossings.

Chapter 4

From Code to Visualization

In this chapter, we present how Wassis combines Multidimensional Scaling and Wave Function Collapse to position software entities on a canvas and draw the relationships between them while preserving connectivity and reducing visual clutter.

The proposed approach consists of four main phases, illustrated in Figure 4.1:

1. **Initialization**: Parsing the source code to extract entities (classes) and their dependencies, initializing an empty grid.
2. **Layout via MDS**: Computing a distance matrix based on structural relationships, then applying Multidimensional Scaling to position the classes on the grid.
3. **Path Generation via WFC**: Defining path tiles and leveraging Wave Function Collapse to procedurally generate paths between the placed classes.
4. **Finalization**: Computing metrics such as path length and edge crossings for the evaluation of the resulting visualization, and rendering the final result.

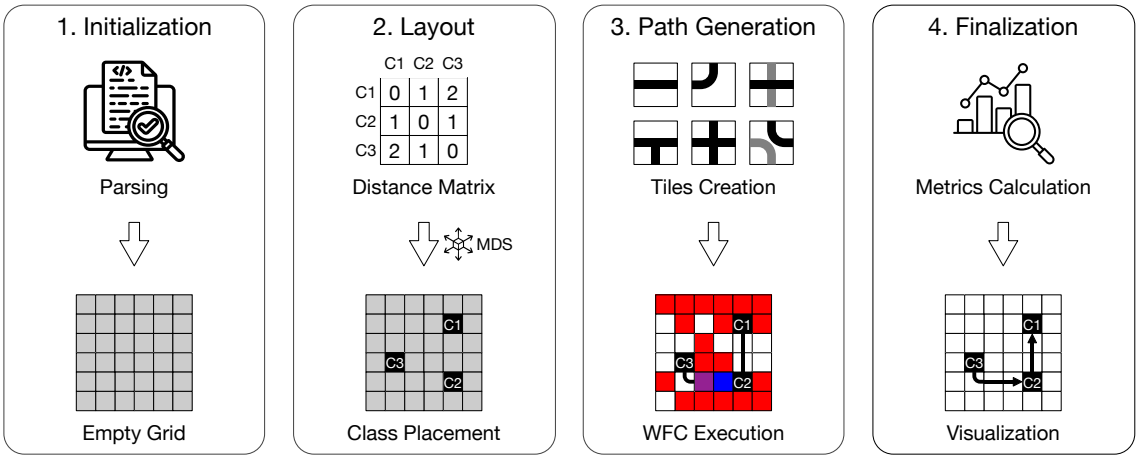


Figure 4.1: General flow of the approach.

4.1 Initialization

After extracting the classes and their relationships from the source code, an empty grid is created. The grid is always a square, and its initial size is computed based on the number of classes. The size is intentionally kept small to make the visualization more compact and to help WFC in finding a solution in a computationally efficient manner.

Each cell starts with 15 possible tiles: one empty tile and 14 path tiles. Therefore, each cell is initially in a *superposition* state. The available tiles are shown in Figure 4.2. They cover the main path shapes, such as straight segments, turns, T-junctions, and crossings. The black segment represents the path, while the gray segment, when present, represents a different path. Each path is identified by a unique ID.

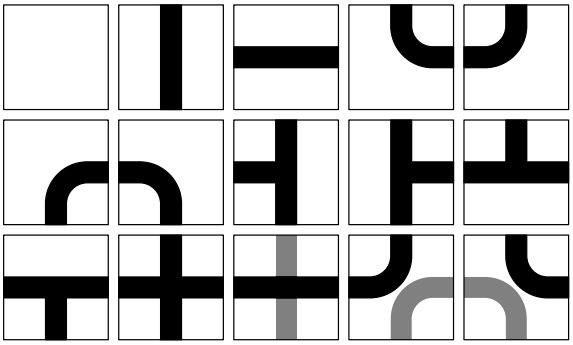


Figure 4.2: Initial possible tiles.

The tiles also define the local constraints used by WFC. These constraints determine which tiles can be placed next to each other. For each tile, the set of compatible tiles is defined separately for each direction (up, left, right, and down). An example is shown in Figure 4.3, where the straight tile can only be placed next to tiles with matching path segments in each direction.

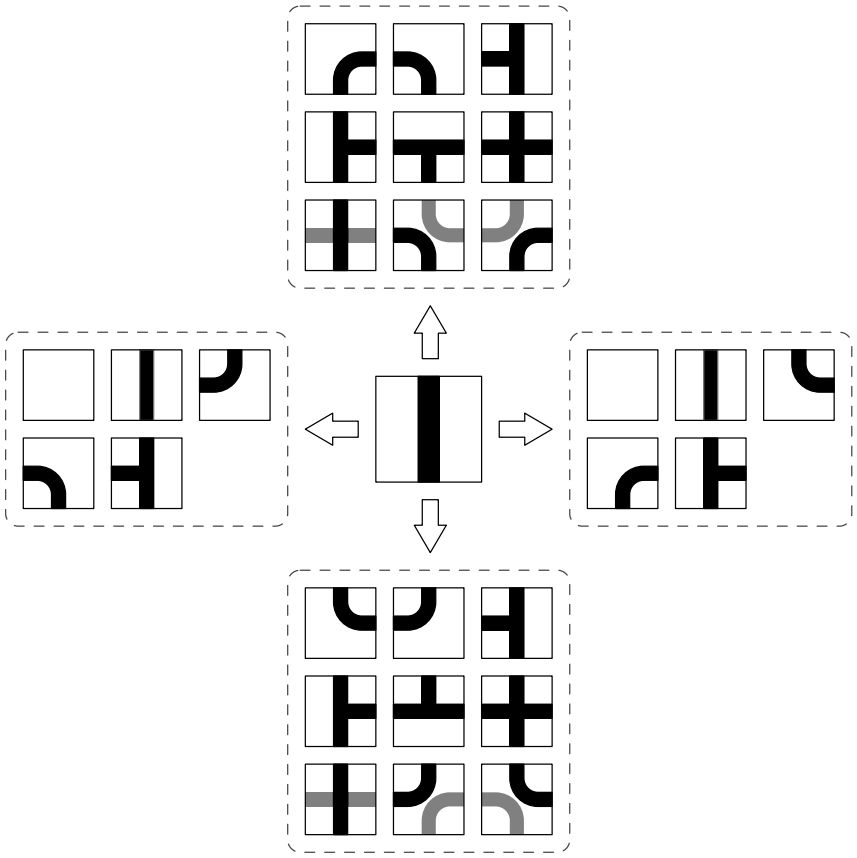


Figure 4.3: Example of compatible tiles for each direction.

Some path tiles cannot be placed on the border of the grid, as they would allow a path to "leave" the grid. For example, a vertical path cannot be placed at position $(1, 1)$. These invalid tiles are manually removed from the possible tiles of border cells before WFC starts.

4.2 Layout via MDS

Before generating the paths, the classes must be placed on the grid. The goal is to position strongly related classes close to each other while leaving enough space for their connections. This process consists of four steps:

1. Determining the size of each class based on its number of connections.
2. Computing a distance matrix between all pairs of classes.
3. Computing target 2D positions using Multidimensional Scaling.
4. Placing the classes on the grid while respecting placement constraints.

4.2.1 Multi-cell Classes

Classes can have different numbers of incoming and outgoing connections. A single cell may not provide enough space for all the paths connected to a class. Therefore, classes can occupy multiple cells, with their size determined by the number of connections they have.

Each class is represented as a black square of cells, with side length $S = \lceil k/4 \rceil$, where k is the total number of incoming and outgoing connections of the class. For example, a class with up to four connections occupies one cell (1×1), while a class with five to eight connections occupies a 2×2 region, and so on. This provides space for paths to enter and leave the class.

4.2.2 Distance Matrix

To position related classes close to each other, we first need to represent the relationships between them as distances. We build an $n \times n$ distance matrix, where n is the total number of classes. The distance between two classes is inversely related to the total weight of their direct connections: stronger connections result in shorter distances.

The distance between Class A and Class B is calculated as the reciprocal of the sum of the weights of their connections:

$$\text{Distance}(A, B) = \frac{1}{\sum \text{Weight of connections between } A \text{ and } B} \tag{4.1}$$

Different relationship types can be assigned different weights. For example, a method call can have a weight of 2, while an attribute access can have a weight of 1. If Class A calls a method in Class B and Class B reads an attribute of Class A, their total connection weight is $2 + 1 = 3$, resulting in a distance of $\frac{1}{3}$.

If two classes have no direct connection, their initial distance is set to a maximum default value.

After filling the matrix with direct distances, we apply the Floyd-Warshall algorithm [29]. This computes the shortest distance between all pairs of classes, enriching the information given to MDS with indirect relationships.

Classes that are not connected, either directly or indirectly, are assigned a maximum default distance from all other classes.

4.2.3 Target Position on the Grid

Once the distance matrix is available, we use Multidimensional Scaling [11, 12] to compute target positions for the classes. MDS converts the distances into 2D coordinates, aiming to preserve the distances between classes as closely as possible.

The resulting coordinates are then normalized and scaled to fit the current grid size. An example of assigned target positions is presented in Figure 4.4. The number inside each square indicates how many classes were assigned to that position by MDS, as strongly connected entities may occasionally be mapped to identical coordinates. These positions are targets rather than final placements, since the next step must also respect the constraints of the grid.

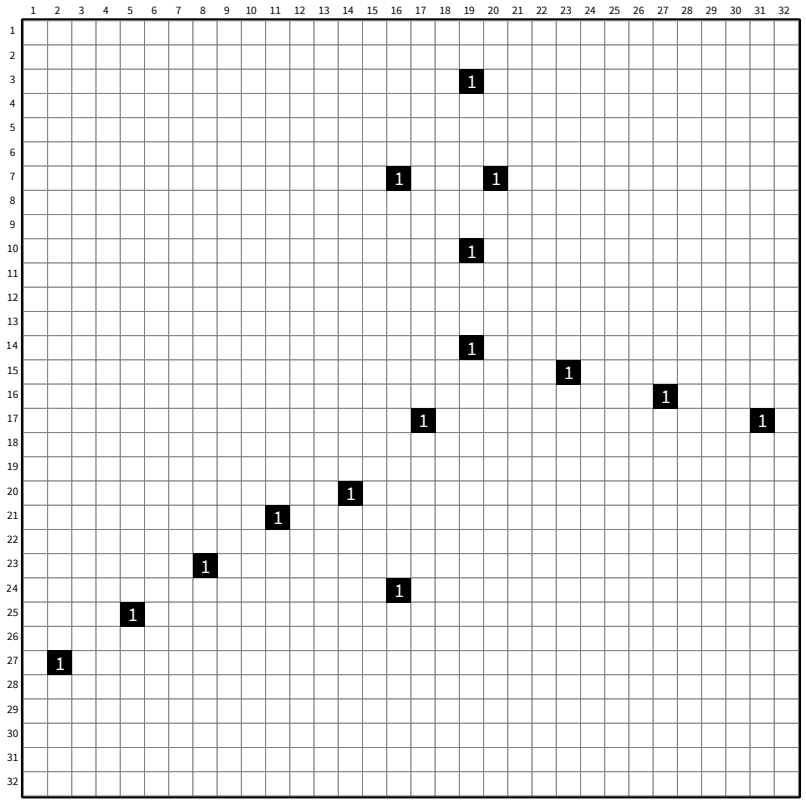


Figure 4.4: Example of target positions calculated by MDS (normalized and scaled to the grid).

4.2.4 Class Placement Strategy

The target positions computed by MDS do not always satisfy the placement constraints. For example, classes may overlap or leave insufficient space for paths. Therefore, classes are placed on the grid one at a time, using a strategy that finds valid positions while keeping them as close as possible to their target positions.

Class Placement Order The order in which classes are placed may affect the final layout, since placing one class can limit the available positions for the others. Our approach supports five ordering strategies:

- **Random**: Classes are placed in a random order.
- **Minimum Connections First**: Classes are sorted by their total number of connections, from the fewest to the most.
- **Maximum Connections First**: Classes are sorted by their total number of connections, from the most to the fewest.
- **BFS Minimum**: Breadth-First Search starting from the class with the fewest connections.
- **BFS Maximum**: Breadth-First Search starting from the class with the most connections.

To leave enough space for paths, a minimum distance is kept between classes. This parameter defines the minimum number of empty cells required between two classes. For example, if the minimum distance is 3, at least three empty cells must separate the borders of two classes horizontally, vertically, and diagonally.

Candidate Search For each class, the algorithm first tries to use the target position computed by MDS. If that position violates the placement constraints, the algorithm searches for an alternative nearby position. The search proceeds as follows:

1. The algorithm first tries to place the class at the target position computed by MDS.
2. If the target position is not valid, the algorithm searches for valid positions in expanding rings around it, starting with the first ring around the target position.
3. If one or more valid positions are found in the current ring, the position with the smallest total Manhattan distance to the already placed connected classes is selected. This criterion favors placements that keep strongly connected classes close to each other.
4. If no valid position is found in the current ring, the algorithm continues with the next ring.
5. This process is repeated until at least one valid position is found.

If no valid position can be found, the grid is expanded and the placement process restarts. This allows the algorithm to create more space when the current grid cannot accommodate all classes under the placement constraints.

An example of this process, illustrating the search for a valid position for a class strongly connected to an already placed class, is provided in Figure 4.5.

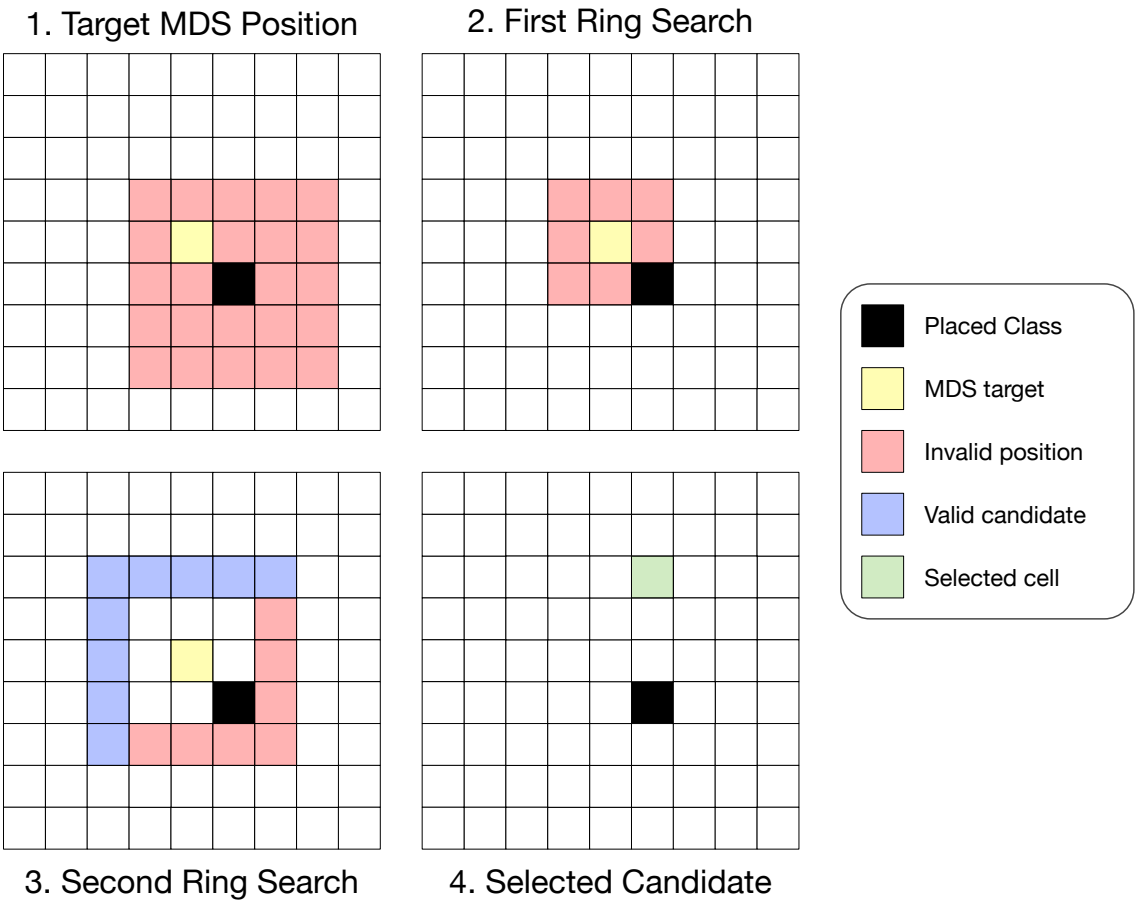


Figure 4.5: Candidate search example (minimum distance: 2).

4.3 Relationship Representation

After placing the classes, the relationships between them are represented as paths. Drawing a separate path for every relationship would quickly lead to a cluttered visualization [1]. Therefore, relationships are merged into a single path when they have the same destination and the same relationship type.

For example, if classes B, C, and D are all subclasses of class A, their relationships can be represented by a single path from B, C, and D to A. Similarly, if classes B and C both call a method of class E, these relationships are represented by a single path from B and C to E. A conceptual example of this merging process is shown in Figure 4.6.

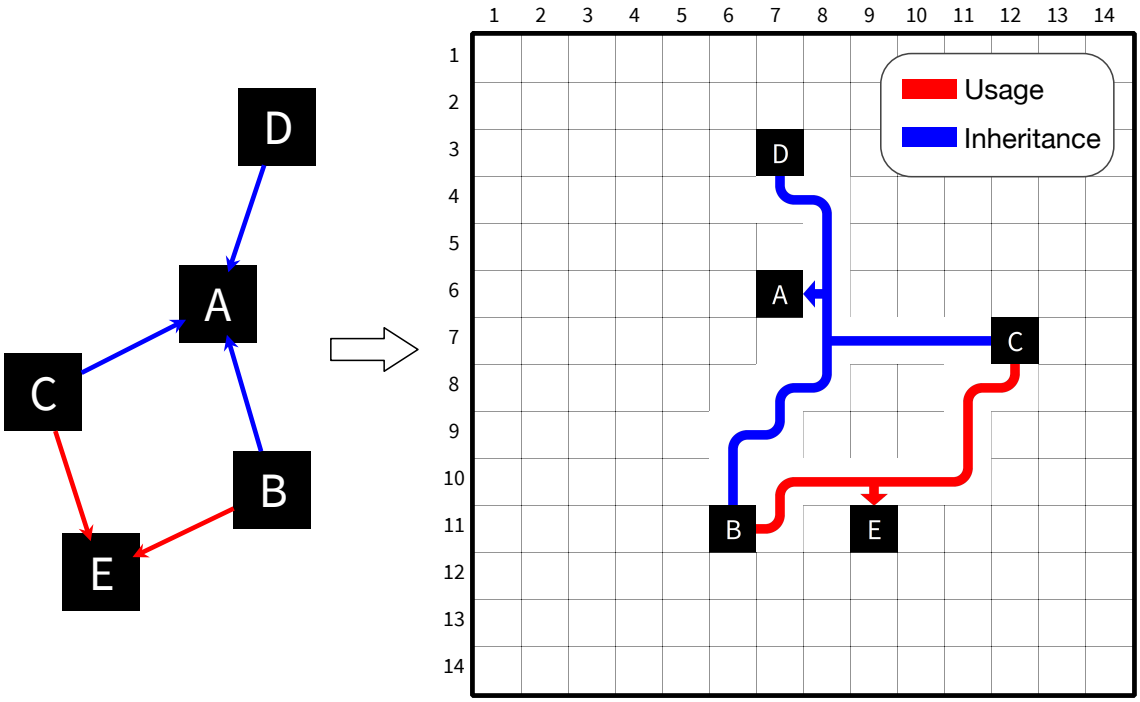


Figure 4.6: Conceptual representation of path merging.

Different relationship types are represented using different colors. For example, hierarchy relationships are shown in blue, while method calls are shown in red. This allows the type of a relationship to be identified directly from the visualization.

Path merging also reduces the total number of paths that WFC has to generate, which can make it easier for WFC to find a valid solution. Moreover, merging paths also reduces the number of border cells that a class needs to expose. Nevertheless, we keep the footprint of the class proportional to the *overall non-merged* number of connections as this serves as an explicit visual encoding to highlight highly connected classes.

Each merged path is assigned a unique path ID based on the IDs of its source classes, its destination class, and its relationship type. The path ID is used to enforce the global connectivity constraint using only the local ones of WFC.

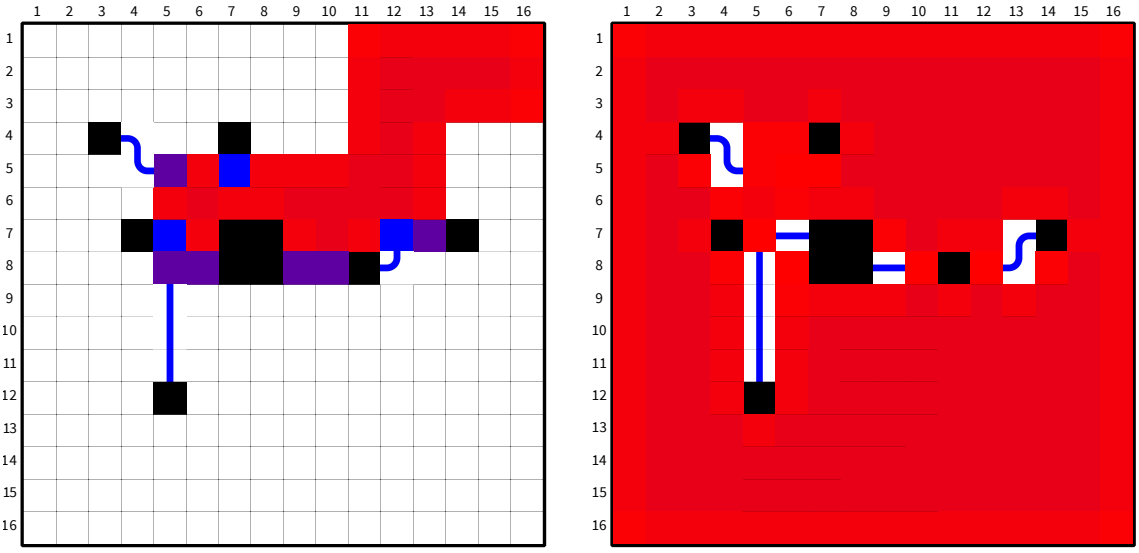
The path ID is embedded in the border cells of the corresponding classes and propagated through the grid during WFC. A path can therefore only start from and terminate at cells containing its ID. This ensures that paths connect the intended classes and prevents them from connecting unrelated classes. When multiple relationships are merged into the same path, they share the same path ID, ensuring that they are represented as a single path.

4.4 Path Generation via WFC

In this section, we describe how we adapted and tuned the Wave Function Collapse algorithm to generate paths between classes.

4.4.1 Observation Phase

The observation phase consists of two steps: selecting the next cell to collapse and collapsing that cell into a tile. While standard WFC usually selects the cell with the minimum entropy, in our approach we select the cell with the *maximum entropy*. Although this choice may seem counterintuitive, it is motivated by the way tile weights are defined in our tileset and by our goal: generating the paths before filling the grid with empty tiles. Empty tiles could otherwise occupy cells that may be needed by paths during generation, as shown in Figure 4.7.



(a) Standard WFC using a min-heap.

(b) Our approach using a max-heap.

Figure 4.7: Comparison of min-heap and max-heap collapse order.

For the second step of the observation phase, we use a weighted random choice to select a tile from the possible tiles of the selected cell. The weights assigned to the tiles are fixed. Therefore, the entropy of a cell can be computed as:

$$H = \ln\left(\sum_{i=1}^{n} w_i\right) - \frac{\sum_{i=1}^{n} w_i \ln w_i}{\sum_{i=1}^{n} w_i} \tag{4.2}$$

where $w_1, \ldots, w_n$ are the weights of the possible tiles of the cell.
Note that Equation (4.2) is the standard Shannon entropy expressed directly in terms of the tile weights instead of normalized probabilities. Proof of the equivalence with the standard formulation is shown in Appendix A.

Tile Weights The weights assigned to the tiles affect both the random tile selection and the entropy used to select the next cell. The empty tile has a very large weight, while the basic path tiles, such as straight and corner tiles, have much smaller and relatively similar weights. This leads to two situations:

- If the empty tile is still among the possible tiles of a cell, its large weight makes it very likely to be selected. The cell therefore has low entropy.
- If constraint propagation removes the empty tile, only path tiles remain. Since their weights are relatively similar, the choice is less predictable and the cell has higher entropy.

Avoiding Unnecessary Paths More complex path tiles, such as T-junctions, crossings, merged paths, and double paths, are assigned a weight of zero. This means they are not selected during random tile selection as long as a positive weight alternative is available. They are only used when the constraints leave no other option.

This strategy helps prevent unnecessary paths. For example, selecting a T-junction when no branching is needed would create an extra branch that WFC would later have to close, potentially creating redundant loops. The same applies to crossings and other complex path tiles, which should only be used when required by the relationships being represented.

Straight and corner tiles, however, cannot have a weight of zero because they are needed to connect classes. Their weights remain much smaller than the weight of the empty tile. Therefore, when both empty and path tiles are possible, the empty tile is much more likely to be selected. This reduces the chance of generating paths in cells where they are not needed. An example of unnecessary paths is shown in Figure 4.8.

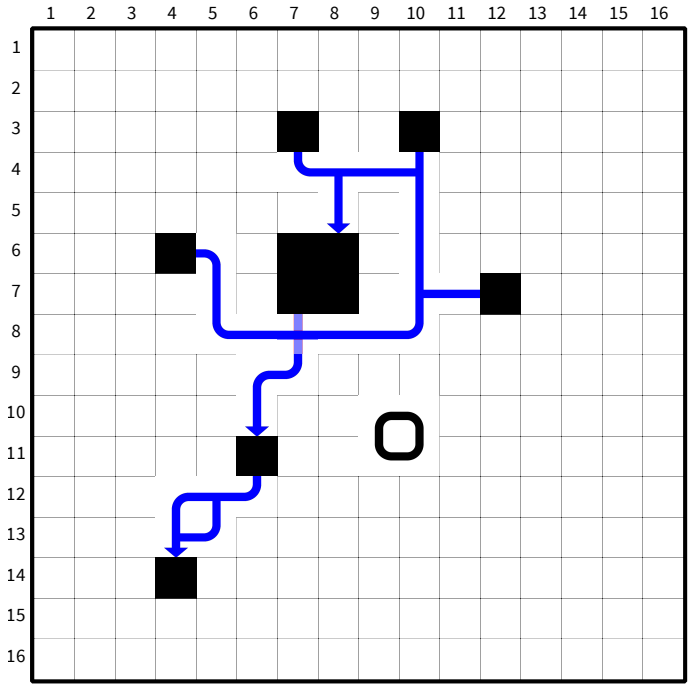


Figure 4.8: Example of unnecessary paths generated by WFC.

Path First Generation The tile weights also affect the order in which cells are selected. When classes are placed on the grid, their path entry and exit points are already fixed. Constraint propagation removes the empty tile from cells next to these points, leaving only path tiles. These cells therefore have higher entropy than cells where the empty tile is still possible. As a result, they are selected first when using maximum-entropy observation. An example is shown in Figure 4.9.

As paths extend, constraint propagation removes the empty tile from neighboring cells along the paths. These cells also gain higher entropy and become candidates for the next observation. This encourages WFC to extend paths from their entry and exit points before filling the remaining grid with empty tiles.

Another consequence of maximum-entropy observation is that paths do not necessarily close as soon as they reach their targets. Constraint propagation may leave the last cell of a path with only one possible tile or with only tiles that have a weight of zero. In both cases, the cell has low entropy and receives lower priority than cells with higher entropy.

As a result, paths can remain open while WFC generates other paths. Even after all paths have reached their targets, some final tiles may still be unassigned. WFC can therefore fill the remaining cells with empty tiles before closing these paths, as shown in Figure 4.10.

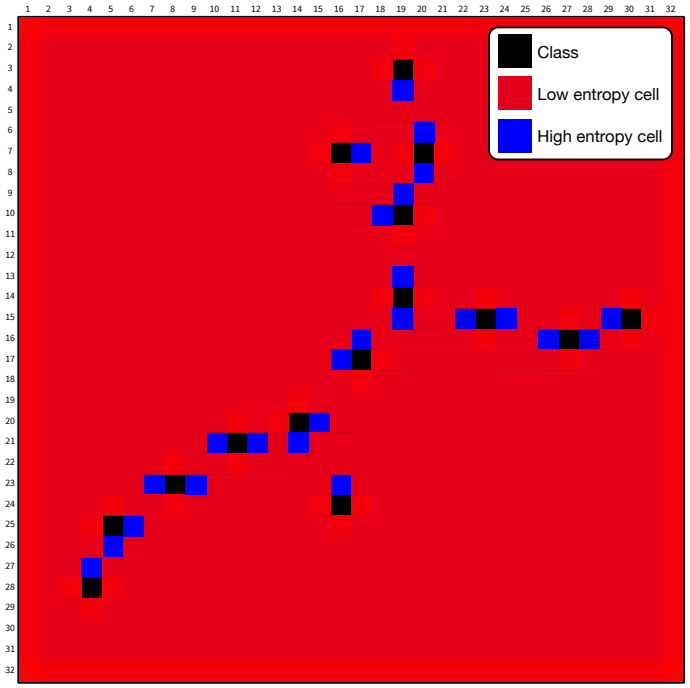


Figure 4.9: Initial class placement on the grid, with fixed path entry and exit points.

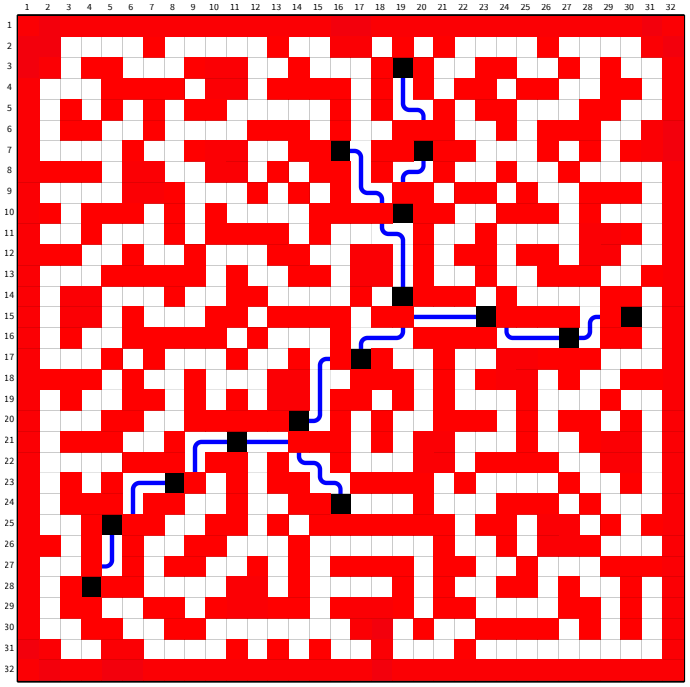


Figure 4.10: Multiple paths remaining open during the WFC generation process.

4.4.2 Propagation Phase

After a cell is collapsed to a tile during observation, WFC propagates this choice across the grid to satisfy the local constraints. This reduces the possible tile options in neighboring cells based on the selected tile.

Adjacency rules are defined using symbols assigned to each edge of a tile. Two tiles can be placed next to each other only if their touching edges have matching symbols.

When a cell loses tile options, this change is propagated to its neighbors through a loop:

1. When a cell loses at least one tile option, it is added to a queue.
2. While the queue is not empty, a cell C is removed from the queue.
3. For each neighbor N of the cell C, WFC removes any tile option whose symbol no longer matches at least one remaining tile in C. If a neighbor N has no valid tile options left, the current configuration is considered invalid and backtracking or restart is required.
4. If any tile is removed from N, N is added to the queue so that the change can be propagated further through the grid.

This loop runs until the queue is empty, ensuring the grid remains consistent before the next cell is observed. Note that propagation can affect multiple cells, including those far from the initially collapsed cell.

An example of complete execution of WFC is provided in Figure 4.11.

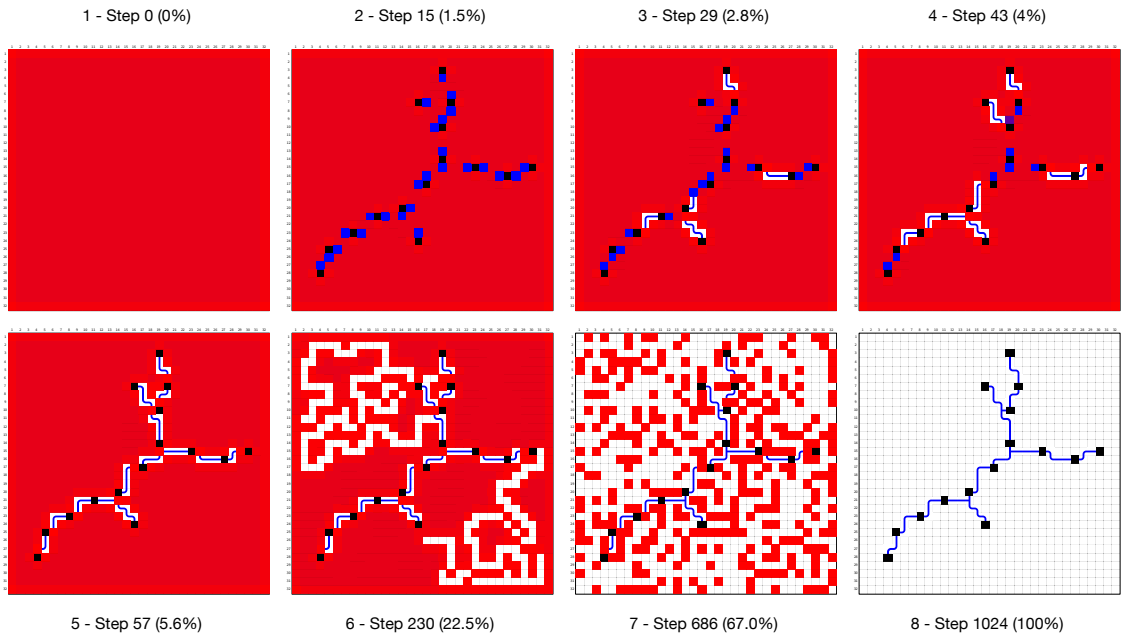


Figure 4.11: Animation of the execution of WFC.

To improve the final visualization, an arrow is added to the last tile of each path to indicate its direction. This makes the direction of the relationship easier to understand, helping distinguish the source from the destination, as shown in Figure 4.12.

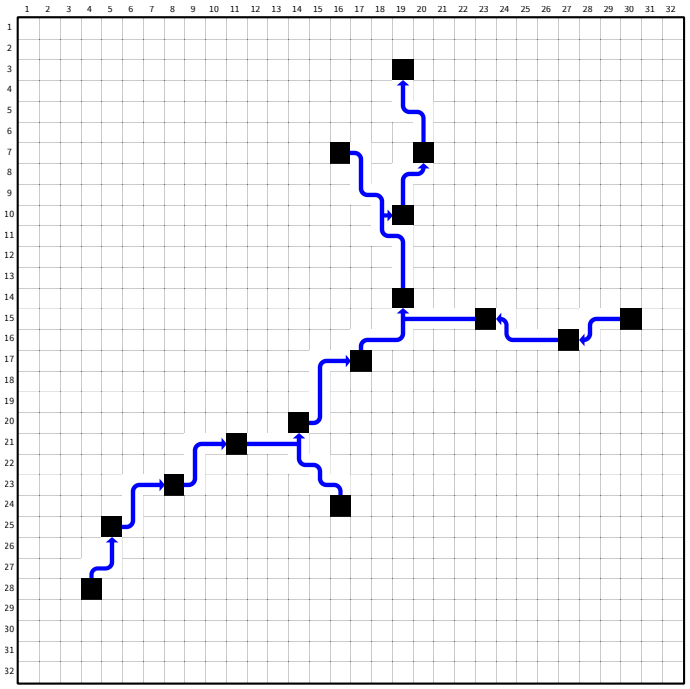


Figure 4.12: Visualization after WFC execution.

4.5 Computed Metrics

After generation is completed, the process computes metrics to describe the resulting visualization. These metrics can be used to evaluate its quality and computational cost.

Grid Metrics The following metrics describe the generated grid:

- **Grid size**: The width and height of the generated grid.
- **Number of nodes**: The total number of cells in the grid.
- **Node types**: The number of cells containing classes, paths, or empty tiles.
- **Number of overlaps**: The number of cells where two paths cross each other.

Path Metrics These metrics describe the paths generated between classes:

- **Total number of paths**: The total number of generated paths.
- **Direct paths**: The number of paths that directly connect two classes without merging with another path.
- **Merging paths**: The number of paths that merge with another path.
- **Path length**: The length of each generated path.

Distance Metrics For each pair of connected classes, WASSIS computes the Manhattan distance between them on the grid. From these values, the average distance of each class to all the classes it is connected to is also computed.

Software System Metrics The following metrics describe the structure of the input system:

- **Number of classes**: The total number of classes in the system.
- **Number of connections**: The total number of connections of each class.
- **Average connections per class**: The average number of connections considering all classes.

Execution Metrics Finally, information about the generation process is recorded:

- **Total execution time**: The total time required to generate the visualization.
- **Number of attempts**: The total number of WFC attempts, including attempts that failed and required a restart or grid expansion.

4.6 Challenges and Enhancements

In this section we present challenges encountered during development, how we solved them, and the enhancements made to adapt WFC to software visualization.

4.6.1 Managing the Exploding Number of Tiles

A major challenge when scaling our approach to larger systems was the explosion in the number of tiles. In a naive implementation, each path requires dedicated tiles that encode the path ID on its edges, even if they were visually identical. We initially created 11 dedicated tiles for each unique path ID, plus additional tiles for every pair of overlapping paths. However, this caused the tileset size to explode as soon as we targeted larger software systems. Since the propagation phase is the most computationally intensive part of WFC, this naive design made the approach computationally infeasible even for medium-sized systems.

To address this issue, we introduced a *placeholder tile system*. At initialization, every grid cell contains the empty tile and a fixed set of 14 generic path tiles, where path identities are represented by placeholders rather than concrete IDs. This strategy bounds the candidate tileset of each cell to 15 tiles regardless of the total number of paths, keeping constraint propagation tractable. Concrete path IDs are originally embedded only in the edges of the class nodes placed on the grid. As path cells collapse, path ID information propagates from cell to cell, resolving placeholders into concrete tiles on the fly.

This works because our cell selection strategy targets the cell with the maximum entropy (Subsection 4.4.1), so cells adjacent to already established paths are prioritized. However, when multiple candidate cells share the exact same entropy, WFC may collapse a path cell that is not yet connected to a resolved neighbor, leaving its path ID temporarily unknown. This is represented as a black path, as shown on the left in Figure 4.13.

To handle these cases, we perform a path ID propagation step immediately after collapsing a cell and before standard WFC propagation begins. The resolution process operates as follows:

1. Once a cell C collapses into a resolved path tile (a tile without placeholders), it is added to a queue to propagate the information to its neighbors.
2. While the queue is not empty, WFC checks the adjacent collapsed neighbors N. If a neighbor N contains a placeholder tile, it resolves N into a concrete path tile using the ID information from C.
3. If neighbor N successfully resolves into a concrete tile, it is added to the queue to further resolve its neighbors, if necessary.
4. After updating placeholder tiles, WFC verifies that touching edges between adjacent resolved tiles match. If an edge mismatch is detected, a contradiction is signaled.

An example of this process is illustrated in Figure 4.13. In the state on the left, a path segment has been collapsed using placeholder tiles (shown as black paths) because its specific path ID was temporarily unknown. In the next step, when the cell highlighted with the green border collapses into a resolved tile (a T-junction connected to the main path), it propagates the information about the concrete path ID to the path segment, immediately resolving it into concrete tiles.

Once all placeholders are resolved, WFC performs standard propagation from the collapsed cell. This ensures that the new constraints are propagated to neighboring cells.

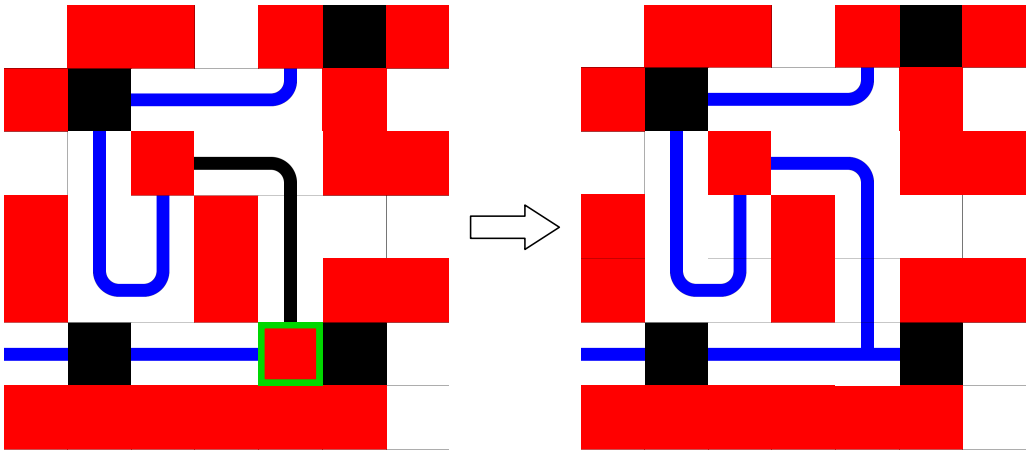


Figure 4.13: Resolution of path ID placeholders during generation.

4.6.2 Guiding Paths Towards Their Targets

A problem with WFC is that, even when classes that should be connected are placed close to each other using MDS, path tiles are still selected through a weighted random choice. Therefore, there is no guarantee that a path will be generated towards the class or path where it should connect. The path may instead grow in a different direction and fail to reach its target. While this works for very small examples (*e.g.,* 5 classes), it becomes impractical for larger software systems.

To address this problem, we implemented two main strategies:

- **Directional Path Exits:** Path exits are fixed for each class before the WFC generation starts. Their direction is chosen based on the position of their target.
- **Dynamic Tile Weighting:** Path tiles are not selected completely randomly. Taking inspiration from the approach proposed by Sandhu *et al.* [23], the weights of tiles that move a path towards its target are temporarily boosted.

Path Exits Before the WFC generation starts, path exits (and entries) are assigned to the borders of each class with concrete path IDs. This fixed assignment is necessary because, as described in Subsection 4.4.1, the generation process starts directly from the classes. Consequently, when a path reaches its target class, it cannot connect to any cell on the class border, but it must connect to the designated entry point.

To determine these exit points, we compute the centroid of all classes sharing that path. The path IDs are then assigned to the edges of the class cells that point towards this centroid. In this way, paths are more likely to start in the direction of their target.

Path Target When a cell that must be a path is selected for collapse, the direction towards its target is known. We temporarily increase the weights of the tiles that continue the path in that direction. This increases the probability of selecting a tile that moves the path closer to its target.

This strategy takes advantage of our observation phase (Subsection 4.4.1), where path cells are collapsed before empty cells. In addition to the path ID, we can pass the target information associated with each path ID to the next path cells.

We support three strategies for determining the target of a path:

- **Centroid:** The target is the centroid of all classes sharing the same path ID.
- **Nearest Class:** The target is the nearest class sharing the same path ID.
- **Random:** No target direction is used and the tile weights are not boosted.

Note that "black paths" (Figure 4.13) lack information about both the path ID and its target. Therefore, they are generated completely randomly.

4.6.3 Reducing the Number of Crossings

Our approach does not explicitly minimize path crossings through a dedicated optimization step. Instead, crossings are naturally minimized during path generation by assigning them a weight of zero (Subsection 4.4.1). As a result, Wave Function Collapse introduces crossing tiles only when constraint propagation eliminates all the alternatives, ensuring they appear only when strictly necessary to complete a path.

4.6.4 Backtracking Strategies

The standard Wave Function Collapse implementation [34] does not support backtracking and instead restarts from scratch when a contradiction occurs. Inspired by the work of Karth and Smith [22], we introduced backtracking to improve the success rate of the generation.

When a contradiction occurs, the generation process follows this recovery process:

1. **Local Backtracking:** The algorithm goes back to a previous decision using one of the strategies described below, up to a fixed number of backtracks.
2. **Restart with Same Placement:** If local backtracking fails, the generation restarts using the same grid size and classes placement. This is repeated for a fixed number of attempts.
3. **Restart with Same Grid Size:** If the previous attempts fail, the generation restarts with the same grid size but computes a new class placement. This is also repeated for a fixed number of attempts.
4. **Grid Expansion:** If all previous attempts fail, the grid size is increased and the generation starts again from the beginning.

The backtracking workflow is shown in Figure 4.14.
We implemented five strategies for backtracking:

- **Fixed Number of Steps:** Goes back a fixed number of steps from the current state. If this goes beyond the beginning of the generation, the process restarts from the beginning.
- **Percentage of Steps:** Goes back by a fixed percentage of the steps performed so far. If this goes beyond the beginning of the generation, the process restarts from the beginning.
- **Many Possible Tiles:** Goes back to a checkpoint saved before a cell with many possible tiles was collapsed. The user defines a minimum number of possible tiles as a threshold for saving the checkpoint.
- **Similar Tile Weights:** Goes back to a checkpoint saved before a cell was collapsed when its possible tiles had similar weights. The user defines the percentage of tiles to consider and the maximum allowed difference between their weights, expressed as a percentage.
- **Restart:** Clears the current grid and starts the generation again from the beginning.

When a checkpoint is restored, the cell is brought back to the state it had immediately before the original collapse. If the tile originally selected was a path tile, it is removed from the cell's possible tiles before the cell is collapsed again. This prevents WFC from making the same choice and allows it to explore a different alternative.

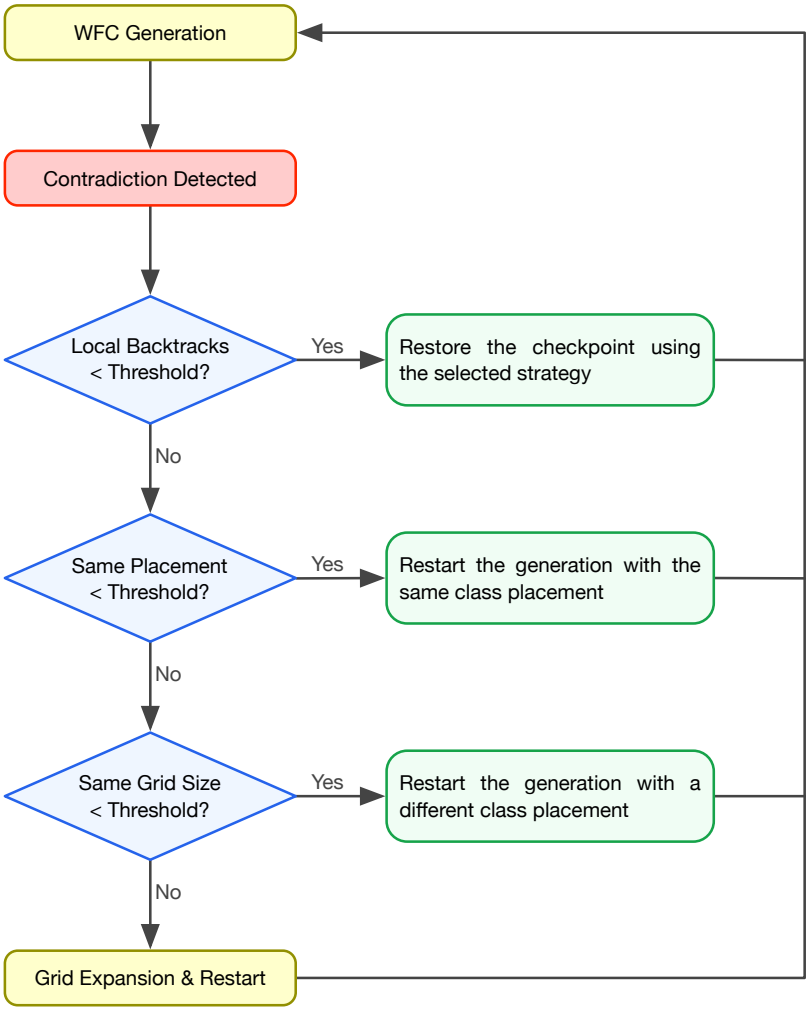


Figure 4.14: Backtracking workflow.

4.6.5 Path Optimization

WFC generates valid paths between classes, but these paths are not always optimal. Since WFC uses local rules and random tile selection, despite weights boosting, paths may contain unnecessary deviations. To reduce these detours, we added an optional *Path Optimization* step after WFC generation. The optimization removes the tiles of a selected path and rebuilds the path using Dijkstra's algorithm:

- **Direct Connections:** If the path connects two classes, Dijkstra's algorithm finds the shortest path between their border nodes.
- **Merged Paths:** If a path connects multiple classes, the algorithm builds a merged path connecting all of them. It works as follows:
  1. *Backbone Creation:* Dijkstra's algorithm is used to find the shortest path between every pair of classes. The longest path is chosen as the backbone.
  2. *Virtual Target Node:* Each remaining class is connected to the closest node of the backbone using the shortest path.
  3. *Backbone Expansion:* After a class is connected, the new branch of the path is added to the backbone. The remaining classes can connect to both the original backbone and the new branches.

Finally, the selected cells are converted into path tiles, while the others into empty tiles.

4.7 Summary

This chapter presented an approach for visualizing software systems by combining Multidimensional Scaling and Wave Function Collapse. MDS provides an initial layout by positioning classes according to their relationships, while WFC generates paths between them on a grid. The main goal is to preserve the connectivity of the software system while producing clear visualizations with limited overlaps and unnecessary crossings.

Adapting WFC to this problem requires more than defining tiles and their local constraints. The approach must also ensure that paths connect the intended classes and that the generation process prioritizes the connections that need to be drawn. To achieve this, relationships are merged when they share the same destination and type, and the observation strategy favors cells that contribute to path generation. Additional mechanisms guide paths towards their targets and help the algorithm recover when its decisions lead to contradictions.

These ideas define the overall approach, but their practical application also depends on how the different components interact and how the algorithm manages the generation process. The next chapter describes the implementation of the approach, including its main components and the mechanisms used to put it into practice.

Chapter 5

Implementation

In this chapter, we present the implementation details of Wassis, our tool developed to explore Wave Function Collapse for software system visualization.

5.1 Development Environment

Wassis is implemented in Pharo [30], a Smalltalk environment. This choice comes from the tradition of Smalltalk in software visualization and software analysis. Various tools such as Moose [31], CodeCity [9], and CodeCrawler [17] were developed using Smalltalk. This makes Pharo a suitable environment for a software visualization tool such as Wassis.

In Pharo, everything is an object. This makes it possible to represent the main concepts of Wassis, such as grids, cells, tiles, and paths, directly as objects. This keeps the implementation close to the conceptual model.

Finally, Pharo provides an interactive and reflective development environment with integrated inspector, debugging, and profiling tools. This was particularly useful for Wassis, as WFC continuously changes the state of cells during execution. For example, individual cells can be inspected to check their possible tiles, entropy, and neighbors during generation. This made it easier to understand and debug the generation process.

5.2 Software Architecture

Wassis is organized into two Pharo packages: `Wassis-Model`, which contains the core model and the algorithms, and `Wassis-Visualization`, which is responsible for rendering the results using Roassal [32]. The core architecture of Wassis is shown in Figure 5.1.

At the highest level, `WSAdaptiveSolver` manages the execution process and the grid size. It creates a new `WSSolver` for each attempt, starting from an initial grid size and increasing it when no valid solution is found. Therefore, each `WSSolver` represents one execution of WFC on a grid of a fixed size. This separation keeps the WFC algorithm independent of the logic used to increase the grid size during multiple attempts.

`WSSolver` is the main orchestrator of a single WFC execution. It operates on a `WSGraph`, which represents the structure on which WFC is executed. The graph is composed of `WSNode` objects, each representing a cell on the grid. A node stores information such as its identifier, position, entropy, and, most importantly, a collection of possible `WSTile` objects (empty, class, and path tiles). Hence, the `WSGraph`, `WSNode`, and `WSTile` classes are the core objects manipulated by WFC.

The topology is represented by `WSTopology`. It defines the structure of the graph, such as how nodes are arranged on the grid and how neighboring nodes are determined. This abstraction allows WFC to work with different grid structures, such as rectangular and hexagonal grids, without depending on a specific topology.

Contradictions are handled by the dedicated `WSBacktracking` class. Different recovery strategies are implemented as separate classes following the Strategy design pattern, allowing the solver to restore a previous consistent state or restart the execution without modifying its core logic.

A checkpoint of the graph state is saved using the Memento pattern. When a contradiction occurs, the solver uses this saved state to undo previous decisions and continue the search from an earlier point. This allows the solver to explore alternative tile assignments instead of restarting the entire process after every contradiction.

After a successful WFC execution, the result is then passed to the visualization package, which uses Roassal to render the generated grid.

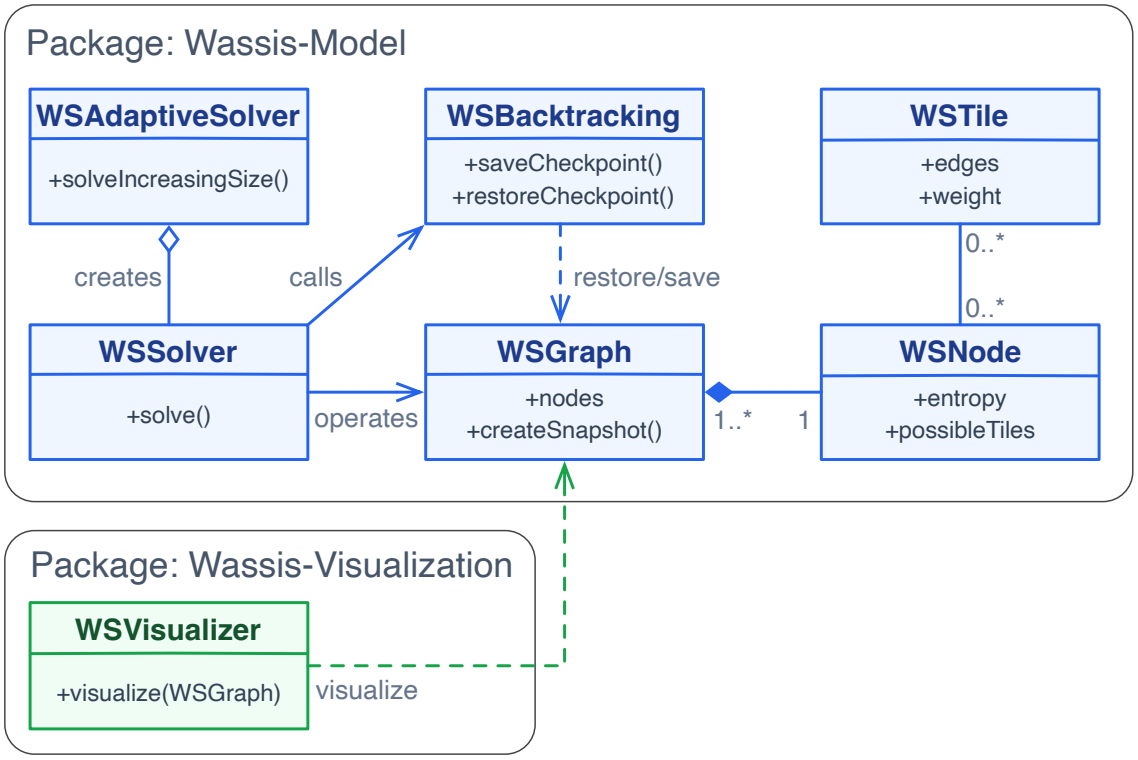


Figure 5.1: Core Architecture of Wassis.

5.3 Code Parsing

The source code parsing is performed using ZION, a tool developed within the REVEAL Research Group by Giannaccari *et al.* [33]. ZION uses CodeQL [40] to extract software entities, relationships, and properties from a source code repository. The extracted data is stored in a JSON file.

Wassis uses ZION's domain model to load the data and access its entities and relationships. These are then mapped to the internal representation used by Wassis. A `WSSoftwareSystem` contains the classes of the analyzed system, represented as `WSClass` objects. Each class stores its incoming and outgoing relationships, which represent its dependencies on other classes. This representation provides the information required by the layout and path generation phases, so the resulting model is used as input for the WFC algorithm.

5.4 Class Layout and Placement

The placement process converts the structural relationships between classes into positions on the grid. The implementation follows the steps described in Section 4.2: computing pairwise distances, obtaining a two-dimensional embedding using Multidimensional Scaling, mapping the resulting coordinates to grid positions, and placing the classes while respecting the defined constraints.

5.4.1 Distance Matrix and MDS

First, an $n \times n$ distance matrix is created, where n is the number of classes. The diagonal is initialized to zero, while all other entries are initialized to infinity. Direct relationships are then used to fill the distances between connected classes. The distance of a relationship is computed as the reciprocal of its weight (Equation (4.1)), so stronger relationships result in smaller distances.

After the direct distances have been added, the Floyd-Warshall algorithm [29] (presented in Algorithm 2) is used to compute the shortest distance between every pair of classes. This enriches the distance matrix with indirect relationships.

Algorithm 2 Floyd-Warshall

```
Initialize matrix D with direct relationship distances
for k = 1 to n do
    for i = 1 to n do
        for j = 1 to n do
            D[i, j] ← min(D[i, j], D[i, k] + D[k, j])
        end for
    end for
end for
```

Figure 5.2 shows an example of the distance matrix before and after applying Floyd-Warshall.

Classes that remain disconnected after Floyd-Warshall are assigned a distance larger than the graph diameter. This keeps disconnected classes farther from the others in the MDS layout.

Before running MDS, a power transformation can be applied to the distances. Each distance δ_{ij} is transformed into δ_{ij}^{p}, where p is a configurable parameter. When $p > 1$, larger distances are emphasized compared to smaller ones. When $0 < p < 1$, smaller distances are emphasized compared to larger ones.

The resulting distance matrix is given to MDS. The implementation uses the SMACOF iterative optimization process. It starts with random positions and updates them iteratively to reduce the difference between the target distances and the Euclidean distances between the classes. The process stops when the maximum number of iterations is reached or when the change in stress becomes smaller than the configured tolerance.

The result is a 2D position for each class. These positions are normalized to the range $[0, 1]$ using a global minimum ($v_{\min}$) and maximum ($v_{\max}$), considering all values across both dimensions.

Each coordinate v is normalized as follows:

$$v_{\text{norm}} = \begin{cases} 0.5 & \text{if } v_{\max} - v_{\min} = 0, \\ \frac{v - v_{\min}}{v_{\max} - v_{\min}} & \text{otherwise.} \end{cases} \tag{5.1}$$

If all classes collapse to the exact same position (*i.e.*, $v_{\max} - v_{\min} = 0$), all normalized coordinates are set to 0.5 as a fallback.

5.4.2 Mapping Positions to the Grid

The normalized coordinates are scaled to the grid size. For each dimension, the normalized value in $[0, 1]$ is multiplied by the current grid size n, and the result is rounded to the nearest integer.

This produces discrete integer coordinates for each class on the grid. Since MDS can place classes at identical or very close positions, the normalization, scaling, and rounding process may map multiple classes to the same target position.

5.4.3 Class Placement

Classes are placed on the grid according to the positions computed by MDS. Since a class can occupy multiple cells, its area is represented as a `WSFootprint`. The footprint contains the nodes occupied by the class, its top-left node, and its side length. A class is always represented by a square footprint.

	Object	ProtoObject	MCReader	MCStReader	MCVersionReader	MCMczReader	MCFileTreeAbstractReader	MCFileTreeStReader	MCFileTreeStSnapshotReader	MCFileTreeStCypressReader	IceLibgitFiletreeReader	IceLibgitFiletreeLogReader	TonelReader	IceLibgitTonelReader	IceLibgitTonelLogReader
Object	0	1.0	1.0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
ProtoObject	1.0	0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCReader	1.0	Inf	0	1.0	1.0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCStReader	Inf	Inf	1.0	0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCVersionReader	Inf	Inf	1.0	Inf	0	1.0	Inf	Inf	Inf	Inf	Inf	Inf	1.0	Inf	Inf
MCMczReader	Inf	Inf	Inf	Inf	1.0	0	1.0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCFileTreeAbstractReader	Inf	Inf	Inf	Inf	Inf	1.0	0	1.0	1.0	Inf	Inf	Inf	Inf	Inf	Inf
MCFileTreeStReader	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCFileTreeStSnapshotReader	Inf	Inf	Inf	Inf	Inf	Inf	1.0	Inf	0	1.0	Inf	Inf	Inf	Inf	Inf
MCFileTreeStCypressReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	1.0	Inf	Inf	Inf	Inf
IceLibgitFiletreeReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	1.0	Inf	Inf	Inf
IceLibgitFiletreeLogReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	Inf	Inf	Inf
TonelReader	Inf	Inf	Inf	Inf	1.0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	0	1.0	Inf
IceLibgitTonelReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	1.0
IceLibgitTonelLogReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	1.0	0



	Object	ProtoObject	MCReader	MCStReader	MCVersionReader	MCMczReader	MCFileTreeAbstractReader	MCFileTreeStReader	MCFileTreeStSnapshotReader	MCFileTreeStCypressReader	IceLibgitFiletreeReader	IceLibgitFiletreeLogReader	TonelReader	IceLibgitTonelReader	IceLibgitTonelLogReader
Object	0	1.0	1.0	2.0	2.0	3.0	4.0	5.0	5.0	6.0	7.0	8.0	3.0	4.0	5.0
ProtoObject	1.0	0	2.0	3.0	3.0	4.0	5.0	6.0	6.0	7.0	8.0	9.0	4.0	5.0	6.0
MCReader	1.0	2.0	0	1.0	1.0	2.0	3.0	4.0	4.0	5.0	6.0	7.0	2.0	3.0	4.0
MCStReader	2.0	3.0	1.0	0	2.0	3.0	4.0	5.0	5.0	6.0	7.0	8.0	3.0	4.0	5.0
MCVersionReader	2.0	3.0	1.0	2.0	0	1.0	2.0	3.0	3.0	4.0	5.0	6.0	1.0	2.0	3.0
MCMczReader	3.0	4.0	2.0	3.0	1.0	0	1.0	2.0	2.0	3.0	4.0	5.0	2.0	3.0	4.0
MCFileTreeAbstractReader	4.0	5.0	3.0	4.0	2.0	1.0	0	1.0	1.0	2.0	3.0	4.0	3.0	4.0	5.0
MCFileTreeStReader	5.0	6.0	4.0	5.0	3.0	2.0	1.0	0	2.0	3.0	4.0	5.0	4.0	5.0	6.0
MCFileTreeStSnapshotReader	5.0	6.0	4.0	5.0	3.0	2.0	1.0	2.0	0	1.0	2.0	3.0	4.0	5.0	6.0
MCFileTreeStCypressReader	6.0	7.0	5.0	6.0	4.0	3.0	2.0	3.0	1.0	0	1.0	2.0	5.0	6.0	7.0
IceLibgitFiletreeReader	7.0	8.0	6.0	7.0	5.0	4.0	3.0	4.0	2.0	1.0	0	1.0	6.0	7.0	8.0
IceLibgitFiletreeLogReader	8.0	9.0	7.0	8.0	6.0	5.0	4.0	5.0	3.0	2.0	1.0	0	7.0	8.0	9.0
TonelReader	3.0	4.0	2.0	3.0	1.0	2.0	3.0	4.0	4.0	5.0	6.0	7.0	0	1.0	2.0
IceLibgitTonelReader	4.0	5.0	3.0	4.0	2.0	3.0	4.0	5.0	5.0	6.0	7.0	8.0	1.0	0	1.0
IceLibgitTonelLogReader	5.0	6.0	4.0	5.0	3.0	4.0	5.0	6.0	6.0	7.0	8.0	9.0	2.0	1.0	0

Figure 5.2: Distance matrix before and after Floyd-Warshall.

For each class, the algorithm first tries to use its target position. A position is valid only if the footprint fits in the grid and the required distance from other classes is maintained. If the target position is not valid, the algorithm searches for candidates in expanding rings around the target position, starting from the first ring. For each ring, all valid positions are collected. If at least one valid position is found, the search stops and the best candidate among those positions is selected.

When multiple candidates are found in the same ring, the candidate with the smallest total Manhattan distance to already placed connected classes is selected. The Manhattan distance between two classes is computed from their border cells. If the class has no connected classes placed yet, one of the valid candidates is selected at random. If no valid position is found in a ring, the algorithm checks the next ring. If no valid position can be found, the placement fails, the grid is expanded, and the placement process restarts.

After all classes have been placed, the nodes in each footprint are converted into class tiles.

5.4.4 Path Exit Assignment

Once a class has been placed, its border cells are used as entry and exit points for its paths. These exits are assigned before running WFC, as they provide the constraints needed to generate paths between classes. Without these, WFC would have no information about where paths should connect to the class footprints.

The exit assignment is performed in two steps. First, the algorithm computes a preference order for each path ID of each class. Second, it assigns each path ID to a specific border cell based on these preferences.

Directional Preference Calculation For each path ID in the system, the algorithm computes a target position as the centroid of all classes connected by that path ID. For this calculation, the position of a class is defined as the center of its footprint. If the footprint has an even side length, one of its four central cells is selected at random.

Then, for each path ID of each class, a vector $\mathbf{A}$ is computed from the center of the class to the target position of the path ID. This vector represents the ideal direction in which the path should leave the class.

The algorithm compares $\mathbf{A}$ with the four directions: up $(0, 1)$, down $(0, -1)$, right $(1, 0)$, and left $(-1, 0)$, using cosine similarity:

$$\text{cosineSimilarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}||\mathbf{B}|}. \tag{5.2}$$

Cosine similarity measures how well a direction matches the direction towards the target. Its value ranges from -1 to 1. A value close to 1 means that the two directions are aligned, a value close to -1 means that they point in opposite directions, and a value of 0 means that they are orthogonal. Since the algorithm uses costs that have to be minimized, cosine similarity is converted into a cost:

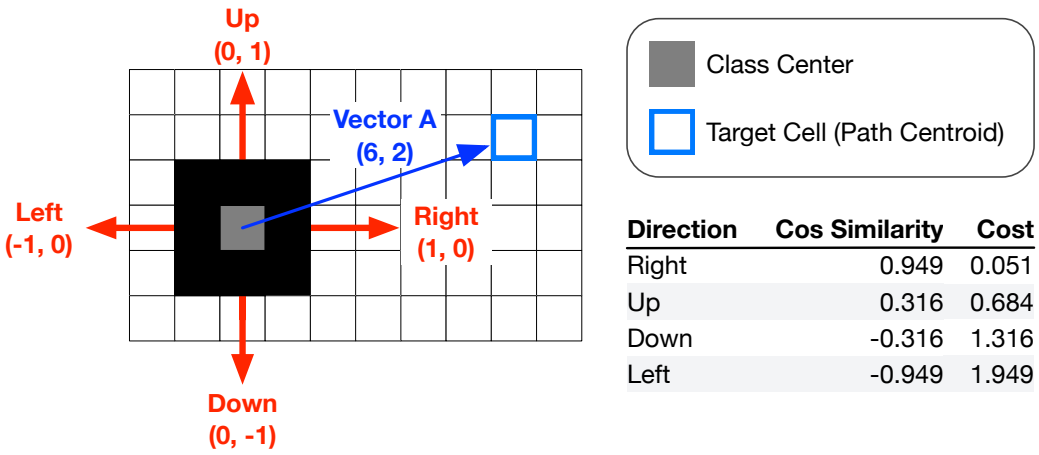
$$\text{cost}(\mathbf{A}, \mathbf{B}) = 1 - \text{cosineSimilarity}(\mathbf{A}, \mathbf{B}). \tag{5.3}$$

Therefore:

- cost $= 0$ represents a perfect alignment (cosineSimilarity $= 1$);
- cost $= 1$ represents an orthogonal direction (cosineSimilarity $= 0$);
- cost $= 2$ represents the worst direction (cosineSimilarity $= -1$).

The four grid directions are then sorted by increasing cost. This produces an ordered list of preferred directions for each path ID of a class. The first direction is therefore the preferred one.

An example is provided in Figure 5.3. Since the source class has a 3×3 footprint, its center cell is used as the reference position. The target position (path centroid) results in a vector $\mathbf{A} = (6, 2)$ from the class center. Comparing this vector with the four grid directions gives the following costs: up $= 0.684$, left $= 1.949$, right $= 0.051$, and down $= 1.316$. The directions are therefore ordered as right, up, down, and left.



Direction	Cos Similarity	Cost
Right	0.949	0.051
Up	0.316	0.684
Down	-0.316	1.316
Left	-0.949	1.949

Figure 5.3: Example of preferred direction of a path ID calculation.

Border Cell Assignment After computing the preferred directions, each path is assigned to a specific border cell of the class footprint. Each path has an ordered list of preferred directions, from the most to the least preferred one. The overall process is illustrated in the workflow of Figure 5.4, while a concrete execution example is shown in Figure 5.5. The assignment is performed as follows:

1. Paths are first grouped by their primary preferred direction. Each group corresponds to one side of the class footprint, and paths in the same group compete for the cells available on that side.

2. Groups with more paths are processed first, and this order never changes. Within each group, paths are ordered by their primary direction cost. Paths with lower costs are assigned first. This gives priority to the paths that have a stronger preference for that side when there are more paths than available cells.

3. For each path in a group, its secondary direction is used to determine its position along the side. The path is assigned to the first available cell in the direction of its secondary preference. For example, if the primary direction is *right* and the secondary direction is *up*, the path is assigned to the highest available cell on the right side (*i.e.,* the cell with the minimum row index).

4. If all cells on a side are already assigned, the remaining paths are moved to the group corresponding to their next preferred direction. When a path is moved to a new group, the direction of the side where it was originally placed becomes its secondary direction. More generally, the secondary direction is the first preferred direction that is different from the direction of the group. For example, if a path has the preference order *up*, *left*, *down*, *right*, but there is no available cell in the *up* group, it is moved to the *left* group. In this case, *up* becomes its secondary direction and is used to choose its position on the left side. The cost of the direction to which the path is moved is then set to zero. This gives the moved path priority within the new group, ensuring that it is assigned before paths that originally preferred that direction. This prevents the path from not being able to compete in the new group and being moved to another direction again.

5. The process is repeated from point 3 until all paths have been assigned. As the footprint size of each class is computed from its total number of connections, enough border cells are available to place all its paths.

This approach keeps paths with similar directions close to each other and helps reduce crossings immediately after leaving the class.

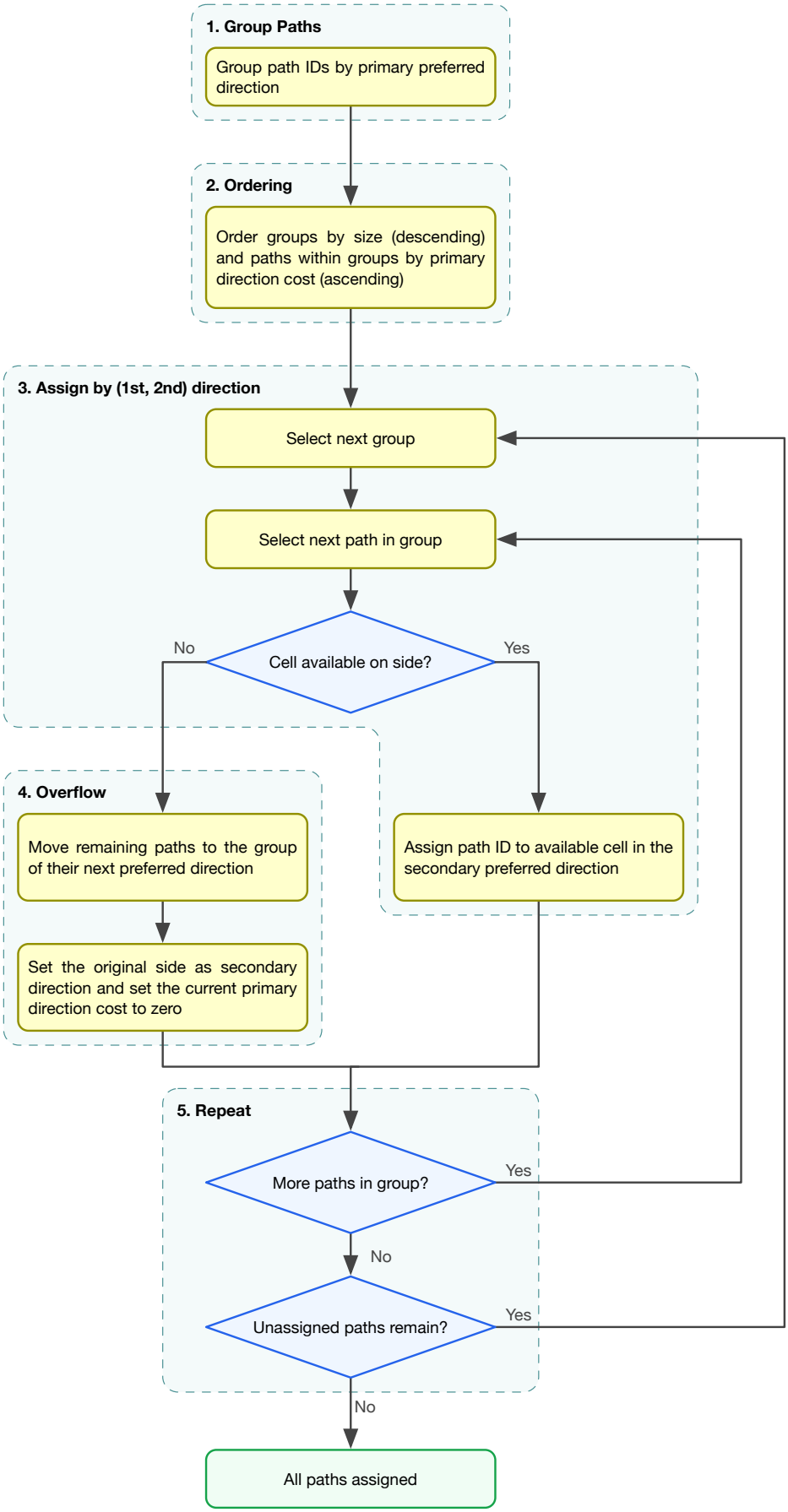


Figure 5.4: Workflow of the border cell assignment process.

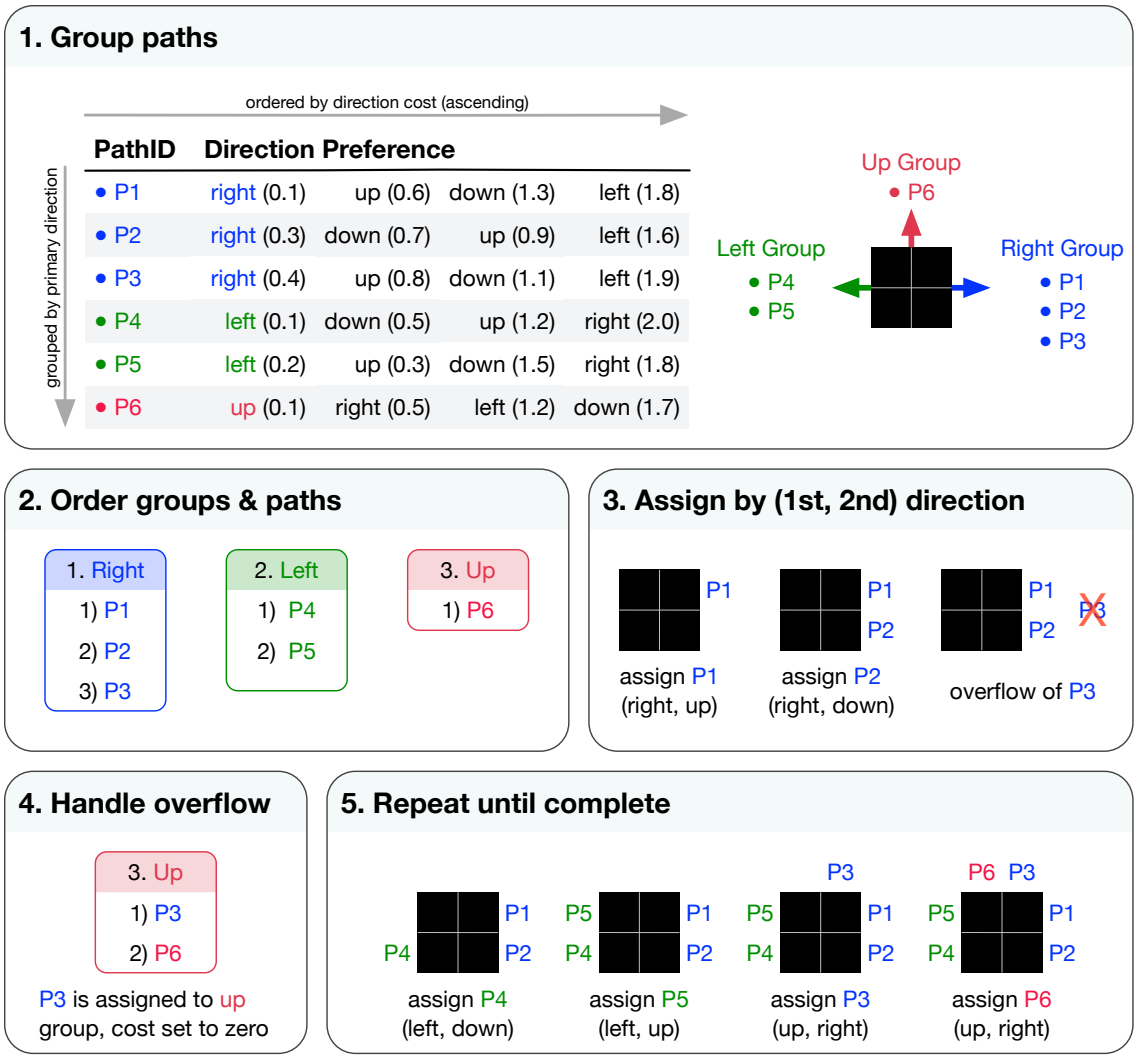


Figure 5.5: Example of the border cell assignment process.

5.5 Wave Function Collapse

The WFC implementation in Wassis follows the general process described in Section 3.1, but introduces some implementation mechanisms to improve the efficiency of the path generation process. This section describes these mechanisms, including heap management, dynamic tile weights, path ID resolution, constraint propagation, and backtracking.

5.5.1 Heap Management

During the observation phase of WFC, the next cell to collapse is selected as the cell with the highest entropy, as described in Subsection 4.4.1. Scanning the whole grid at every step would require $O(N)$ time, where N is the total number of cells. To avoid this cost, Wassis uses a max binary heap containing `WSHeapEntry` objects. The root of the heap contains the cell with the highest entropy, which can be accessed in $O(1)$ time, while removing it requires $O(\log M)$ time, where M is the current size of the heap. The overall process is shown in Figure 5.6.

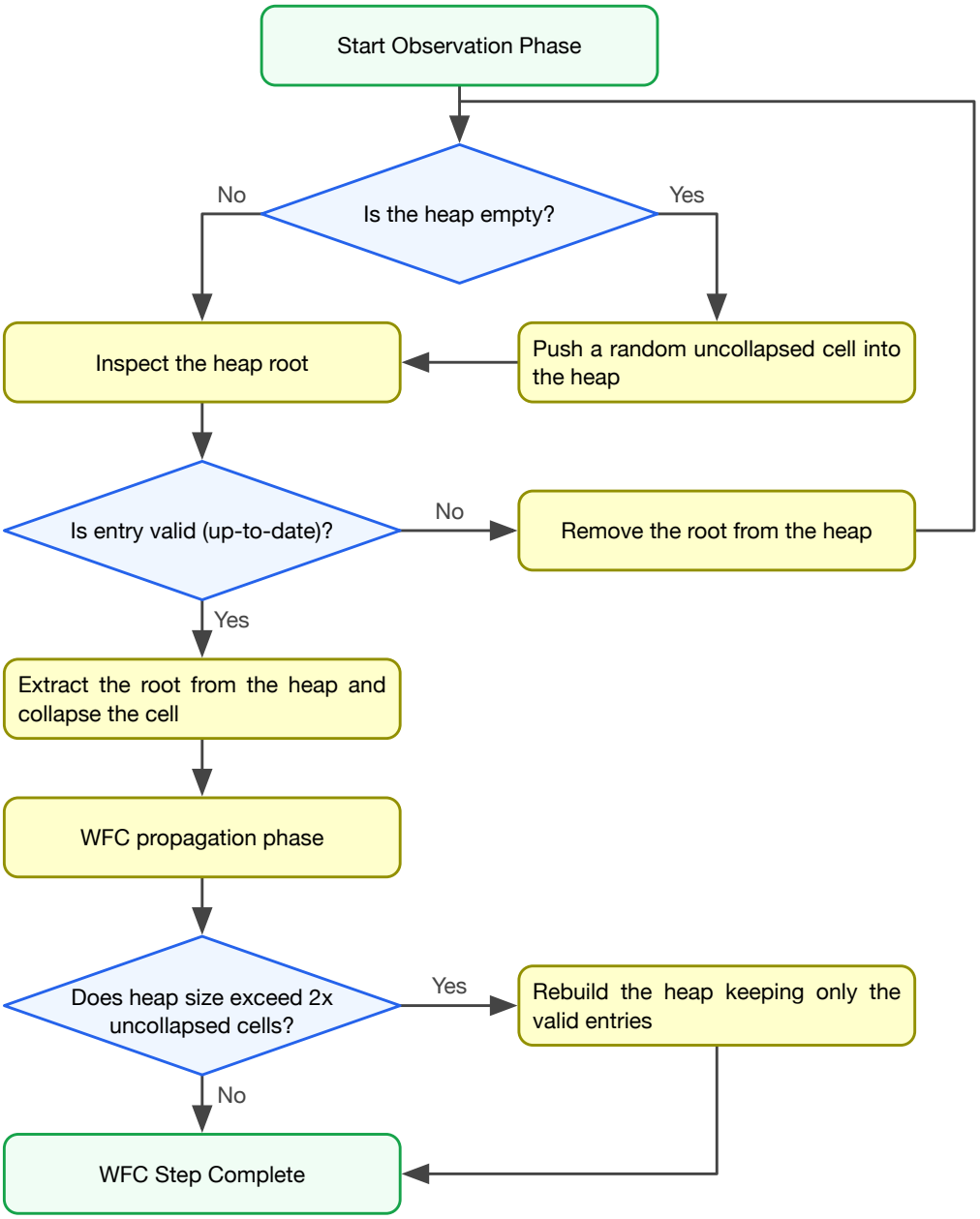


Figure 5.6: Next cell selection workflow.

Initial Insertion The heap does not initially contain all grid cells. Only cells occupied by software classes are inserted, as they are the starting points of the WFC generation. Untouched cells are not inserted because initially they all have the same set of possible tiles and therefore the same entropy.

After a propagation step, the cells that were touched by propagation are added to the heap. These cells have fewer possible tiles and therefore a different entropy from untouched cells. Since WFC starts from the class cells and propagation only affects part of the grid at each step, only these cells are added to the heap, keeping its size relatively small.

A cell may be affected multiple times during the same propagation phase. Inserting it into the heap after every change would result in unnecessary operations, especially since propagation is the most computationally intensive part of WFC. Therefore, affected cells are first collected and inserted into the heap only once, after the propagation phase is complete.

Lazy Heap Entries Updating a cell that is already in the heap is expensive with the heap implementation available in Pharo. Finding a specific element inside the heap requires $O(M)$ time. Instead of removing and reinserting a node every time its entropy changes, WASSIS uses `WSHeapEntry` objects.

A `WSHeapEntry` stores a reference to a `WSNode`, together with the entropy and the number of possible tiles of that node at the time the entry is created. As a result, more than one entry referring to the same node can exist in the heap. Older entries are not immediately removed when the node changes.

When the root of the heap is selected, the stored values are compared with the current values of the corresponding `WSNode`. If they do not match, the entry is no longer valid and is removed from the heap in $O(\log M)$ time. The next entry is then checked in the same way until a valid entry is found for collapse.

The number of possible tiles is stored in addition to the entropy because some tiles have weight zero. These tiles do not contribute to the entropy calculation; therefore, entropy alone is not sufficient to determine whether a heap entry represents the current state of a node.

An example of a lazy heap entry is provided in Figure 5.7.

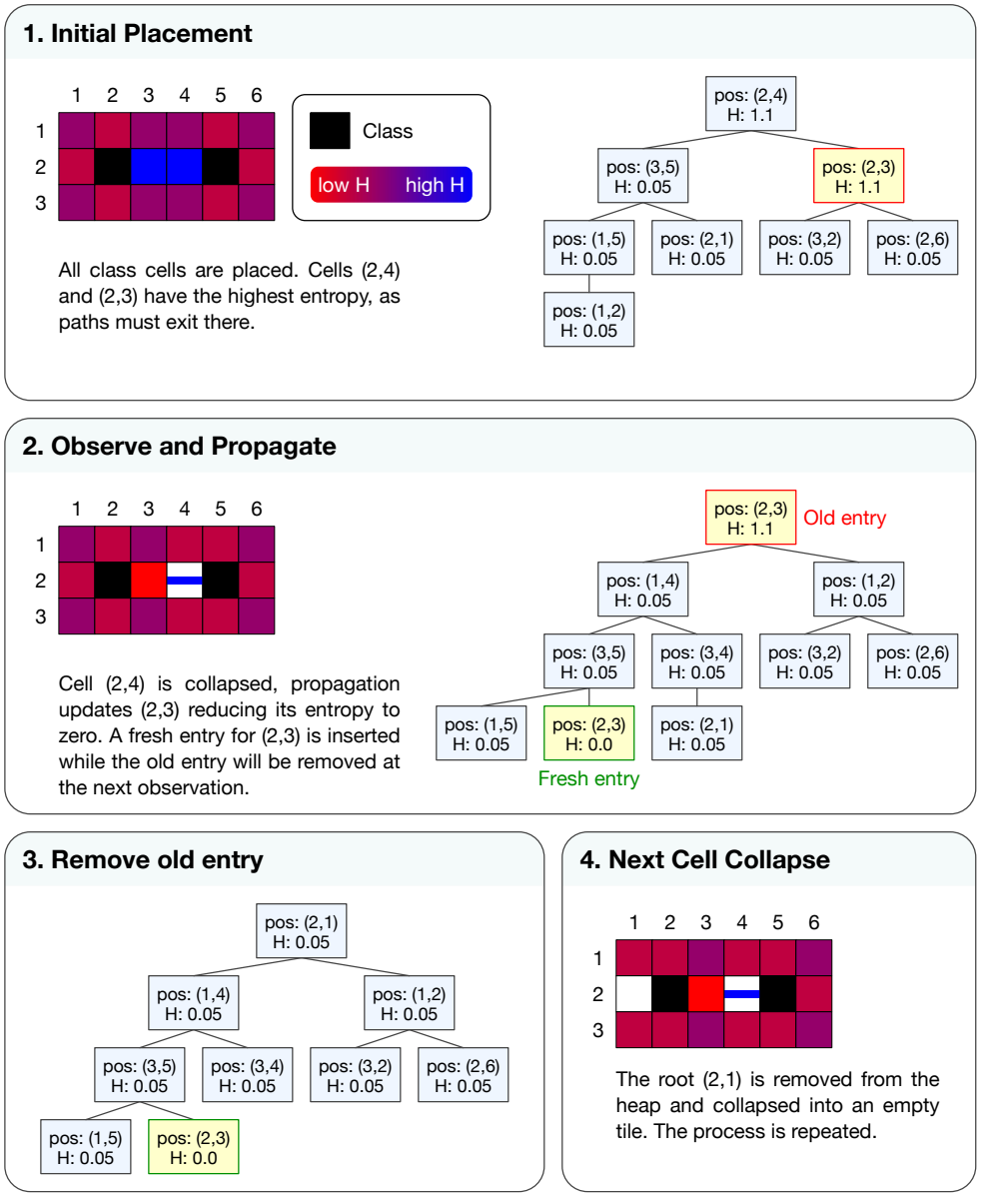


Figure 5.7: Lazy heap entry example.

Heap Compaction and Empty Heap Handling The use of old heap entries means that the heap can contain entries that are no longer valid. To prevent the heap from becoming too large, after each propagation phase WASSIS compares the heap size with the number of uncollapsed cells. If the heap contains more than twice as many entries as there are uncollapsed cells, it is rebuilt. The rebuild removes old entries and keeps only valid entries of uncollapsed cells.

Finally, because cells are added to the heap only after being touched by propagation, the heap can become empty even though some cells are still uncollapsed. This can happen when the propagation step does not affect any new cell. In this case, a random uncollapsed cell is selected and added to the heap. This is valid because all cells that have never been touched have the same set of possible tiles and therefore the same entropy. The random choice allows the execution to continue until propagation reaches other cells.

5.5.2 Dynamic Placeholder Resolution

A *resolved tile* is a path tile whose edges contain concrete path IDs instead of placeholders. When a node is collapsed to a path tile, WASSIS checks its already collapsed neighbors for a resolved path tile. If one is found, its concrete path ID is used to replace the placeholders of the new collapsed tile. If no path ID is available, the tile remains unchanged.

If the new collapsed node is resolved, WASSIS then propagates its path ID to already collapsed neighboring nodes containing placeholder tiles. The resolution uses a queue starting from the collapsed node. For each node in the queue, its collapsed neighbors are checked. If a neighbor contains a placeholder tile and its path ID can be determined from the current node, the placeholder is replaced with a concrete tile. Successfully resolved nodes are added to the queue so that the path ID can continue to propagate through adjacent collapsed nodes.

After this resolution process, WASSIS checks the edges of the resolved tiles against their collapsed neighbors. If two adjacent resolved tiles have incompatible edges, a contradiction is signaled.

5.5.3 Dynamic Tile Weights

To guide paths toward their targets, WASSIS uses dynamic tile weights in addition to fixed path exits. When a cell is selected for collapse, its possible tiles are assigned weights based on the direction of the target for the weighted choice. If the cell has a path target, WASSIS increases the weight of tiles whose shape points toward that target.

Path target information is propagated in the same way as path IDs. Initially, target information is associated with the classes placed on the grid. When a path ID is propagated to a resolved path tile, the corresponding target information is also propagated. This allows cells along the path to know the target associated with the path they will collapse to.

On a 2D grid, WASSIS compares the position of the current cell with the position of the target along the two axes. The distance along each axis is used to determine which directions lead to the target. The boost per direction depends on how much of the remaining distance is along that axis.

As illustrated in Figure 5.8, if the current cell is at position $(4, 4)$ and the target is at position $(2, 7)$, the remaining distance is 2 cells along the vertical axis and 3 cells along the horizontal axis, for a total of 5. Therefore, the horizontal direction receives a larger boost than the vertical direction. The boost is controlled by a value chosen by the user. For example, with a boost value of 10, the vertical direction receives a factor of $10 \cdot \frac{2}{5} = 4.0$, while the horizontal direction receives a factor of $10 \cdot \frac{3}{5} = 6.0$. If a tile has an original weight of 2 and points in the horizontal direction, its new weight becomes $2 \cdot 6.0 = 12.0$. If boost is 0, the tile keeps its original weight. Zero-weight tiles are unaffected by the boost.

When a tile can help guide the same path in more than one direction, only the highest boost is used. For example, if a tile points both vertically and horizontally toward a target, with boost factors of 4.0 and 6.0 respectively, the tile receives a boost of 6.0. This prevents the tile from receiving too large a boost just because it has multiple useful directions for the same path.

If a tile can help guide multiple paths, the highest boost is selected for each path. These boosts are then multiplied together. The original tile weight is multiplied by the resulting value. This makes tiles that help guide multiple paths more likely to be selected.

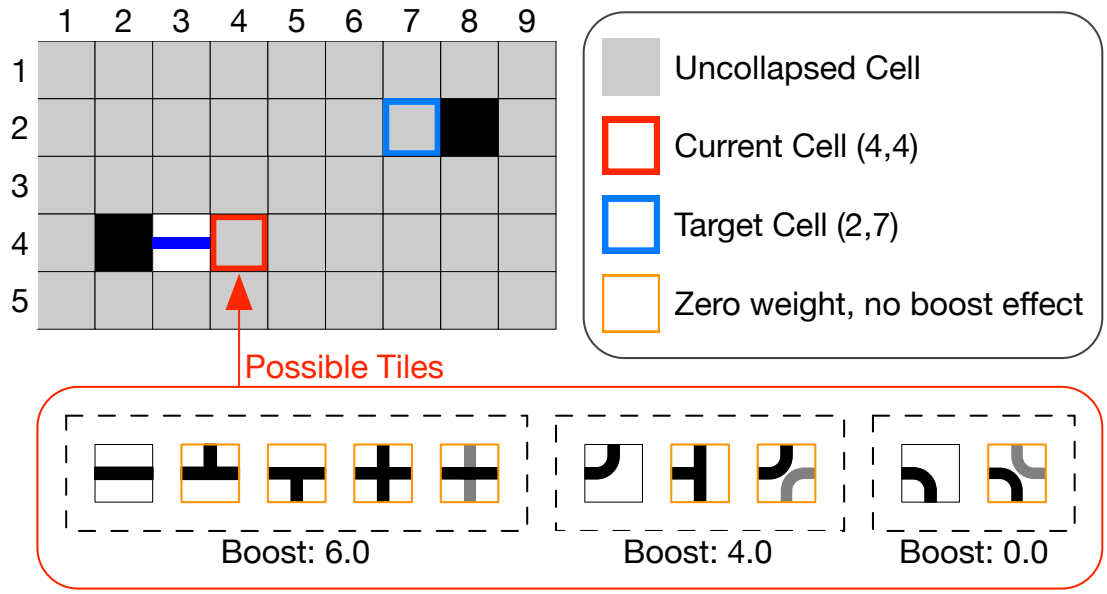


Figure 5.8: Example of tile weight boost.

5.5.4 Constraint Propagation

After the collapse of a node and the path ID resolution step, constraint propagation is performed. First, propagation starts from the new collapsed node and uses a FIFO queue. The initial node is added to the queue, and nodes are processed until the queue is empty. For each node, its neighbors are examined. Already collapsed neighbors are skipped, since their tile is already fixed and cannot be changed. For each uncollapsed neighbor, the possible tiles are reduced according to the current constraints. A tile is kept only if the symbol on its edge facing the current node is compatible with the corresponding symbol of the tile selected for the current node.

If one or more tiles are removed from a neighbor, its entropy is updated and the neighbor is added to the queue so that the updated constraints can be propagated recursively to its neighbors.

If a neighbor loses all its possible tiles, a contradiction is detected and the backtracking process described in Subsection 4.6.4 starts.

The propagation process keeps track of all nodes whose possible tiles have changed. Once the queue is empty, these nodes are used to update the max-heap described in Subsection 5.5.1. Updating the heap only once at the end of propagation avoids repeated heap operations if the same node is affected multiple times during the same propagation step.

5.5.5 Backtracking

To make backtracking possible, the state of the grid must be saved at different points during the generation. The `WSBacktracking` class decides when to create a checkpoint depending on the selected backtracking strategy. The checkpoint is evaluated after a cell has been selected for collapse but before a tile is actually chosen. This allows the checkpoint to represent the exact state from which the current decision is made. If the decision later leads to a contradiction, the state can be restored and the tile originally selected at that decision point can be removed from the cell's possible tiles before collapsing it again.

The initial implementation used a deep copy of the `WSGraph` object. However, this was very expensive and had a significant impact on performance. To avoid copying the entire object, the Memento pattern was used. The `WSGraph` and `WSNode` classes provide methods to create snapshots of their current state, represented by `WSGraphSnapshot` and `WSNodeSnapshot`. These snapshots store only the mutable state needed to restore the current state. A `WSGraphSnapshot` contains the snapshots of its nodes and a copy of the heap, reducing the cost of saving checkpoints.

It is then stored in a `WSCheckpoint` along with other information (*e.g.,* a timestamp), while `WSBacktracking` keeps the collection of available checkpoints.

When a checkpoint has to be restored, `WSBacktracking` asks the `WSGraph` to restore its state from the corresponding snapshot. This approach avoids the expensive deep copy of the whole graph and improves performance at the cost of additional memory for the stored snapshots.

While the other backtracking strategies are relatively simple to evaluate, the *similar weights* strategy requires a more complex check. With the fixed and percentage strategies, a checkpoint is created at every step. The *many possible tiles* strategy only requires checking if the number of possible tiles is above the selected threshold, while the *restart* strategy does not create checkpoints.

The *similar weights* strategy instead requires checking whether a sufficient number of possible tiles have similar weights. The user defines the minimum percentage of tiles that must have similar weights and the maximum allowed difference between their weights, also expressed as a percentage.

To perform this check, tiles with zero weight are first ignored and the remaining weights are sorted. A sliding window is then used to examine groups of the required size. For each group, the difference between the smallest and largest weight is compared with the allowed difference. If at least one group satisfies this condition, a checkpoint is created. The sliding window allows all groups of the required size to be checked efficiently after sorting the weights.

5.6 Path Optimization

Wassis uses Dijkstra's algorithm during the optional path optimization step described in Subsection 4.6.5. Instead of implementing the algorithm from scratch, Wassis uses the `AIDijkstra` implementation provided by Pharo's `AIGraphAlgorithm` package.

For direct connections, Dijkstra is used to find the shortest path between the border nodes of two classes. For merged paths, a class must be connected to the existing backbone. Since the class can connect to any node of the backbone, Wassis uses a virtual target node to find the shortest path to the backbone.

The virtual target is an additional node that does not correspond to any grid cell. A directed edge with cost zero is added from every node belonging to the current backbone to this virtual target. Dijkstra is then run from each border node of the class to the virtual target. Since reaching the virtual target has zero cost from every backbone node, the shortest path found corresponds to the shortest path from that border node to the backbone.

The paths obtained from the border nodes are compared, and the shortest is selected.

5.7 Memory Management

During WFC generation, Wassis creates many temporary objects, such as graph and node instances, heap entries, and snapshots. Pharo stores all active objects in memory. Therefore, repeated WFC attempts can cause memory usage to grow over time as temporary objects from previous attempts become unused.

To avoid this problem, Wassis runs a garbage collection after each failed attempt that requires a grid expansion. This allows unused objects from the previous attempt to be removed from memory before the next generation starts.

The garbage collection has a small performance cost, but prevents memory usage from increasing indefinitely across multiple WFC attempts.

5.8 Parallel Execution

Pharo is a single-threaded environment and does not support true multithreading. This is a limitation for Wassis, since running multiple WFC generations in parallel can be useful to obtain different valid solutions and compare their metrics. It can also be used to explore different generation choices, which may lead to different valid or better-looking solutions.

To overcome this limitation, Wassis uses multiple independent Pharo images to achieve parallel execution. Shell scripts are provided to start multiple instances of the same Pharo image.

Each instance runs an independent WFC generation. This allows multiple generations to run in parallel without requiring changes to the WFC algorithm to support multithreading.

Two execution modes are available. In the first, all instances run until they complete. Each solution is serialized and saved in a separate folder, so it can be loaded later from another Pharo image. Serialization is implemented using Fuel [41].

The second mode is designed to find a valid solution as quickly as possible. All instances run simultaneously, and the execution stops when one of them finds a solution. The winning instance sends its process identifier to the shell script through a pipe. The script then terminates the other Pharo processes and opens the winning image to display the solution.

This approach provides parallel execution while keeping each WFC generation single-threaded.

5.9 Tool

To make Wassis easier to use and to inspect the generated visualizations, we developed a GUI in the Pharo environment. This section describes the main parts of the tool, including class selection, configuration, the main and additional visualizations.

5.9.1 Class Selection and Configuration Setup

The procedure starts with the panel shown in Figure 5.9. Here, users can search and select a class or component from the target system. They can also choose which relationships to visualize and set the number of instances to run in parallel.

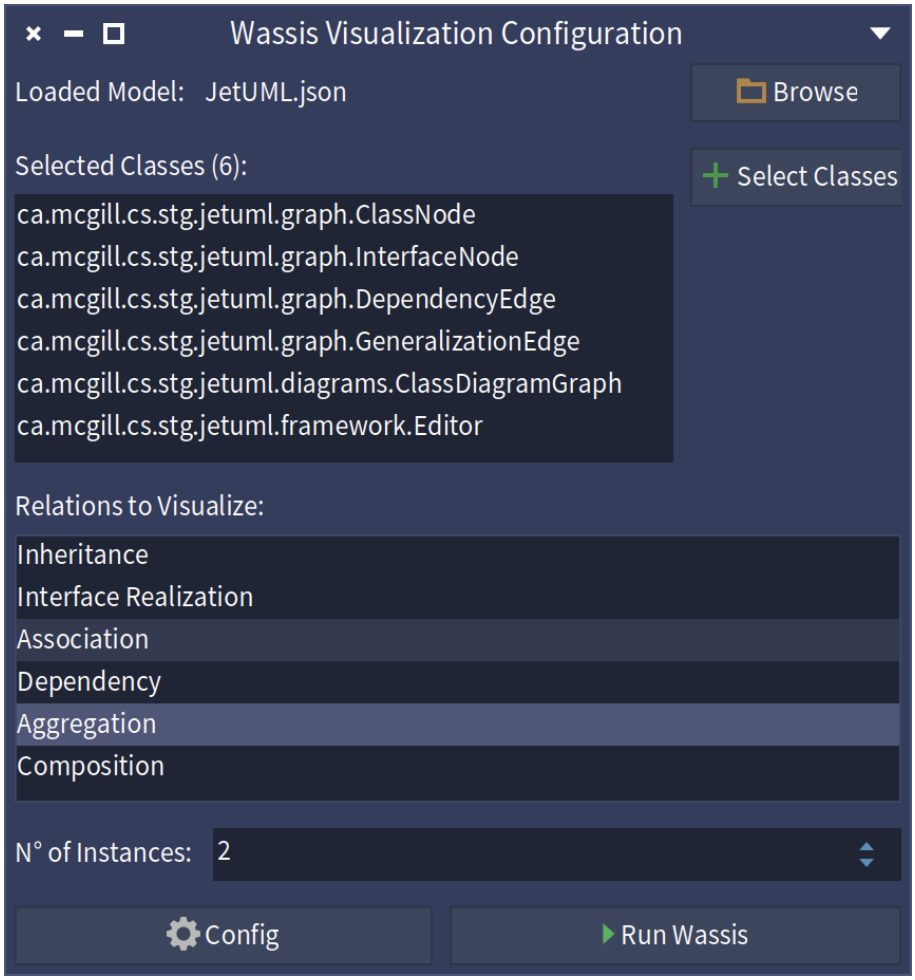


Figure 5.9: Wassis visualization configuration.

The `Config` button opens the settings window shown in Figure 5.10. The settings are divided into four groups:

- **General Configuration:** Controls the heap type, the class placement order, and the minimum desired distance between classes.
- **Path Configuration:** Sets the exit assignment strategy for the class footprints and the boost value applied to all path directions.
- **Tile Weights:** Sets the weights for empty, straight, and corner tiles.
- **Backtracking Configuration:** Sets the retry limits for backtracking on the same graph, same class placement, and same grid size. It also provides different backtracking strategies and their specific parameters.

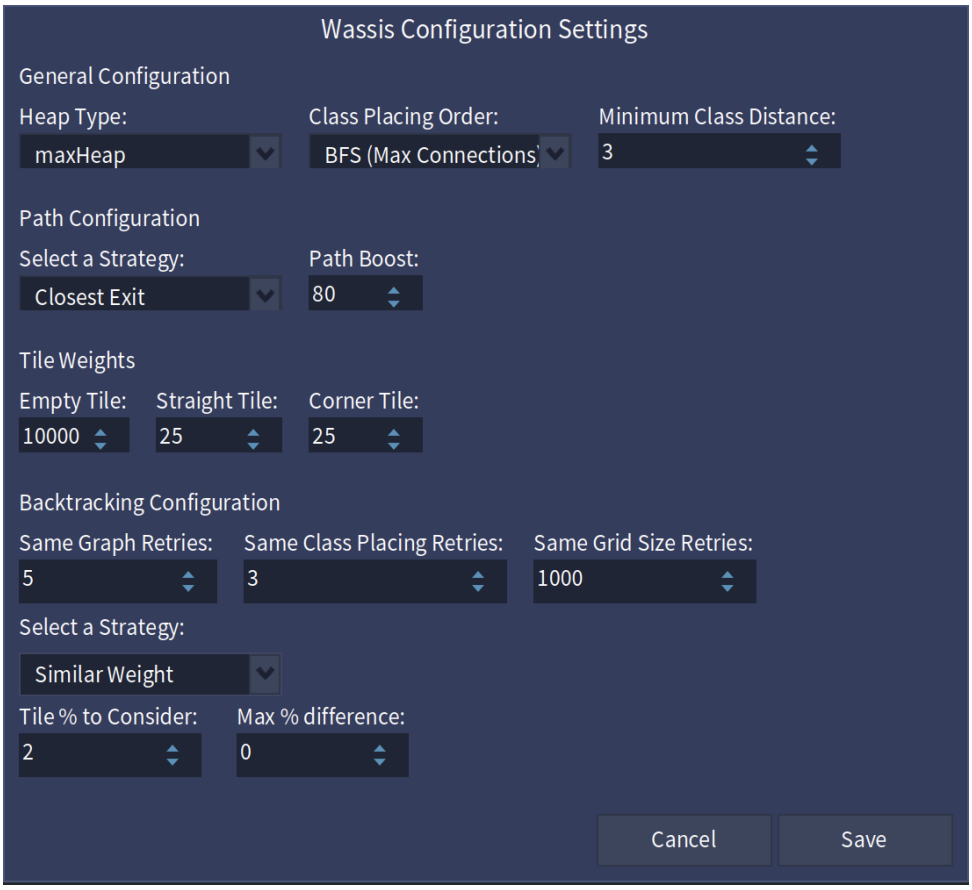


Figure 5.10: Wassis configuration settings window.

5.9.2 Visualization and Additional Views

Once a valid solution is found, Wassis displays the generated 2D visualization on an interactive canvas using Roassal (Figure 5.11). Users can inspect the objects using the Pharo Inspector, save the solution using Fuel, and open additional views.

The additional views include the step-by-step animation, the metrics dashboard, the system graph, the distance matrix, and the MDS placement.

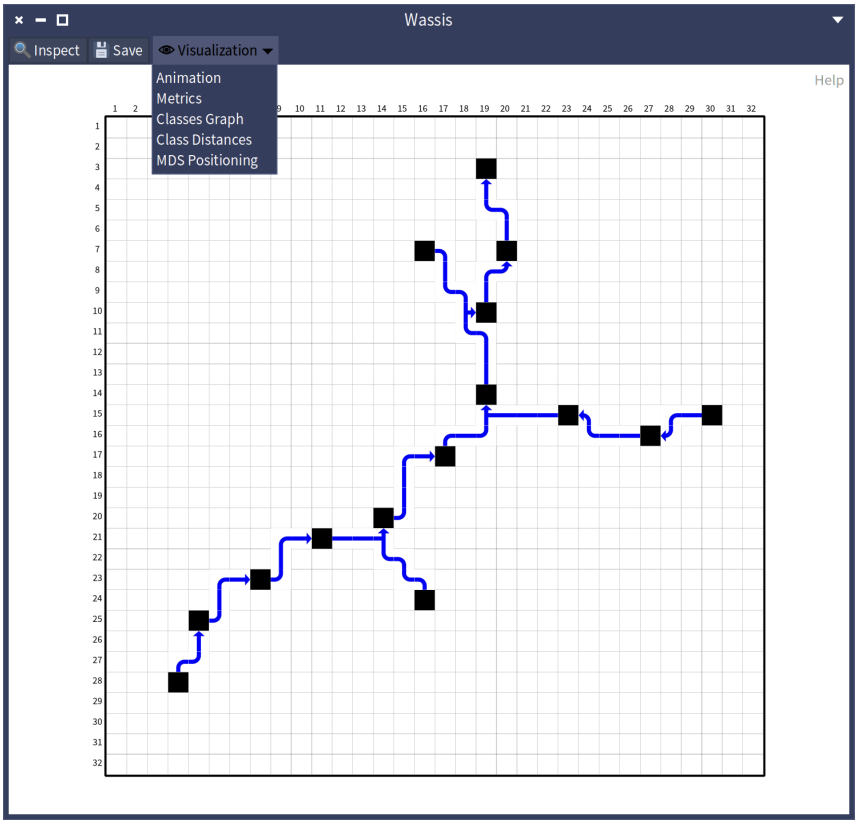


Figure 5.11: Main grid visualization.

5.9.3 Step-by-Step Animation

Wassis provides a step-by-step animation to inspect the WFC generation process (Figure 5.12). The animation shows the state of the grid at each step, from start to finish. The `Reset`, `Step Back`, `Play`, and `Step Forward` buttons allow users to move through the generation process, along with the slider.

5.9.4 Metrics Dashboard

Wassis includes a metrics dashboard with several tabs. Each tab provides information about a different part of the generated layout and the WFC execution:

- **Graph Metrics** (Figure 5.13a): Shows the grid size, the number of class cells, path cells, and empty cells in the grid, as well as the number of overlaps.
- **Performance Metrics** (Figure 5.13b): Shows the total execution time, the WFC solving time, and the number of attempts needed to find the solution. The total execution time is measured from the moment the user clicks `Run` until the final visualization is displayed. The WFC solving time refers only to the time spent by WFC to generate the solution in the successful attempt.
- **Placement Metrics** (Figure 5.13c): Shows the average distance between each class and its connected classes, as well as the pairwise distances between classes on the grid, computed using Manhattan distance.
- **System Metrics** (Figure 5.13d): Shows the total number of classes, the average number of connections per class, and the number of connections of each class.



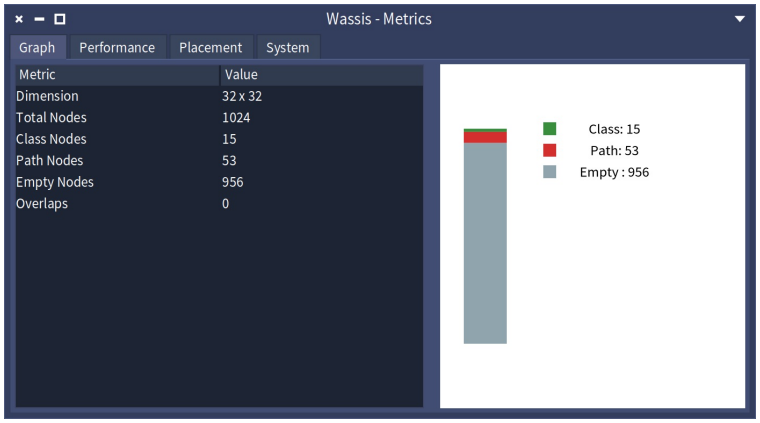
Figure 5.12: WFC step-by-step animation window.

5.10 Summary

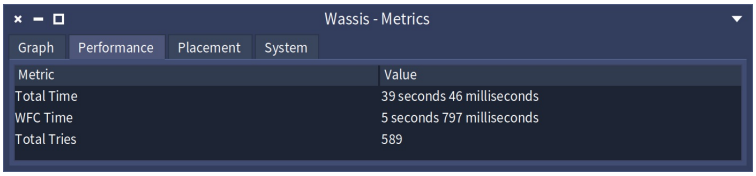
This chapter presented the implementation of Wassis, a Pharo tool that applies the approach described in the previous chapter to software system visualization. The implementation combines class placement, path generation, and visualization in a single tool, using the software model extracted by ZION as input.

A key implementation challenge was making WFC practical for path generation. The solver includes mechanisms to manage cell selection efficiently, resolve path identifiers during generation, guide paths towards their targets, and recover from contradictions. Additional mechanisms address memory usage and allow multiple independent generations to run in parallel. The tool also provides an interface for configuring the algorithm, inspecting the generated visualizations, and examining the generation process through additional views and metrics.

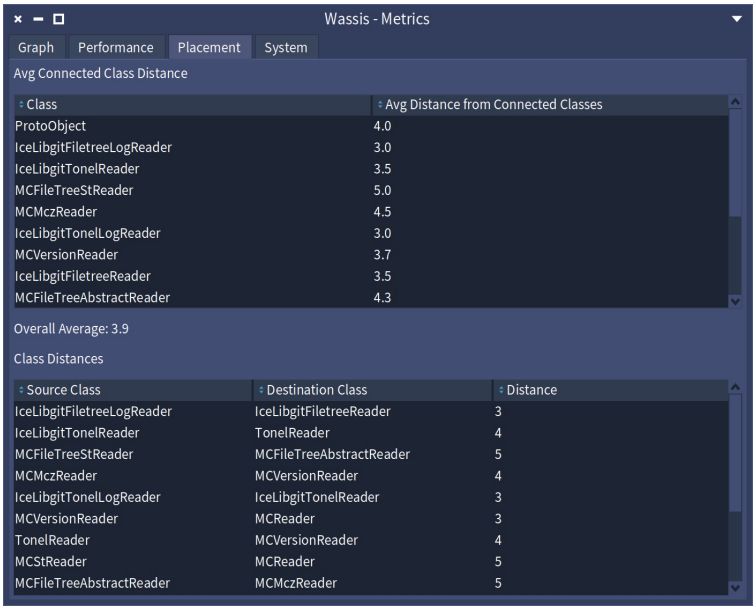
The next chapter evaluates how Wassis performs in practice. The experiments examine the generated visualizations and how configuration choices affect the ability of WFC to find valid solutions and the time required to do so.



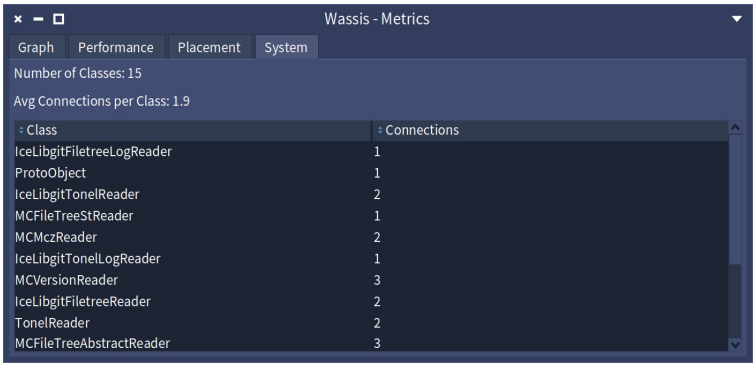
(a) Graph metrics panel.



(b) Performance metrics panel.



(c) Placement metrics panel.



(d) System metrics panel.

Figure 5.13: Overview of the tool metrics panels.

Chapter 6

Experiments and Results

In this chapter, we explore different aspects of Wassis through a series of experiments. We first compare the visualizations produced by Wassis and PlantUML. We then study how different configuration choices affect the path generation, including minimum class distance, path boost, heap type, and backtracking strategies. Finally, we analyze path generation during the WFC generation process. All experiments were performed on a Linux server equipped with two Intel Xeon E5-2695 v3 CPUs, providing 56 logical threads in total, and 384 GB of RAM. The server runs Ubuntu 24.04 LTS.

6.1 Class Hierarchy: Wassis vs. PlantUML

To evaluate Wassis, we visualize the complete inheritance hierarchy of the `Magnitude` class in Pharo 13. The hierarchy contains 41 classes at different levels. We compare the result produced by Wassis with the diagram generated by PlantUML [42], as shown in Figure 6.1. Specifically, Figure 6.1a displays the traditional layout produced by PlantUML, while Figure 6.1b shows the output generated by Wassis.

6.1.1 Layout

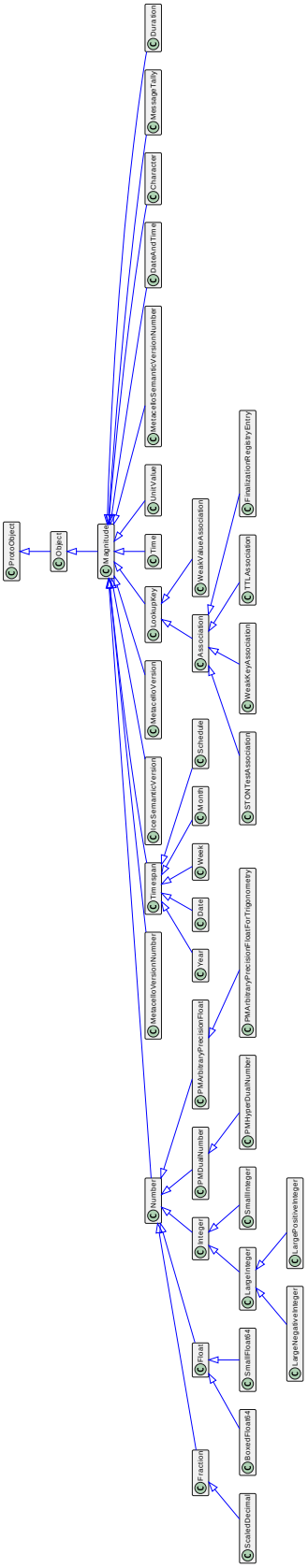
The two tools organize the hierarchy in different ways. Wassis produces a compact two-dimensional layout, where classes can be placed in all directions. The root class, `Magnitude`, is placed at the center, while the other classes are distributed around it, creating a star structure. The use of MDS keeps related classes close to each other, while the grid allows the hierarchy to use the available space in all directions.

This layout makes the complete hierarchy fit in a relatively small area, making it possible to see the whole structure at a glance.

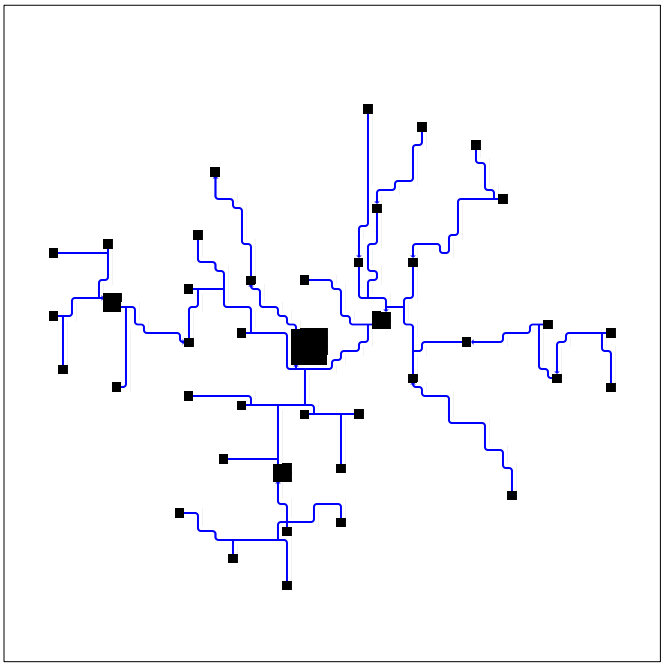
PlantUML uses a hierarchical layout, where classes are organized into horizontal levels. Classes at the same hierarchy depth are placed on the same level, so the hierarchy grows from top to bottom and in width.

In this example, the large number of subclasses of `Magnitude` makes the first level very wide. If the diagram were kept in its original orientation, it would need to be scaled to fit the available width, making it difficult to read. It is therefore rotated by 90 degrees in Figure 6.1a. For larger hierarchies, horizontal scrolling is required to inspect the complete diagram. As a result, the complete structure cannot be viewed at once without scrolling, unlike in Wassis, where the compact layout allows the whole hierarchy to be seen at a glance.

PlantUML nevertheless has an important advantage: classes at the same hierarchy depth are placed on the same level. This makes the depth of the hierarchy immediately visible and makes it easy to follow the structure from one level to the next. In Wassis, classes are positioned according to the structure of the graph rather than their hierarchy depth. Therefore, classes at the same hierarchy level are not necessarily placed at the same distance from the root, making this information less direct.



(a) PlantUML `Magnitude` hierarchy.



(b) Wassis `Magnitude` hierarchy.

Figure 6.1: Comparison of the `Magnitude` class hierarchy in PlantUML (a) and Wassis (b).

6.1.2 Path Representation

The two visualizations also differ in how relationships between classes are represented. Wassis uses path merging to combine paths that have the same target, reducing the number of paths and keeping the visualization cleaner.

In the shown example, the paths generated by Wassis do not overlap. This makes the relationships easier to follow, even in areas with many connections. Path merging is most useful when several relationships share the same target, as it avoids drawing redundant path segments.

PlantUML, on the other hand, generates a separate arrow for each relationship. When several relationships connect to the same class, the arrows can overlap or become very close to each other. In the shown example, several arrows overlap, making it difficult to follow them individually.

Path merging therefore helps Wassis keep the visualization clean. In PlantUML, representing each relationship with a separate arrow makes all connections explicit, but can also lead to visual clutter when many relationships are present.

6.1.3 Class Identification

A clear advantage of PlantUML is that class names are displayed in the diagram, allowing the user to identify a class directly.

In contrast, Wassis intentionally omits class names on the main canvas to prevent text label collisions. Class information can instead be accessed on demand through interactive features such as hover tooltips.

This is an important trade-off. PlantUML provides more textual information directly on the canvas, while Wassis gives more space to the visual structure of the system and maintains readability at scale.

6.1.4 Execution Time

The main limitation observed in this example is execution time. Generating the Wassis visualization shown in Figure 6.1b required approximately two weeks of computation. This is mainly because WFC has to find a valid arrangement of all paths.

PlantUML, in contrast, generates the corresponding diagram in a much shorter time. This makes PlantUML more suitable when a diagram has to be generated quickly.

The execution time of Wassis is therefore an important limitation for larger software systems. This cost should be considered when choosing Wassis for practical use, especially when the visualization needs to be regenerated frequently.

The main differences between Wassis and PlantUML are summarized in Table 6.1.

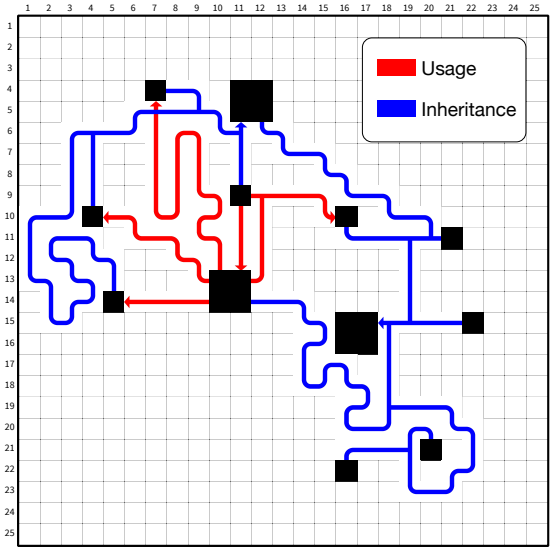
	Wassis	**PlantUML**
Layout	Compact star layout, the root is placed near the center.	Traditional hierarchical layout.
Space	Uses space in multiple directions.	May become very wide when many subclasses are present.
Paths	Paths remain separated and easier to follow, thanks to path merging.	Several arrows can become close or overlap with others.
Hierarchy	Less explicit because classes are positioned by graph distances	More explicit because classes are organized by hierarchy level.
Class Names	Not shown on visualization.	Shown directly on each class.
Time	Took approximately two weeks for this example.	Milliseconds.

Table 6.1: Comparison between Wassis and PlantUML for the `Magnitude` hierarchy.

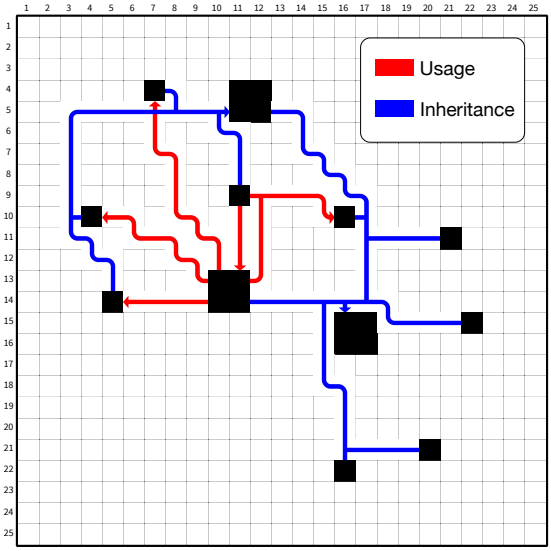
6.2 Multi-Relationship: Wassis vs. PlantUML

To show how Wassis handles multi-relationship visualization, we used a subset of 12 classes from the `org.jetuml.diagram` package. The system contains two types of relationship: `Inheritance` and `Usage`. In Wassis, each relationship is represented with a different color. In this example, we use blue for `Inheritance` and red for `Usage`.

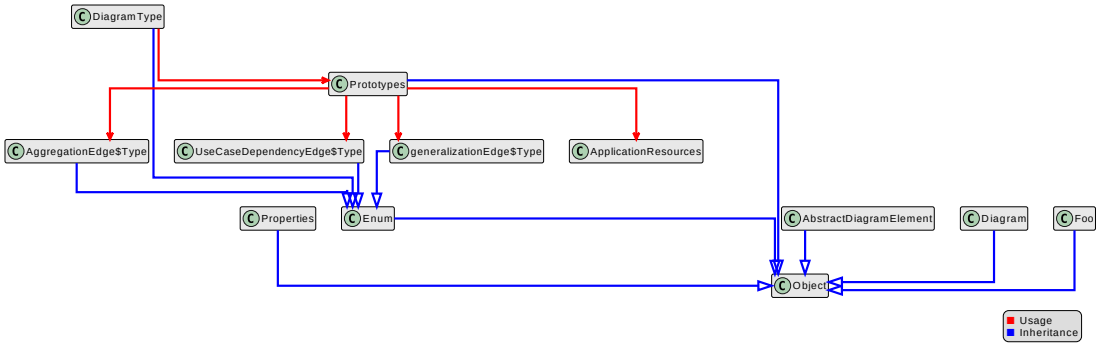
We compare the layout generated by Wassis with the layout generated by PlantUML. For Wassis, we considered both the pure WFC result and the result after path optimization, as shown in Figure 6.2.



(a) Wassis (WFC-pure).



(b) Wassis (after path optimization).



(c) PlantUML.

Figure 6.2: Comparison of Wassis and PlantUML for multi-relationship visualization.

6.2.1 Path Crossings and Visual Clutter

Both Wassis and PlantUML have one crossing in this example. In Wassis, the crossing is between a red `Usage` path and a blue `Inheritance` path. This crossing is caused by the position of the classes and could have been avoided with a different class placement.

The main difference is the amount of visual clutter. PlantUML draws a separate arrow for each relationship. This creates many lines around classes such as `Object` and `Enum`. In Wassis, paths of the same type are merged before reaching the target class. This reduces the overall number of lines in the canvas, making the visualization cleaner.

6.2.2 Execution Statistics

PlantUML generates the layout almost immediately, while Wassis needed more time to find a valid WFC solution on the 25×25 grid. The execution metrics are shown in Table 6.2.

Metric	Value
Classes	12
Paths	7
Grid Size	25×25
Path Crossings	1
Retries	11,282
Execution Time	17m 15s

Table 6.2: Wassis execution metrics.

Wassis found a valid solution after 17 minutes and 15 seconds and 11,282 retries.

6.2.3 Summary

The main differences between Wassis and PlantUML are summarized in Table 6.3.

	Wassis	PlantUML
Crossings	1	1
Visual Clutter	Lower, due to path merging.	Higher, due to separate arrows.
Layout	Star layout.	Hierarchical layout.
Execution Time	17m 15s.	Milliseconds.

Table 6.3: Comparison of Wassis and PlantUML.

6.3 Minimum Distance Between Classes Comparison

This experiment evaluates how the minimum class distance affects the execution time and success rate of WFC. The distance is measured as the Manhattan distance between class nodes on the grid. We used a subset of 9 classes from `JetUML` (shown in Figure 6.3) and tested four minimum distances: 2, 3, 5, and 7 cells. For each distance, 24 solver instances were executed with a time limit of 24 hours.

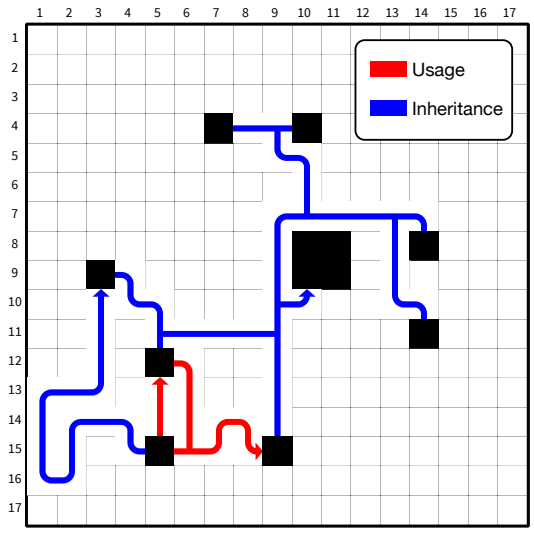


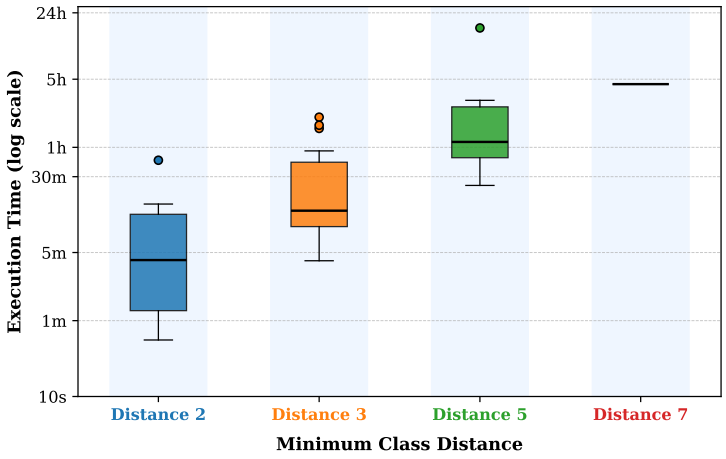
Figure 6.3: Subset of 9 classes from the `org.jetuml.diagram` package.

We expect that a larger minimum distance between classes requires a larger initial grid and longer paths between connected classes. This should increase the search space and make the generation more difficult.

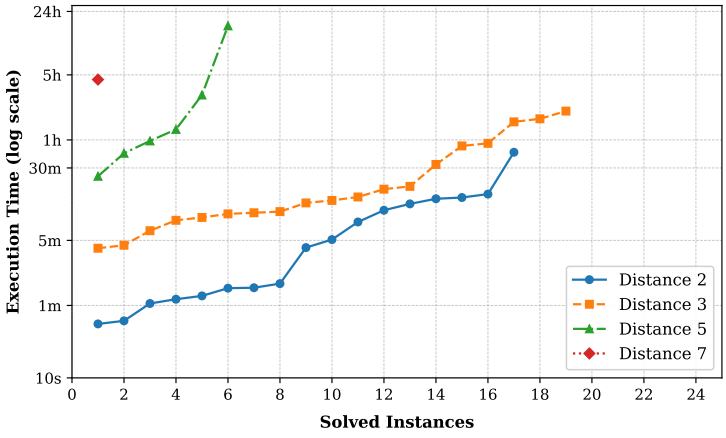
The results are shown in Figure 6.4. The boxplots in Figure 6.4a clearly move towards higher execution times as the minimum distance increases. This shows that larger distances generally require more time to find a solution. Additionally, the cactus plot in Figure 6.4b displays the number of solved instances relative to execution time.

- **Distance 2:** The average initial grid size is 17×17. 17 out of 24 instances were solved (70.8%), with execution times between 38 seconds and approximately 44 minutes.
- **Distance 3:** The average initial grid size is 22×22. 19 out of 24 instances were solved (79.2%), with execution times between 4 minutes and approximately 2 hours.
- **Distance 5:** The average initial grid size is 29.71×29.71. 6 out of 24 instances were solved (25.0%), with execution times between 24 minutes and nearly 17 hours.
- **Distance 7:** The average initial grid size is 40.75×40.75. Only 1 out of 24 instances was solved (4.2%), taking nearly 5 hours.

The results confirm this expectation. As the minimum distance increases, the initial grid becomes larger and the execution times generally increase. At the same time, the number of successful instances decreases.



(a) Execution times boxplot.



(b) Execution times cactus plot.

Figure 6.4: Execution times for different minimum class distances.

6.4 Impact of Path Boost

This experiment evaluates the effect of path boost on generating valid solutions. We used the complete class hierarchy of the Pharo `Slot` class, which contains 30 classes.

We ran 48 instances over a period of four days. Half of the instances were run without path boost, while the other half used a path boost of 50. All other configuration parameters were kept the same. For both configurations, the 24 instances were divided into three groups of 8, with a minimum class distance of 1, 2, and 3.

The results are summarized in Table 6.4. Without path boost, no valid solution was found in any of the 24 instances. WFC performed a total of 2,332,200 attempts: 804,250 with distance 1, 808,700 with distance 2, and 719,250 with distance 3. Despite the large number of attempts, none produced a valid solution.

Path Boost	Min Distance	Instances	Solutions	Success Rate
No	1	8	0	0% (0/8)
No	2	8	0	0% (0/8)
No	3	8	0	0% (0/8)
50	1	8	1	12.5% (1/8)
50	2	8	4	50.0% (4/8)
50	3	8	2	25.0% (2/8)

Table 6.4: Results obtained with and without path boost for each minimum class distance.

Without path boost, paths are not guided toward their target classes. The fixed exit directions only give them an initial direction, after which they are generated completely randomly based on the weighted choice. Therefore, the paths must reach their targets by chance. When many paths are generated at the same time, it is very unlikely that all of them will meet the correct targets.

With path boost enabled, 7 valid solutions were found. For distance 1, one solution was found after 23 hours, 42 minutes, and 44 seconds. For distance 2, four solutions were found after 37.43 seconds, 32 minutes and 47 seconds, 10 hours, 25 minutes and 30 seconds, and 18 hours and 14 minutes. For distance 3, two solutions were found after 35 minutes and 36 seconds, and 1 hour, 14 minutes and 33 seconds. Execution times are shown in Figure 6.5.

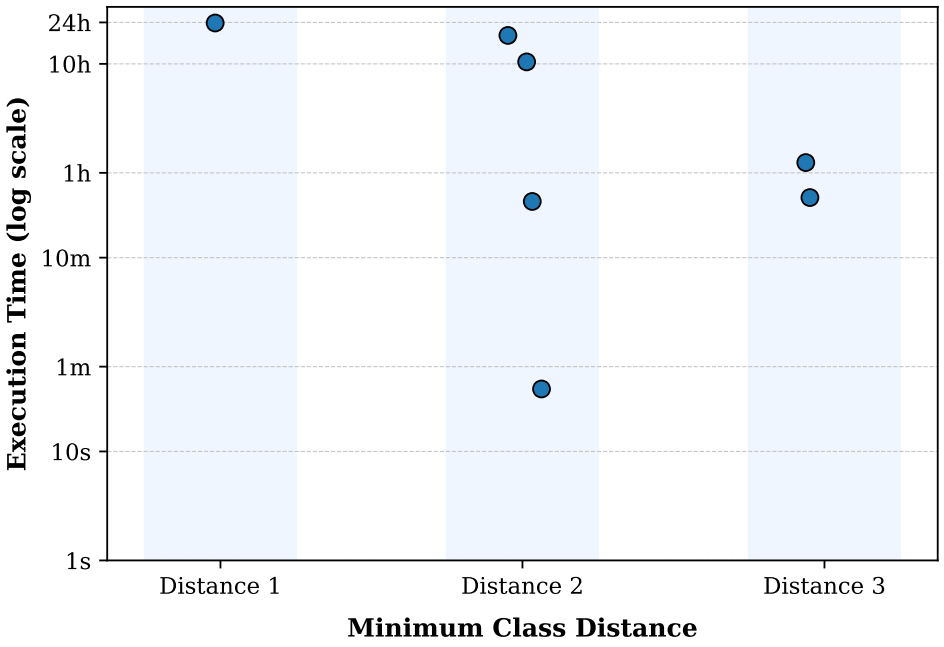


Figure 6.5: Execution times grouped by minimum class distance (with path boost).

The comparison shows that path boost has a strong effect on the ability of WFC to find valid solutions. It is also interesting to note that after 1 day no other solutions were found, suggesting that when the grid becomes too large it becomes more difficult for WFC to find a valid solution. This may be correlated with what was shown in Section 6.3.

Figure 6.6 shows the solution found after 37.43 seconds with a minimum class distance of 2.

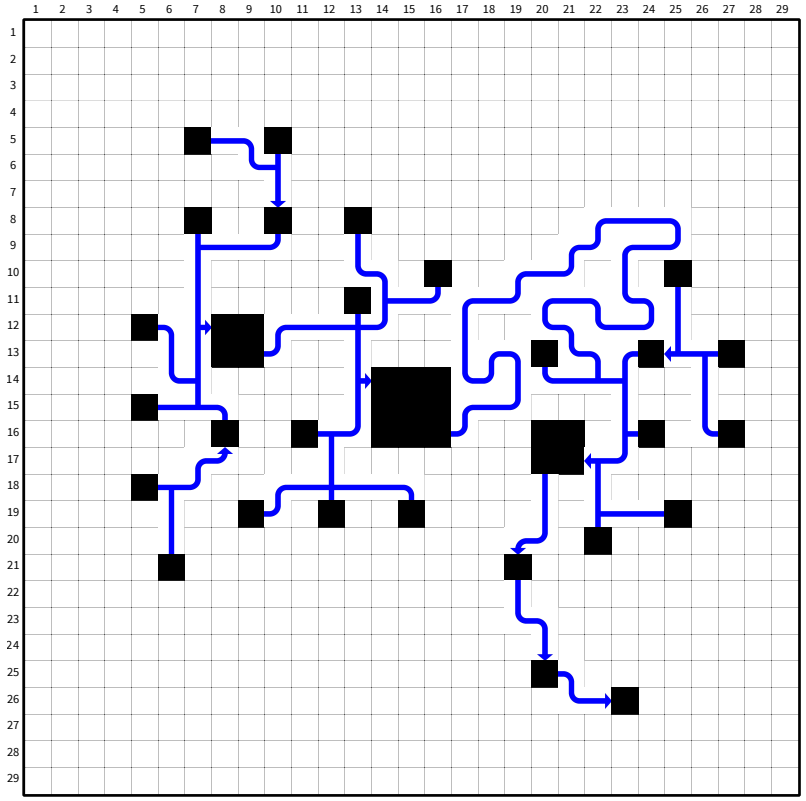


Figure 6.6: `Slot` hierarchy generated with path boost 50 and minimum distance 2 (37.43s).

6.5 Min-Heap Behavior

In the previous sections, we explained why we use a **max-heap** during the WFC observation phase. Empty tiles have higher weights than path tiles. So, as path cells have a higher entropy, with a max-heap they are selected first. This allows paths to be generated before the empty tiles occupy the whole grid, removing space for paths.

Standard WFC implementations usually use a **min-heap**, which selects the cells with the lowest entropy first. In our case, this means that empty tiles are selected before path tiles. This creates an interesting behavior: sometimes, empty tiles guide paths instead of blocking them.

6.5.1 How Min-Heap Can Guide Paths

With a min-heap, empty tiles quickly fill the majority of the grid. As more cells become empty, the available space for paths becomes smaller.

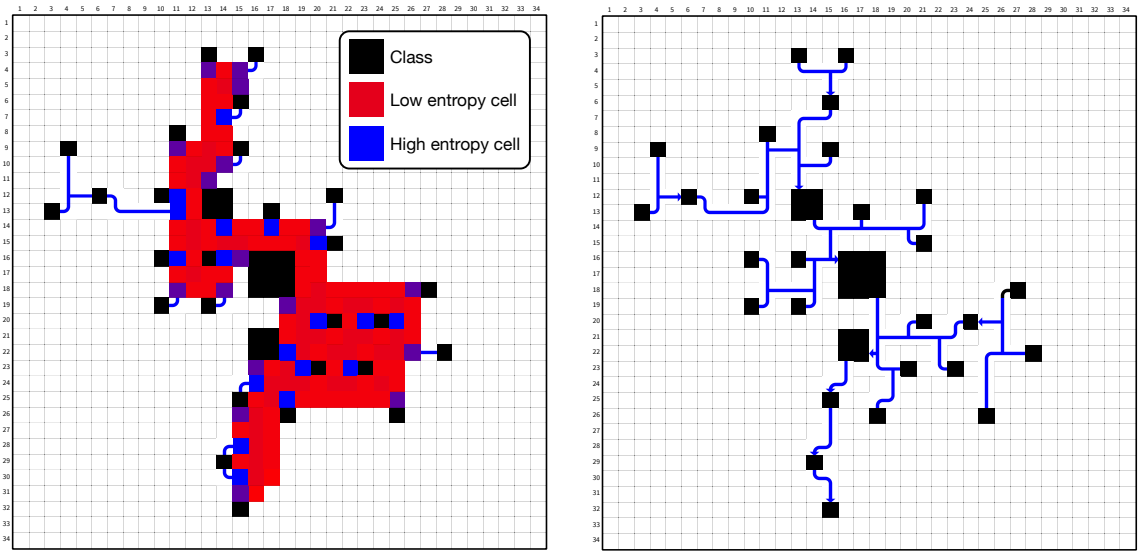
The remaining uncollapsed cells form narrow areas between the classes. Since the paths still need to connect the classes, they are forced to use these remaining areas. In this way, the empty tiles naturally guide the paths through the grid.

This behavior allows min-heap to succeed in generating a solution even without path boost, which we showed is really important in Section 6.4.

6.5.2 Example: Slot Hierarchy

To show this behavior, we use the complete class hierarchy of the `Slot` class in Pharo 13. The system contains 30 classes, which were placed on a 34×34 grid.

The generation took 14 minutes and 54 seconds and required 1,139 retries. Figure 6.7 shows an intermediate step and the final result.



(a) Step 1006 / 1156 (87% complete). (b) Final visualization (Step 1156).

Figure 6.7: Min-heap generation process.

In Figure 6.7a, most of the grid has already been filled with empty tiles, shown as white areas. This leaves only uncollapsed cells between the classes. These remaining cells form narrow corridors that guide the paths. Figure 6.7b shows the result, where the paths have followed these remaining areas and successfully connected all classes.

6.6 Evaluation of Backtracking Strategies

In this experiment, we compare five backtracking strategies for handling contradictions during WFC path generation. We used the complete class hierarchy of the Pharo 13 `HashedCollection` class (Figure 6.8).

For each of the 5 strategies, we ran 24 instances for 4 hours. All other configuration parameters were kept the same. In total, 120 instances were executed, and the initial grid had an average size of 31.12×31.12 cells.

During each generation, the solver allowed up to 5 backtracks on the same grid. If no solution was found, it discarded the current class placement and started again with the same grid size.

The five strategies were:

- **Fixed Number of Steps:** Goes back a fixed number of collapse steps. We used 50 steps.
- **Percentage of Steps:** Goes back by a fixed percentage of the collapse steps. We used 20%.
- **Many Possible Tiles:** Goes back to a checkpoint saved before a cell with many possible tiles was collapsed. The minimum number of tiles was 7.
- **Similar Tile Weights:** Goes back to a checkpoint saved before a cell with similar tile weights was collapsed. This was triggered when at least 30% of the candidate tiles had a maximum weight difference of 20%.
- **Restart:** Discards the current solution and starts the path generation from the beginning.

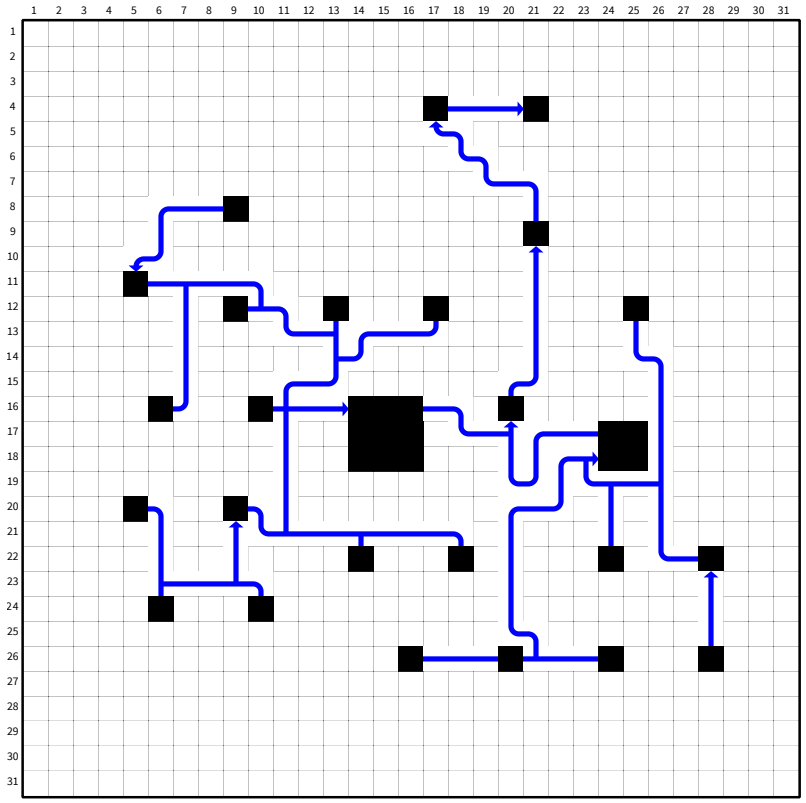


Figure 6.8: `HashedCollection` full hierarchy.

6.6.1 Experimental Results and Analysis

The results are shown in Figure 6.9. Out of 24 instances, the strategies solved:

- **Fixed Number of Steps:** 17 instances (70.8%).
- **Percentage of Steps:** 19 instances (79.2%).
- **Many Possible Tiles:** 16 instances (66.7%).
- **Similar Tile Weights:** 20 instances (83.3%).
- **Restart:** 21 instances (87.5%).

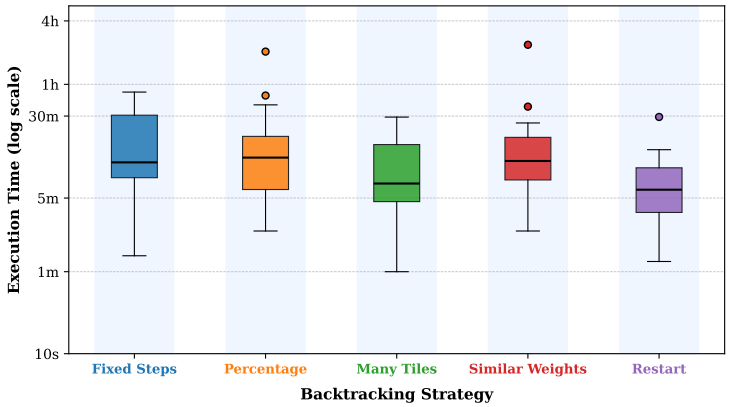
The boxplots in Figure 6.9a show the execution times, while the cactus plot in Figure 6.9b shows how many instances were solved over time.

The results are similar for the five strategies, but the **Restart** strategy solved the most instances and generally found solutions faster.

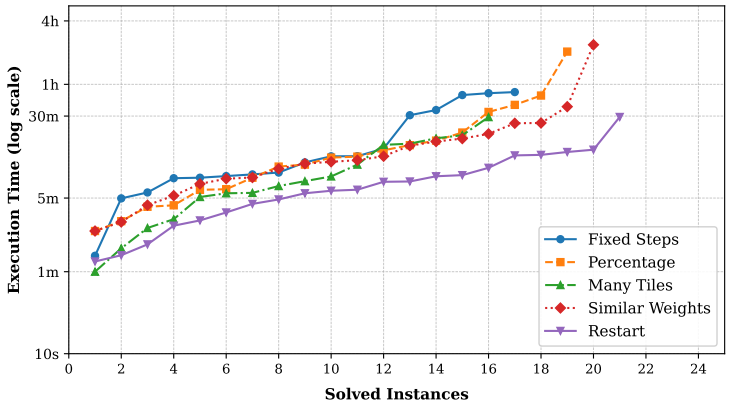
One possible reason is the small size of the tested system. For a class hierarchy such as `HashedCollection`, a contradiction can be caused by an early class placement that makes the rest of the grid difficult to solve. In this case, going back step by step may spend time trying to fix a layout that is already difficult to solve.

With **Restart**, the solver can discard this layout and try a new class placement immediately. For this relatively small system, starting again has a low cost.

For larger and more complex systems, this may change [22]. In such cases, restarting from the beginning can waste more work, while backtracking strategies that keep part of the current solution may become more useful.



(a) Execution times.



(b) Solved instances over time.

Figure 6.9: Results for the five backtracking strategies.

6.7 WFC Generation Analysis

In this section, we analyze when the path generation is completed during the WFC process. As already explained in Subsection 4.4.1 and shown in Figure 4.10, the max-heap makes WFC select path cells before empty cells. Therefore, when the first empty tile is placed, all paths are already generated. The remaining steps only fill the grid with empty tiles and close the paths, but the tile has already been assigned (only remaining possibility).

To study this behavior, we define the **Path Completion Point** (PCP). It represents the percentage of the observation phase completed when the first empty tile is placed. More precisely, it is computed as:

$$\frac{\text{index of the first empty tile} - 1}{\text{total number of nodes}} \times 100$$

The subtraction of one gives the percentage of steps completed before the first empty tile is placed. Since WFC places one tile at each step, the total number of observation steps is equal to the number of grid cells.

Figure 6.10 shows the PCP for 193 tested instances. The histogram is strongly shifted towards low values, with an average PCP of around 20%. This means that, on average, the path generation is completed after only 20% of the observation phase. Therefore, more than 80% of the observation phase on average is spent filling the grid with empty tiles and closing the paths.

This information can also be used for backtracking. If a generation reaches more than 30%-40% of the observation steps without placing an empty tile, it can be stopped early, since it is unlikely to find a valid solution.

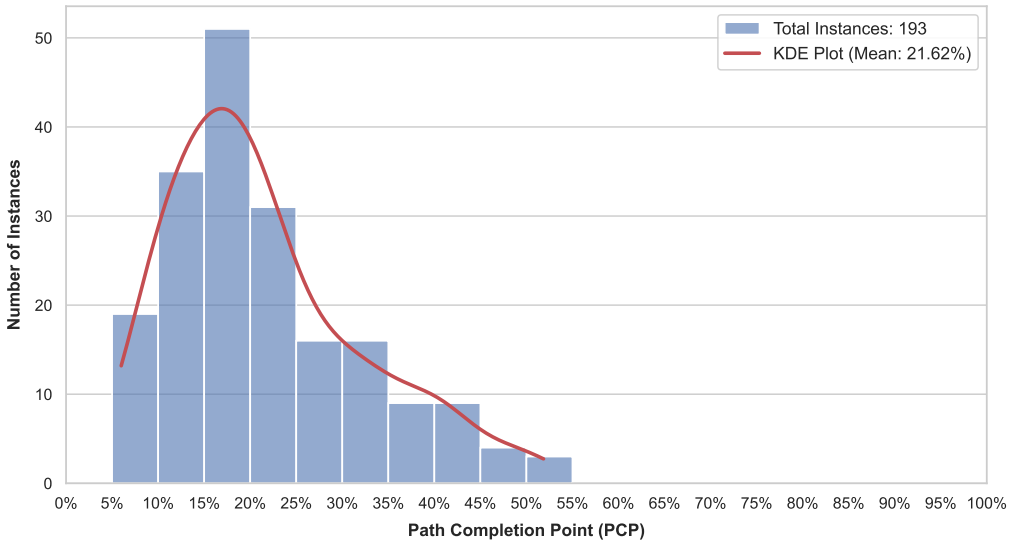


Figure 6.10: PCP distribution for 193 instances.

Figure 6.11 shows this behavior in more detail. The instances are grouped by their PCP. Each stacked bar shows the average number of empty, path, and class nodes. The blue line shows the ratio between path nodes and all the available space for paths:

$$\frac{\text{path nodes}}{\text{total nodes} - \text{class nodes}}$$

This ratio measures how much of the available space is occupied by paths.

As expected, this ratio increases when the PCP increases. However, we can also see that the total number of nodes decreases. This means that solutions with a higher PCP have smaller grids.

These solutions are less efficient in terms of path space. In the last groups, more than 60% of the available space is occupied by path nodes.

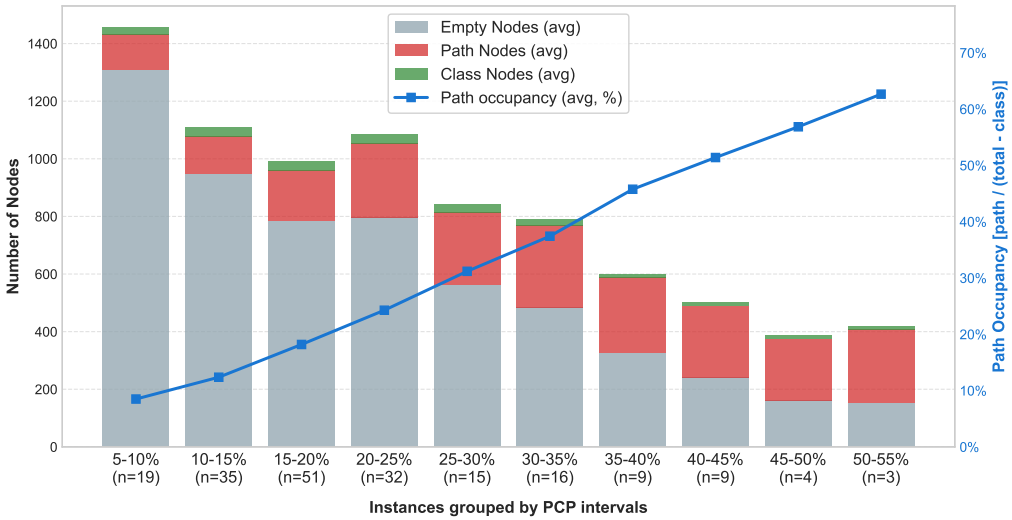
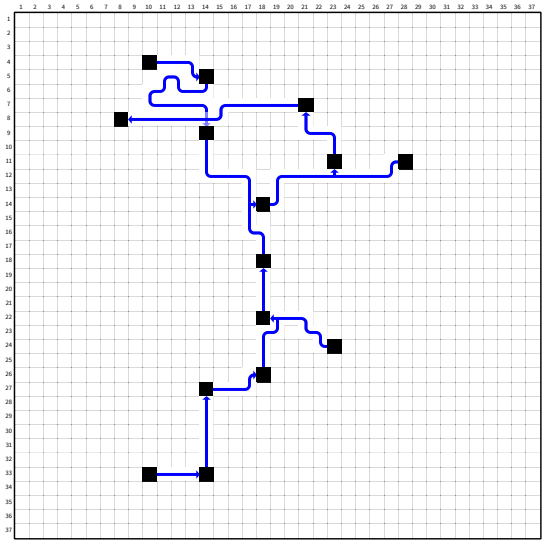


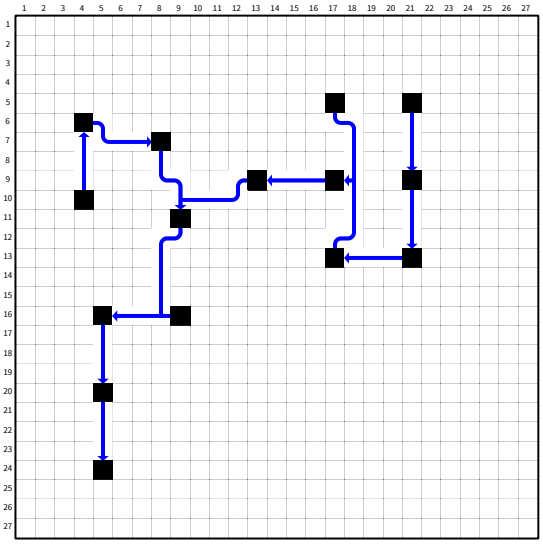
Figure 6.11: Average node counts and path space occupancy for different PCP values.

To better show this difference, Figure 6.12 compares four solutions generated with pure WFC. The first two have the lowest PCP, 5.99% and 7.41%, with shorter, more direct paths. The last two have the highest PCP, 50.52% and 50.62%, with longer, more complex paths.

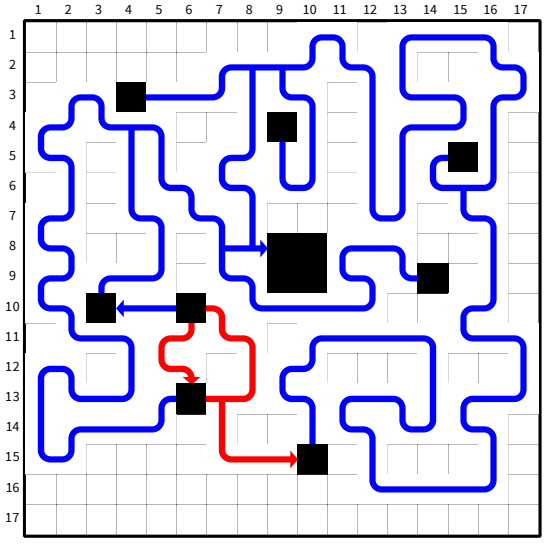
All four solutions were generated using pure WFC. These examples show that the PCP can also give an indication of path quality. A lower value generally corresponds to a more efficient use of the available space.



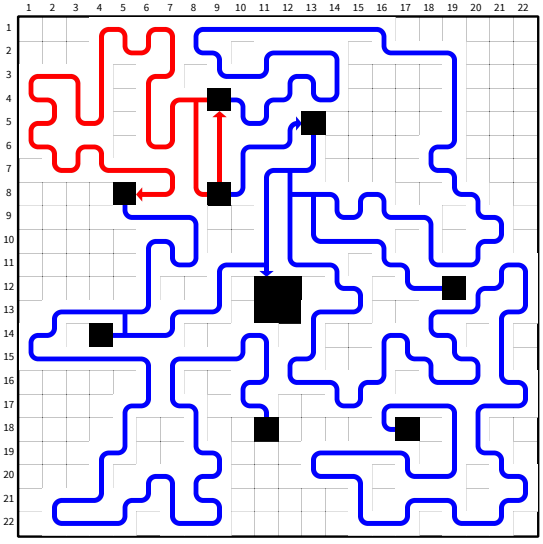
(a) First of 193 instances, PCP = 5.99%.



(b) Second of 193 instances, PCP = 7.41%.



(c) Next to last of 193 instances, PCP = 50.52%.



(d) Last of 193 instances, PCP = 50.62%.

Figure 6.12: Examples of solutions with low and high PCP values.

6.7.1 PCP versus Execution Time

Since a lower PCP corresponds to fewer path tiles that needed to be generated, we expect that a lower PCP also leads to a shorter execution time. Figure 6.13 compares PCP and execution time for the 193 tested instances.

As shown in the scatter plot, the points are widely scattered, showing no clear relationship between the two metrics. For every PCP value, the execution time varies significantly.

The absence of a clear correlation is further confirmed by the trend line, which is almost flat. The Spearman correlation is also low ($\rho = 0.23$), indicating a very weak positive correlation, although it is statistically significant ($p = 0.001$). Therefore, a lower PCP does not guarantee a faster generation time.

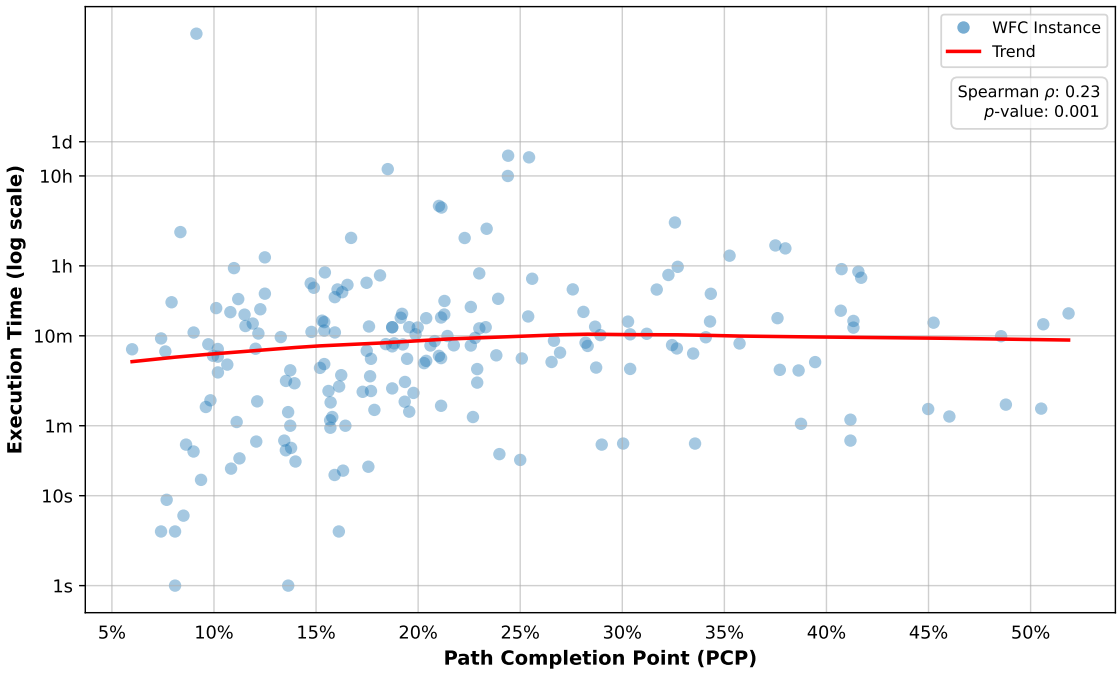


Figure 6.13: PCP over execution time for 193 WFC instances.

6.8 Summary

This chapter evaluated Wassis through experiments on software class hierarchies and systems with multiple types of relationships. The results show that Wassis can produce compact visualizations with less visual clutter through path merging, although class names are not displayed directly and generation can be computationally expensive.

The experiments also showed that the ability of WFC to find valid solutions depends strongly on the configuration. Increasing the minimum distance between classes makes the problem harder to solve, while path boost helps guide paths towards their targets. The choice of heap and backtracking strategy can also affect the generation process. Finally, the Path Completion Point (PCP) provides insight into how paths are generated and how efficiently they use the available grid space, although it does not reliably predict execution time.

Overall, the experiments demonstrate the potential of Wassis for producing compact software visualizations, while highlighting execution time and scalability as important limitations. These findings provide a basis for discussing the contributions of this work, its limitations, and possible directions for future research in the next chapter.

Chapter 7

Conclusion

This thesis explored the use of Wave Function Collapse for software visualization. We developed Wassis, a tool that combines Multidimensional Scaling for class placement with WFC for generating the connections between classes. We generated compact visualizations that represent software relationships while reducing path crossings and visual clutter.

The main contribution of this work is the adaptation of WFC to software visualization. Standard WFC is designed to generate locally compatible tiles, whereas software visualization requires global constraints that ensure paths connect the intended classes. To address these requirements, several changes were made to the original algorithm. The observation process was modified to control the generation of empty and path tiles by introducing a max-entropy heap. A dynamic placeholder system was developed to manage path IDs without requiring a separate tile for every possible path. Directional exits and dynamic tile weights were added to guide paths towards their targets. Finally, different backtracking strategies and a path optimization step were introduced to improve the ability of WFC to find solutions.

The experiments show that these changes allow WFC to generate software visualizations while respecting the structural relationships between classes. The results also show that the configuration settings have an important effect on the generation process. For instance, path boosting can help the paths' creation, while the choice of backtracking strategy can affect the number of successful generations. Increasing the minimum distance between classes makes the problem harder, as it requires larger grids and longer paths.

A key advantage is the way class placement and path generation work together. MDS provides initial positions for the classes, while WFC determines how to connect them on the grid. This combination allows the layout and the connections to be considered together, rather than treating them as independent steps. Path merging is a further benefit, as multiple relationships can share parts of the same path, reducing visual clutter in the visualization, compared with drawing a separate path for each relationship.

However, these advantages come with a high computational cost. Some instances require many attempts and long execution times, while others cannot be solved within the given time limits.

Overall, our results show that WFC can be used for software visualization when its original mechanisms are adapted to the specific requirements of this problem. Wassis implements this approach and demonstrates that WFC can be adapted to handle the global constraints required by software visualization. At the same time, we highlight the need for further work to improve the scalability and efficiency of our approach.

In summary, this work provides a starting point for further research on the use of tile-based constraint solvers in software visualization. The results suggest that this approach is worth exploring further, particularly to improve its overall computational efficiency and evaluate its practical applicability to larger, more complex software systems.

7.1 Limitations

The main limitations of Wassis are related to execution time, scalability, and the difficulty of finding solutions when many constraints are present.

The first limitation is the time required to generate a visualization. WFC explores different tile configurations, and the number of possible configurations increases as the grid becomes larger. When a valid solution is difficult to find, WFC may perform many collapses, propagations, and backtracking steps before finding one. As a result, even relatively small examples can require a long time to complete.

A second limitation is the difficulty of handling multiple relationship types at the same time. When several types of relationships need to be represented, more constraints have to be satisfied. Paths have fewer possible ways to reach their targets, which can lead to more contradictions and backtracking. In some cases, no valid solution is found within the configured limits. Path merging helps reduce the number of paths that need to be generated, but this reduction is not sufficient to avoid the contradictions caused by a large number of relationships.

Another limitation is the class placement process. MDS provides target positions based on the distances between classes, but these positions do not always produce a grid that is easy for WFC to solve. Wassis can increase the grid size and try different placements, but this increases the execution time and does not guarantee that a solution will be found.

Finally, the evaluation was performed on selected software systems and subsets of classes. The experiments provide useful information about the behavior of Wassis, but they do not show how it performs on much larger and denser software systems. In addition, the generated layouts were mainly evaluated using visual inspection and quantitative metrics. A user study with developers is needed to test whether the generated visualizations actually improve software comprehension.

7.2 Future Work

Several improvements could be explored to reduce generation time, handle larger and more complex software systems, and better evaluate the quality of the generated visualizations.

7.2.1 Performance and Scalability

The first set of improvements focuses on reducing the computational cost of WFC:

- **GPU-based Propagation:** The propagation phase could be improved by using a GPU to process several cells in parallel. When a tile collapses, the constraints of its neighbors could be checked simultaneously, reducing the time required to remove incompatible tiles. However, the propagation queue would need to be handled carefully. A parallel implementation would also need to ensure that changes to the same cell do not cause inconsistencies.

- **Nested WFC:** Large grids could be divided into smaller subgrids, with a separate WFC instance solving each one. This idea is inspired by Nested Wave Function Collapse [27], which applies WFC to smaller parts of a larger map. In Wassis, the main challenge would be to define the paths that cross the boundaries between subgrids. These connections would need to be assigned before or during generation so that the different subgrids remain consistent. This approach could reduce the size of each individual problem, although the division of the grid and the assignment of boundary connections would require further study.

- **Parallel Search from Saved Checkpoints:** The current backtracking strategies decide when to save and restore checkpoints using predefined rules. A possible improvement would be to estimate how promising a checkpoint is by considering factors such as the remaining uncollapsed cells, the number of unresolved paths, and the contradictions found so far. Different WFC instances could then start from the same checkpoint and explore different choices in parallel. This could increase the chances of finding a solution.

7.2.2 WFC and Layout Improvements

These improvements focus on giving WFC more flexibility during generation and improving how classes and paths are placed in the grid:

- **Improved Backtracking:** During backtracking, WFC may explore configurations that are very similar to those that have already failed. This could be detected by comparing grids using edit distance or by comparing the resulting visualizations. If several failed attempts are too similar, WFC could restart the generation with a different class placement. This could reduce the time spent exploring configurations that may lead to the same contradictions.
- **Modification of Collapsed Tiles:** In the current implementation, a collapsed tile cannot change unless the solver performs backtracking. This can cause a contradiction even when a small change to an existing path could solve the problem. For example, a path may need to connect to another path that is already represented by a straight tile. If this tile does not allow the required connection, WFC may reach a contradiction even when a crossing or a junction could provide a valid solution. Future work could allow selected collapsed tiles to be modified when new paths reach them, as long as the path IDs and adjacency constraints remain valid. This would give the solver more flexibility and could reduce unnecessary backtracking.
- **Better Tile Configuration:** The current tile weights and path boost values are configured by the user. Future work could investigate whether these values can be adjusted automatically during generation. For example, the solver could reduce the weight of tiles that are likely to create contradictions. Results from previous generations could also be used to select suitable parameters for similar software systems. This could reduce the need for manual configuration and improve the chances of finding a solution.
- **Improved Class Placement:** The current class placement is based on MDS and placement constraints. Future work could also consider path generation when selecting the final positions of classes. The solver could try several candidate placements and prefer those that provide more space for paths or reduce the number of difficult connections. This could reduce the number of failed WFC attempts and increase the chances of finding a valid solution.

7.2.3 Evaluation

Further work is also needed to evaluate the approach on larger software systems and to better measure the quality of the generated visualizations.

- **More Extensive Evaluation:** Wassis should be tested on larger software systems and with more types of relationships. Future experiments could compare different placement, tile weight, and backtracking strategies. The evaluation could also examine how the number of classes, relationships, and graph density affect execution time and the probability of finding a solution. This would help better understand when Wassis can be used effectively.
- **Automatic Evaluation of Visualization:** The current evaluation mainly uses visual inspection and metrics such as path crossings, grid size, and execution time. Future work could introduce a combined quality measure based on path length, number of bends, crossings, and distance between classes. This measure could be used to compare different valid solutions and select the best one. A user study could also investigate whether the generated layouts help developers understand software relationships more easily.

7.3 Closing Words

This thesis explored the use of Wave Function Collapse for software visualization. By combining MDS with WFC, Wassis provides a way to create clear and compact software representations.

This work opens the door to further research on procedural techniques for software visualization. Improvements in scalability and performance could make this approach more practical and help developers better understand complex software systems.

Appendix A

Shannon Entropy Derivation

In this appendix, we derive Equation (4.2) from the standard Shannon entropy formula. We also explain the computational benefits of using this formulation in Wave Function Collapse.

Computational Motivation

During the observation phase of WFC, the entropy of candidate cells is calculated repeatedly to select the next cell to collapse. In the standard formulation, Shannon entropy requires the tile weights $w_1, \ldots, w_n$ to be normalized into probabilities $p_i = \frac{w_i}{\sum_{j=1}^{n} w_j}$.

By computing Shannon entropy directly from the unnormalized weights w_i, explicit probabilities p_i do not need to be calculated.

Algebraic Derivation

Let $w_1, \ldots, w_n$ be the weights of the possible tiles of a cell. Let W be the sum of all weights:

$$W = \sum_{j=1}^{n} w_j \tag{1}$$

The probability p_i of selecting tile i is obtained by dividing its weight by the total weight W:

$$p_i = \frac{w_i}{W} \tag{2}$$

The standard Shannon entropy H is defined as:

$$H = -\sum_{i=1}^{n} p_i \ln p_i \tag{3}$$

Substituting $p_i = \frac{w_i}{W}$ gives:

$$H = -\sum_{i=1}^{n} \left(\frac{w_i}{W}\right) \ln \left(\frac{w_i}{W}\right) \tag{4}$$

Using the logarithm rule $\ln\left(\frac{a}{b}\right) = \ln a - \ln b$, we obtain:

$$H = -\sum_{i=1}^{n} \frac{w_i}{W} \left(\ln w_i - \ln W\right) = -\frac{1}{W} \sum_{i=1}^{n} w_i \ln w_i + \frac{\ln W}{W} \sum_{i=1}^{n} w_i \tag{5}$$

Since $\sum_{i=1}^{n} w_i = W$, the second term becomes:

$$\frac{\ln W}{W} \sum_{i=1}^{n} w_i = \frac{\ln W}{W} \cdot W = \ln W \tag{6}$$

Therefore, the entropy can be written as:

$$H = \ln W - \frac{\sum_{i=1}^{n} w_i \ln w_i}{W} \tag{7}$$

Finally, replacing W with $\sum_{i=1}^{n} w_i$ gives the unnormalized Shannon entropy formula:

$$H = \ln \left(\sum_{i=1}^{n} w_i \right) - \frac{\sum_{i=1}^{n} w_i \ln w_i}{\sum_{i=1}^{n} w_i} \tag{8}$$

Bibliography

[1] Thomas Ball and Stephen Eick. Software visualization in the large. *Computer*, 29(4):33–43, 1996.

[2] Peter Naur and Brian Randell, editors. *Software Engineering: Report on a conference sponsored by the NATO Science Committee, 7th to 11th October 1968*. Scientific Affairs Division, NATO, 1969.

[3] Roberto Minelli, Andrea Mocci, and Michele Lanza. I know what you did last summer: an investigation of how developers spend their time. In *Proceedings of the 23rd International Conference on Program Comprehension (ICPC 2015)*, pages 25–35. IEEE, 2015.

[4] Anneliese Von Mayrhauser and A. Marie Vans. Program comprehension during software maintenance and evolution. *Computer*, 28(8):44–55, 1995.

[5] Janet Siegmund. Program comprehension: Past, present, and future. In *Proceedings of the 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER 2016)*, pages 13–20. IEEE, 2016.

[6] Thomas D. LaToza, Gina Venolia, and Robert DeLine. Maintaining mental models: A study of developer work habits. In *Proceedings of the 28th International Conference on Software Engineering (ICSE 2006)*, pages 492–501. ACM, 2006.

[7] Olena Andriyevska, Natalia Dragan, Bonita Simoes, and Jonathan I. Maletic. Evaluating UML class diagram layout based on architectural importance. In *Proceedings of the International Workshop on Visualizing Software for Understanding and Analysis (VISSOFT 2005)*, pages 1–6. IEEE, 2005.

[8] Nour Jihene Agouf, Stéphane Ducasse, Anne Etien, and Michele Lanza. A new generation of Class Blueprint. In *Proceedings of the 10th Working Conference on Software Visualization (VISSOFT 2022)*, pages 29–39. IEEE, 2022.

[9] Richard Wettel and Michele Lanza. Visualizing software systems as cities. In *Proceedings of the International Workshop on Visualizing Software for Understanding and Analysis (VISSOFT 2007)*, pages 92–99. IEEE, 2007.

[10] Adrian Kuhn, David Erni, Peter Loretan, and Oscar Nierstrasz. Software cartography: Thematic software visualization with consistent layout. *Journal of Software Maintenance and Evolution: Research and Practice*, 22(3):191–210, 2010.

[11] Warren S. Torgerson. Multidimensional scaling: I. Theory and method. *Psychometrika*, 17(4):401–419, 1952.

[12] J. Douglas Carroll and Phipps Arabie. Multidimensional scaling. In *Measurement, Judgment and Decision Making*, pages 179–250. Academic Press, 1998.

[13] Giuseppe Di Battista, Peter Eades, Roberto Tamassia, and Ioannis G. Tollis. *Graph Drawing: Algorithms for the Visualization of Graphs*. Prentice Hall, 1998.

[14] Thomas M. J. Fruchterman and Edward M. Reingold. Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11):1129–1164, 1991.

[15] Roberto Tamassia. On embedding a graph in the grid with the minimum number of bends. *SIAM Journal on Computing*, 16(3):421–444, 1987.

[16] Danny Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006.

[17] Michele Lanza and Stéphane Ducasse. Polymetric views—a lightweight visual approach to reverse engineering. *IEEE Transactions on Software Engineering*, 29(9):782–795, 2003.

[18] Pierre Caserta, Olivier Zendra, and Damien Bodénès. 3D hierarchical edge bundles to visualize relations in a software city metaphor. In *Proceedings of the 6th International Workshop on Visualizing Software for Understanding and Analysis (VISSOFT 2011)*, pages 1–8. IEEE, 2011.

[19] David Moreno-Lumbreras, Roberto Minelli, Andrea Villaverde, Jesús M. González-Barahona, and Michele Lanza. CodeCity: A comparison of on-screen and virtual reality. *Information and Software Technology*, 153:107064, 2023.

[20] Adrian Hoff, Christoph Seidl, and Michele Lanza. Immersive software archaeology: Exploring software architecture and design in virtual reality. In *Proceedings of the International Conference on Software Analysis, Evolution and Reengineering (SANER 2024)*, pages 47–51. IEEE, 2024.

[21] Mattia Giannaccari, Marco Raglianti, and Michele Lanza. Manipulating VR-native user interfaces for software visualization customization. In *Proceedings of the 12th Working Conference on Software Visualization (VISSOFT 2024)*, pages 111–115. IEEE, 2024.

[22] Isaac Karth and Adam M. Smith. WaveFunctionCollapse: Content generation via constraint solving and machine learning. *IEEE Transactions on Games*, 14(3):364–376, 2022.

[23] Arunpreet Sandhu, Zeyuan Chen, and Joshua McCoy. Enhancing wave function collapse with design-level constraints. In *Proceedings of the 14th International Conference on the Foundations of Digital Games (FDG 2019)*, pages 17:1–17:9. ACM, 2019.

[24] Thijmen Stefanus Leendert Langendam and Rafael Bidarra. miWFC—designer empowerment through mixed-initiative wave function collapse. In *Proceedings of the 17th International Conference on the Foundations of Digital Games (FDG 2022)*, pages 66:1–66:8. ACM, 2022.

[25] Hwanhee Kim, Seongtaek Lee, Hyundong Lee, Teasung Hahn, and Shinjin Kang. Automatic generation of game content using a graph-based wave function collapse algorithm. In *Proceedings of the Conference on Games (CoG 2019)*, pages 1–4. IEEE, 2019.

[26] Hwanhee Kim, Beomjoo Seo, and Shinjin Kang. Puzzle-level generation with simple-tiled and graph-based wave function collapse algorithms. *IEEE Transactions on Games*, 17(1):52–61, 2025.

[27] Yuhe Nie, Shaoming Zheng, Zhan Zhuang, and Julian Togelius. Nested wave function collapse enables large-scale content generation. *IEEE Transactions on Games*, 16(4):892–902, 2024.

[28] Rolf Piepenbrink and Rafael Bidarra. Non-uniform tile wave function collapse. In *Proceedings of the Conference on Games (CoG 2025)*, pages 1–8. IEEE, 2025.

[29] Robert W. Floyd. Algorithm 97: Shortest path. *Communications of the ACM*, 5(6):345, 1962.

[30] Andrew Black, Stéphane Ducasse, Oscar Nierstrasz, Damien Pollet, Damien Cassou, and Marcus Denker. *Pharo by Example*. Square Bracket Associates, 2009.

[31] Oscar Nierstrasz, Stéphane Ducasse, and Tudor Gîrba. The story of Moose: An agile reengineering environment. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE 2005)*, pages 1–10. ACM, 2005.

[32] Alexandre Bergel. *Agile Visualization with Pharo: Crafting Interactive Visual Support Using Roassal*. Apress, 2022.

[33] Mattia Giannaccari, Marco Raglianti, and Michele Lanza. ZION: System exploration with class contours. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE 2026)*, pages 162–166. ACM, 2026.

Online Sources

[34] Maxim Gumin. WaveFunctionCollapse (WFC) algorithm. https://github.com/mxgmn/WaveFunctionCollapse, 2016.

[35] Joseph Parker and Ryan Jones. Proc Skater 2016. https://arcadia-clojure.itch.io/proc-skater-2016, 2016.

[36] Oskar Stålberg. Bad North. https://www.badnorth.com/, 2018.

[37] Oskar Stålberg. Townscaper. https://oskarstalberg.com/Townscaper, 2020.

[38] Marian Kleineberg. Infinite City. https://marian42.itch.io/wfc, 2018.

[39] Martin O'Leary. Poetry generator. https://x.com/mewo2/status/789167437518217216, 2016.

[40] Oege de Moor. CodeQL. https://codeql.github.com/, 2023.

[41] Max Leske. Fuel. https://github.com/theseion/Fuel, 2018.

[42] Arnaud Roques and PlantUML Contributors. PlantUML Software. https://github.com/plantuml/plantuml.